DATA STRUCTURES

MSE Master Study Guide

Module 1: Stacks and Queues | Module 2: Linked Lists

Course Data Structures (CMCOR1PC203)

Semester III, Computer Engineering, SAKEC (as printed on the lecture slides)

Language C (the language used in every program in the notes)

Prepared from the "Data Structure/Notes" folder (Module 1 and Module 2)

Companion files: DSA_MSE_Practice_Questions.docx and DSA_MSE_Quick_Revision.docx

About This Guide

How to Use This Guide

1. Chapters follow the order of the notes: fundamentals first, then stacks, queues, pointers, dynamic memory, and linked lists.
2. Each chapter ends with Quick Revision, Important Definitions, a Complexity Summary, Common Mistakes and Practice Questions with answers.
3. Programs are complete, standard C. Every program in this guide was compiled with gcc (-Wall -Wextra -std=c99) and run with test input under AddressSanitizer (a memory-error checker). Sample outputs shown are real outputs from those runs (menu prompts trimmed for space).
4. Labels used throughout:

```
   [SOURCE]      taken from your notes / slides / programs
   [SUPPLEMENT]  standard DSA knowledge added because the notes were
                 blank, incomplete or only had a title for that part
   [CORRECTION]  the notes contain a mistake; the fix is explained
Practice questions are tagged [S] (based directly on the notes) or
[A] (additional practice). No question is claimed to be "sure to come".
```

5. Turbo C vs standard C: your notes use void main(), <conio.h>, clrscr(), getch() and gets(). These work only in Turbo C. This guide uses int main(void) and no conio.h so the code runs on any modern compiler. In a Turbo C lab you may add #include<conio.h>, clrscr() and getch() back; the logic is unchanged. Avoid gets(): it was removed from C11 because it cannot stop a buffer overflow; use fgets() or scanf("%s").

Notation

```
O(1)   constant time     O(n)   linear time     O(n^2) quadratic time
n      number of elements in the structure
->     in diagrams, "points to";  X marks a NULL pointer
TOP, FRONT, REAR, START, HEAD are written in capitals in algorithms and
in lower case in C code (top, front, rear, start, head).
```

Table of Contents

Chapter 1: Introduction to Data Structures 7
1.1 Why Learn Data Structures 7
1.2 Definition of Data Structure 7
1.3 Classification of Data Structures 8
1.4 Brief Description of Each Structure 9
1.5 Areas Where Data Structures Are Used 10
1.6 Operations on Data Structures 10
1.7 Data Type 10
1.8 Abstract Data Type (ADT) 11
1.9 Quiz Questions From the Notes [SOURCE] 12
1.10 Chapter 1 Exam Preparation 12
Chapter 2: Stack 14
2.1 Linear Lists: General and Restricted 14
2.2 Definition and Concept 14
2.3 Stack ADT (Operations) 14
2.4 Push, Pop and Peek Illustrated 15
2.5 Implementation Choices 15
2.6 Array Representation of Stack 15
2.7 Algorithms for Stack Operations (Array) 16
2.8 Program: Stack Using Array 17
2.9 Linked Representation of Stack (Linked Stack) 20
2.10 Program: Stack Using Linked List 21
2.11 Array Stack vs Linked Stack 23
2.12 Chapter 2 Exam Preparation 23
Chapter 3: Applications of Stack 25
3.1 List of Applications 25
3.2 Notations of Arithmetic Expressions 25
3.3 Manual Conversion (Bracketing Method) 25
3.4 Algorithm: Infix to Postfix Using Stack 26
3.5 Worked Examples (Dry Runs) 27
3.6 Program: Infix to Postfix (INFIXPOSTFIX.C, corrected) 28
3.7 Infix to Prefix Using Stack 30
3.8 Evaluation of Postfix Expression 31
3.9 Program: Postfix Evaluation (EVALPOST.C, corrected) 33
3.10 Parenthesis Checker (Balanced Parentheses) 34
3.11 System Stack in Function Calls 36
3.12 Recursion 37
3.13 Reversal of Data [SUPPLEMENT] 38
3.14 Chapter 3 Exam Preparation 38
Chapter 4: Queue 41
4.1 Definition and Concept 41
4.2 Queue ADT 41
4.3 Implementation Choices 41
4.4 Array Representation of Queue 41
4.5 Algorithms (Array Queue) 42
4.6 Program: Linear Queue Using Array (LinearQueue_Array.c) 43

4.7 Linked Representation of Queue 44
4.8 Program: Queue Using Linked List (Linked QUEUE.C, corrected) 45
4.9 Stack vs Queue 47
4.10 Chapter 4 Exam Preparation 48
Chapter 5: Types of Queues and Applications 50
5.1 Types of Queues 50
5.2 Drawback of Linear Queue 50
5.3 Circular Queue 50
5.4 Program: Circular Queue Using Array (CircularQueue.txt) 52
5.5 Linear Queue vs Circular Queue 53
5.6 Priority Queue 54
5.7 Program: Ascending Priority Queue Using Array (corrected) 54
5.8 Program: Priority Queue Using Linked List [SUPPLEMENT] 56
5.9 Double-Ended Queue (Deque) 58
5.10 Program: Deque Using Circular Array (corrected) 58
5.11 Comparison of Queue Types 61
5.12 Applications of Queue 61
5.13 Chapter 5 Exam Preparation 62
Chapter 6: Pointers and Array vs Linked List 64
6.1 What Is a Pointer 64
6.2 Types of Pointers 64
6.3 Pointer Concepts in Detail 64
6.4 Call by Value and Call by Reference 65
6.5 Void Pointer and Null Pointer 66
6.6 Static vs Dynamic Data Structures 67
6.7 Array vs Linked List 67
6.8 Chapter 6 Exam Preparation 68
Chapter 7: Dynamic Memory Management 70
7.1 Dynamic Memory Allocation 70
7.2 malloc() 70
7.3 calloc() 70
7.4 free() 71
7.5 malloc vs calloc 71
7.6 Program: malloc, calloc and free [SUPPLEMENT] 71
7.7 Using malloc for a Linked List Node 72
7.8 Chapter 7 Exam Preparation 73
Chapter 8: Linked List and Singly Linked List Operations 74
8.1 Introduction to Linked List 74
8.2 Representation of a Linked List 74
8.3 Types of Linked List 75
8.4 Basic Operations Pictured (Lecture 4) 75
8.5 Operations on Singly Linked List (from the notes) 76
8.6 Display (Traversal) 76
8.7 Create 77
8.8 Count Number of Nodes 78
8.9 Search a Particular Element 78
8.10 Insert at Beginning 79
8.11 Insert at End 79
8.12 Insert After an Element 80

8.13 Insert Before an Element 81
8.14 Insert at a Particular Position 82
8.15 Delete at Beginning 83
8.16 Delete at End 83
8.17 Delete After an Element 84
8.18 Delete Before an Element 84
8.19 Delete at a Particular Position 85
8.20 Reverse a Linked List 86
8.21 Sort and Copy 88
8.22 Complete Menu-Driven Program: Singly Linked List 89
8.23 Chapter 8 Exam Preparation 97

Chapter 9: Linked List Lab Programs (LL Prog Folder) 100

9.1 Common Building Blocks: createNode and createList 100
9.2 Display 101
9.3 Count Number of Nodes (Count Number of nodes.docx) 101
9.4 Sum and Average of All Elements (sum and Avg.docx) 102
9.5 Search an Element (SEARCH element.doc) 102
9.6 Reverse a Linked List (REVERSEL.doc) 103
9.7 Combined Program (all of the above) 103
9.8 Employee List Program (EMP.docx) 106
9.9 Chapter 9 Exam Preparation 107

Chapter 10: Circular Linked List 109

10.1 Definition and Concept 109
10.2 Traversal / Print 109
10.3 Algorithms (with START and LAST pointers) 110
10.4 Program: Circular Singly Linked List (CIR_SLL.C, corrected) 110
10.5 Singly vs Circular Linked List 114
10.6 Chapter 10 Exam Preparation 114

Chapter 11: Doubly Linked List and Circular Doubly Linked List 115

11.1 Doubly Linked List: Definition and Representation 115
11.2 Algorithms 115
11.3 Program: Doubly Linked List 116
11.4 Insert After / Delete a Given Node in a DLL [SUPPLEMENT] 119
11.5 Circular Doubly Linked List [SUPPLEMENT] 119
11.6 Singly vs Doubly Linked List 121
11.7 Comparison of All Linked List Types 121
11.8 Chapter 11 Exam Preparation 121

Chapter 12: Applications of Linked Lists 123

12.1 Stack and Queue Using Linked List 123
12.2 Polynomial Representation Using Linked List 123
12.3 Polynomial Addition 123
12.4 Program: Polynomial Addition [SUPPLEMENT] 124
12.5 Music Player / Playlist [SOURCE slide 83, picture only] 126
12.6 Choosing the Right Linked List [SUPPLEMENT, for the slide-82 outcome] 127
12.7 Chapter 12 Exam Preparation 127

Chapter 13: Master Summary Tables for Last-minute Revision 128

13.1 All Structures at a Glance 128
13.2 Master Complexity Table 128

13.3 Key Formulas and Conditions 129
13.4 Exam Answer Writing Tips (general) 129
Appendix A: Source Files Reviewed and Where They Are Covered 130
Appendix B: Index of Corrections to the Source Notes 132
Appendix C: Turbo C Style vs Standard C 133

Chapter 1: Introduction to Data Structures

Sources: *Module 1 "Introduction to Data Structures.pdf"; Module 2 "Lecture 2 Importance of Data Structures, Linear vs Nonlinear Structures.pdf" (and the same slides 5-15 of "Module 2 Linked List.pptx").*

1.1 Why Learn Data Structures

[SOURCE] **The lecture opens with a problem:** find your roll number in a book of 10,000 pages where roll numbers are arranged in increasing order.

```
Sequential way : start at page 1 and check every page. Very slow.
Smart way      : open page 5000. If all roll numbers there are smaller
                 than yours, your number is in the second half, so jump
                 to 7500. If all numbers there are larger, go back
                 towards the middle of 5000-7500 (the slide says 6450),
                 and keep halving the range.
```

The smart way is the BINARY SEARCH algorithm. It works only because the data is ORGANISED (sorted). This is the whole point of the subject: the way data is organised decides how fast we can work with it.

```
Halving search illustrated (pages 1..10000):
   [1 ................ 5000 ................ 10000]   look at middle
                     [5000 ...... 7500 ..... 10000] go right
                     [5000 .. 6250 .. 7500]         go left
   ... after about 14 halvings (2^14 = 16384 > 10000) only one page is left.
```

[SOURCE] **The slide also asks:**

1. How would you keep a deck of cards arranged? (list / sorting)
2. How are files and folders organised on a disk? (tree)
3. How can Facebook friends, friends of friends and mutual friends be represented? (graph)
4. How do you search a word in a dictionary? (sorted data, search)

[SOURCE] **Reasons given in the notes:** solving complex real-world problems, cracking interviews at product-based companies, and preparing for GATE. The slides also ask why engineers at companies like Google, Microsoft, Facebook and Amazon are valued so highly (they can design efficient solutions using the right data structures), and show Bloom's taxonomy as the scale used to judge student competency (remember, understand, apply, analyse, evaluate, create).

1.2 Definition of Data Structure

Definition (exam-ready):

> **A data structure is a way of organising data in the memory of a computer that considers not only the elements stored but also their relationship to each other, so that the data can be used efficiently.**

[SOURCE] **Module 2 adds:** "Advance knowledge about the relationship between data items allows designing of efficient algorithms for the manipulation of data."

Simple explanation:

A data structure is a "container plus rules". The container holds data; the rules say how items are related (next to each other, parent-child, connected) and which operations are allowed.

[SOURCE] A data structure must address two fundamental concerns:

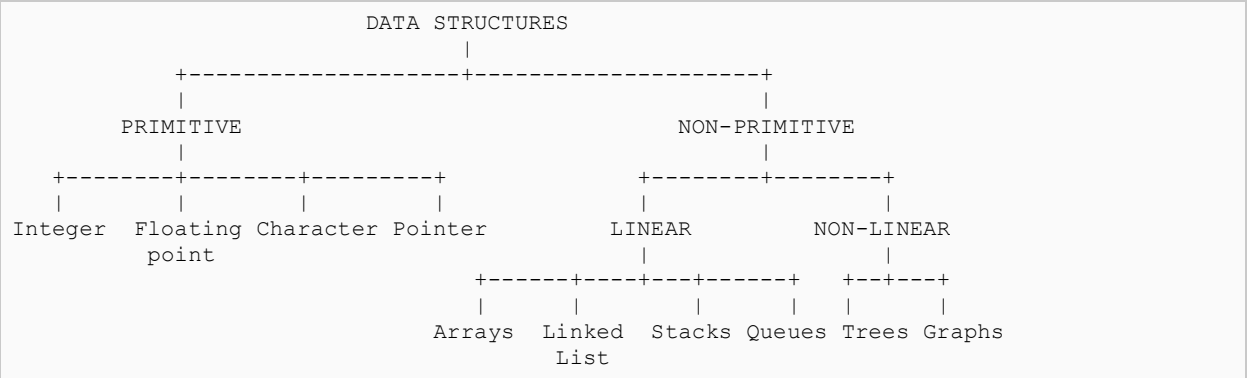
I. How the data will be stored.

II. What operations will be performed on it.

The way data is organised affects the performance of a program for different tasks (for example, searching a sorted array is much faster than searching an unsorted one).

1.3 Classification of Data Structures

[SOURCE] **Classification tree from the Module 1 slides:**



1.3.1 Primitive data structures

Basic types directly supported by the machine/language: int, float (floating point), char, and pointers. They hold a single value.

1.3.2 Non-primitive data structures

Built using primitive types. They hold a collection of values and are divided into linear and non-linear.

1.3.3 Linear data structures

[SOURCE] "In a linear data structure the elements are stored in sequential order." Each element has a unique successor (except the last). Examples: array, linked list, stack, queue.

1.3.4 Non-linear data structures

[SOURCE] "Elements are stored based on the hierarchical relationship among the data." An element can be connected to many elements. Examples: tree, graph.

1.3.5 Linear vs Non-linear (comparison)

Point	Linear	Non-linear
Arrangement	Sequential, one after another	Hierarchical or network (many connections)
Successor	Each element has at most one next element	An element may have many next elements (children / neighbours)
Levels	Single level	Multiple levels
Traversal	All elements visited in a single run	Needs special methods (tree/graph traversals)
Implementation	Easier	More complex
Examples	Array, linked list, stack, queue	Tree, graph

1.4 Brief Description of Each Structure

1.4.1 Array

[SOURCE] An array is a set of a finite number of HOMOGENEOUS elements (all of the same type) stored in CONSECUTIVE memory locations and referred to by a common name. Types: 1-D, 2-D, and so on.

```
int a[6];          index:  0    1    2    3    4    5
                          +----+----+----+----+----+----+
                       a  | 10 | 20 | 30 | 40 |    |    |
                          +----+----+----+----+----+----+
```

1.4.2 Linked list

[SOURCE] A linear collection of data elements in which the linear order is NOT given by their physical placement in memory. Items need not be in consecutive locations; each node stores the address of the next. Elements can be added at the front, middle or end. Linked lists can be used for dynamic implementation of stacks and queues.

Module 2 (Lecture 2) words it as: "a collection of data of the same data type, but the data items need not be stored in consecutive memory locations". (A heterogeneous list, 6.5, is the exception that stores different types using void pointers.)

```
START -> [10| ]-> [20| ]-> [30| X]
```

1.4.3 Stack

[SOURCE] A Last-In-First-Out (LIFO) linear structure; insertion and deletion happen only at one end called TOP. Uses: reversing a string, expression evaluation, recursive functions, checking well-formedness of parentheses, backtracking.

```
| 20 | <- top
| 10 |
+----+
```

1.4.4 Queue

[SOURCE] A First-In-First-Out (FIFO) linear structure; insertion at REAR, deletion at FRONT. Uses: printer spooler, CPU task scheduling, handling of interrupts in real-time systems.

```
front                rear
  |                    |
[10] [20] [30] [40]
```

1.4.5 Tree

[SOURCE] A non-linear hierarchical structure of nodes connected by edges. Used to represent data with a hierarchical relationship. Applications: manipulate hierarchical data, make information easy to search, manipulate sorted lists, workflow for compositing digital images.

```
    A
   / \
  B   C
 / \
D   E
```

1.4.6 Graph

[SOURCE] A collection of nodes that hold data and are connected to other nodes through edges. The relationship is between pairs of elements and is not necessarily hierarchical. Uses: shortest

routes, searching, social networks, internet routing, electrical and communication networks, airline routes.

```
A ----- B
| \     |
|  \    |
C ----- D
```

1.5 Areas Where Data Structures Are Used

[SOURCE] Compiler design, operating systems, database management systems, statistical analysis, networking, graphics, artificial intelligence, simulation, and many more.

1.6 Operations on Data Structures

[SOURCE] **The ten operations listed in the notes:**

Operation	Meaning
Creation	Allocating/declaring the structure
Insertion	Adding a new element
Deletion	Removing an element
Traversal	Visiting every element once (e.g. to print)
Destroy	Releasing the whole structure's memory
Updating	Changing the value of an existing element
Merging	Combining two structures into one
Selection	Choosing/accessing a particular element
Sorting	Arranging elements in an order
Searching	Finding the location of a given element

1.7 Data Type

> **Definition** [SOURCE]**:** A data type consists of two parts:

(a) a set of values, and
(b) a set of operations on those values.

Example: int has values like ..., -1, 0, 1, 2, ... and operations +, -, *, /, %, comparison.

[SOURCE] The slides show library functions as examples of operations provided on built-in data:

```
<math.h>   : double sqrt(double x), double ceil(double x),
             double floor(double x), double fabs(double x)
<string.h> : char *strcat(char *dest, const char *src),
             int strcmp(const char *str1, const char *str2),
             char *strcpy(char *dest, const char *src)
We use these functions without knowing how they are written inside.
That idea - using operations without knowing their implementation - leads
to the Abstract Data Type.
```

1.8 Abstract Data Type (ADT)

Definition (exam-ready) [SOURCE]**:**

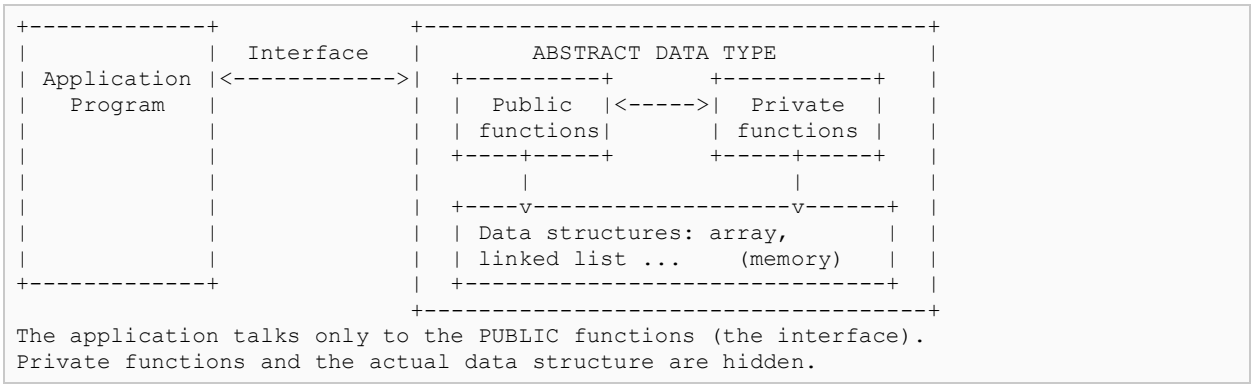
> **An Abstract Data Type (ADT) is a logical description of how we view the data and the operations that are allowed on it, without regard to how they will be implemented.**

Key points [SOURCE]**:**

1. An ADT is composed of (a) a collection of data and (b) a set of operations on that data.

2. The specification of an ADT says WHAT the operations do, not HOW they are implemented.
3. Implementing an ADT includes choosing a particular data structure. "A data structure is the implementation for an ADT."
4. Encapsulating the implementation details hides them from the user's view. This is called INFORMATION HIDING.
5. Quiz answer in notes: "A mathematical model with a collection of operations defined on that model is called an Abstract Data Type."
6. Quiz answer in notes: all of these are true - an ADT is a model of a certain kind of data structure; we know what it can do but how it does it is hidden; an ADT is a user-defined type.

1.8.1 ADT model [SOURCE, redrawn from the slide figure]



1.8.2 ADT example for a List [SOURCE]

```
User program  : main, merge, compare, ...
Public funcs  : create list, traverse list, destroy list,
                add an element, remove an element
Private funcs : insert, search, delete
The user calls "add an element"; internally the ADT may call search and
insert. The user never touches the nodes or array directly.
```

1.8.3 Data type vs ADT vs Data structure [SUPPLEMENT]

Point	Data type	ADT	Data structure
What it is	Values + operations built into language	Logical model (what)	Concrete implementation (how)
Example	int, float, char	Stack, Queue, List	Array-based stack linked stack
Hides how?	Yes (by compiler)	Yes - by definition	No - shows the memory layout

1.9 Quiz Questions From the Notes [SOURCE]

Q1. A _____ organises data like a line in a supermarket, where the first one in line is the first one out. Answer: Queue (FIFO).

Q2. Which of these is a non-linear data structure: Stacks, List, Strings, Trees? Answer: Trees.

Q3. Queue works on: LIFO / FIFO / FILO / none. Answer: FIFO.

Q4. A mathematical model with a collection of operations defined on it is:
Answer: Abstract Data Type.

Q5. Choose the correct option about ADT. Answer: All of the above.

1.10 Chapter 1 Exam Preparation

Quick Revision

- Data structure = organisation of data + relationship + allowed operations.
- **Two concerns:** how data is stored, what operations are performed.
- **Primitive:** int, float, char, pointer. Non-primitive: linear, non-linear.
- **Linear:** array, linked list, stack, queue. Non-linear: tree, graph.
- Stack = LIFO (one end, TOP). Queue = FIFO (REAR insert, FRONT delete).
- Data type = set of values + set of operations.
- ADT = what operations do, not how; data structure implements an ADT; hiding the implementation = information hiding.
- **Ten operations:** creation, insertion, deletion, traversal, destroy, updating, merging, selection, sorting, searching.

Important Definitions

- **Data structure:** a way of organising data in memory that considers both the elements and their relationship, so it can be used efficiently.
- **Linear data structure:** elements are arranged sequentially; each element has a unique successor.
- **Non-linear data structure:** elements are arranged hierarchically or as a network; an element may relate to many others.
- **Abstract Data Type:** a logical description of data and allowed operations, independent of implementation.

Common Mistakes

- Calling a stack or queue "non-linear". They are linear.
- Saying a linked list stores elements in consecutive memory. It does not.
- **Mixing ADT and data structure:** stack is an ADT; "stack using array" is a data structure implementing it.
- Writing "binary search works on any array". It needs SORTED data.

Practice Questions (Answers Follow Each Question)

1. [S] Define data structure. What two concerns must it address?

   Ans: See 1.2. (1) how data is stored (2) what operations are performed.

2. [S] Classify data structures with a neat diagram.

   Ans: Draw the tree in 1.3 and give one line on each class.

3. [S] Differentiate linear and non-linear data structures.

   Ans: Use the table in 1.3.5 (at least 5 points with examples).

4. [S] What is an ADT? Explain with the List example.

   Ans: Definition 1.8, points 1-4, and the public/private functions of 1.8.2. Mention information hiding.

5. [S] List operations performed on data structures.

   Ans: Table in 1.6.

6. [S] List areas where data structures are used extensively.

   Ans: Section 1.5.

7. [A] Which structure best models (a) folders on a disk (b) a printer's job list (c) the undo feature in an editor (d) road network?

Ans: (a) tree (b) queue (c) stack (d) graph.

8. [A] Why is binary search faster than sequential search for the roll- number problem? How many pages at most are checked for 10,000 pages?

Ans: Each check halves the search range, so about log2(10000), i.e. at most 14 checks, versus up to 10,000 checks sequentially.

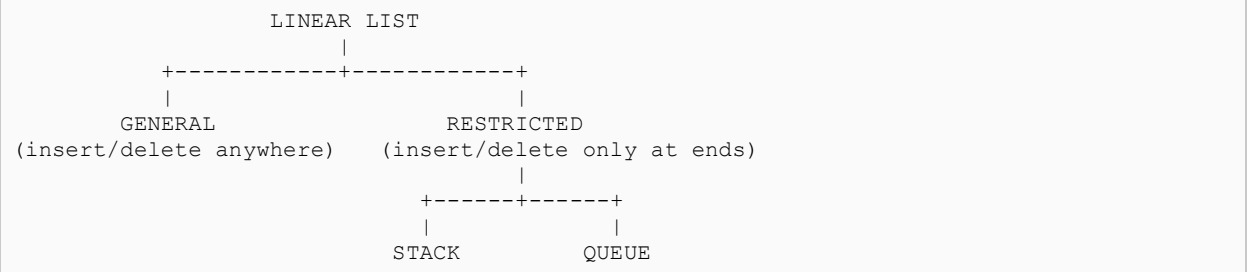
Chapter 2: Stack

Sources: *Module 1 "Stack-1.pdf" (stack ADT, array implementation) and "Stack-2.pdf" slides 4-6 (linked implementation).*

2.1 Linear Lists: General and Restricted

[SOURCE] A linear list is a list in which each element has a unique successor.

Element 1 -> Element 2 -> Element 3 -> ... -> Element N



Stack and queue are RESTRICTIVE data structures because operations are allowed only at specific ends.

2.2 Definition and Concept

Definition (exam-ready):

> **A stack is a linear data structure (and an Abstract Data Type) in which insertion and deletion of elements take place only at one end, called the TOP of the stack. The element inserted last is removed first, so a stack follows the Last-In-First-Out (LIFO) principle.**

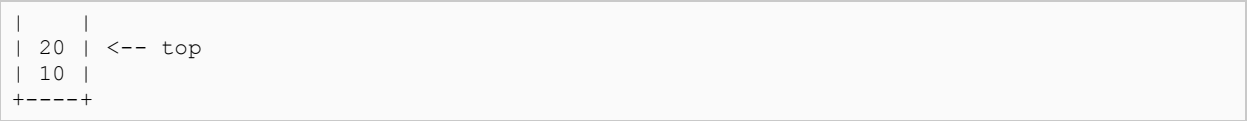
Simple explanation:

Think of a pile of plates. You put a plate on top and you also take a plate from the top. The plate placed last is taken first.

[SOURCE] **Real-world examples:** deck of cards, stack of books, pile of plates.

[SOURCE] **Key facts:**

- A stack is a collection of elements stored and retrieved one at a time.
- At any time only the TOP element can be accessed.
- It is LIFO (equivalently, First-In-Last-Out, FILO).
- It is a restrictive data structure (operations at one end only).



2.3 Stack ADT (Operations)

[SOURCE]

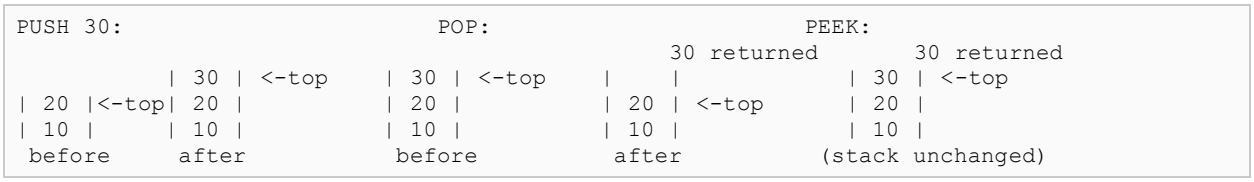
Operation	Description
push(x)	Insert x on top of the stack
pop()	Delete (and return) the top element
peek()	Return the top element WITHOUT deleting it (stack top)
isFull()	Check whether the stack is full
isEmpty()	Check whether the stack is empty
size()	Return the number of elements present in the stack

Two error conditions:

```
OVERFLOW  : trying to PUSH onto a FULL stack.
UNDERFLOW : trying to POP (or peek) from an EMPTY stack.
```

2.4 Push, Pop and Peek Illustrated

[SOURCE diagrams redrawn]



[SOURCE] Worked sequence from the slides (stack written bottom -> top):

```
Empty stack            top = -1
Push(5)   -> [5]                    top = 0
Push(7)   -> [5, 7]                 top = 1
Push(18)  -> [5, 7, 18]             top = 2
Push(15)  -> [5, 7, 18, 15]         top = 3
pop()     -> returns 15 -> [5,7,18] top = 2
Push(30)  -> [5, 7, 18, 30]         top = 3
pop()     -> returns 30 -> [5,7,18] top = 2
pop()     -> returns 18 -> [5,7]    top = 1
pop()     -> returns 7  -> [5]      top = 0
(The slide draws this sequence as a "snake": the second row is read
from right to left - pop() 15, Push(30), pop() 30, pop() 18, pop() 7.)
```

2.5 Implementation Choices

[SOURCE] Stacks are usually implemented using arrays or linked lists.

1. Static implementation using arrays: suitable when the maximum stack size is known ahead of time.
2. Dynamic implementation using linked list: used when the maximum size is not known beforehand.

2.6 Array Representation of Stack

[SOURCE]

- The stack is stored in an array.
- A variable TOP stores the index (the slide says "address") of the topmost element.
- A variable MAX is the maximum number of elements the stack can hold.

```
#define MAX 20
int stack[MAX], top = -1;

Empty  : TOP = -1
Full   : TOP = MAX - 1

Memory picture, MAX = 5, after pushing 10, 20, 30:
     index :   0    1    2    3    4
             +----+----+----+----+----+
     stack   | 10 | 20 | 30 |    |    |
             +----+----+----+----+----+
                        ^
                      top = 2
```

2.7 Algorithms for Stack Operations (Array)

2.7.1 Push [SOURCE]

```
Input : stack array, TOP, MAX, VALUE.   Output: VALUE stored on top.
Step 1: IF TOP = MAX-1
           PRINT "OVERFLOW" and go to Step 4
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
[CORRECTION] The slide's Step 1 prints OVERFLOW but does not say "go to
Step 4". Without it, the algorithm would still increment TOP and write
outside the array. Always stop after printing OVERFLOW.
```

2.7.2 Pop [SOURCE]

```
Step 1: IF TOP = -1
           PRINT "UNDERFLOW" and go to Step 5
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: RETURN VAL
Step 5: END
(Same correction: stop after UNDERFLOW.)
```

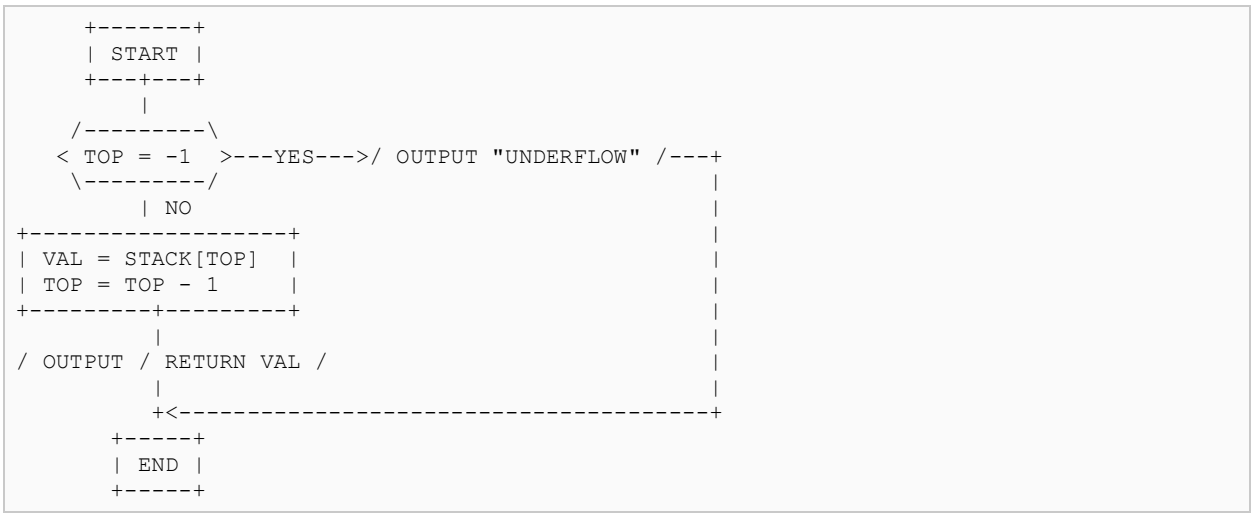
2.7.3 Peek [SOURCE]

```
Step 1: IF TOP = -1
           PRINT "STACK IS EMPTY" and go to Step 3
Step 2: RETURN STACK[TOP]
Step 3: END
```

2.7.4 Flowchart of PUSH [SUPPLEMENT]



2.7.5 Flowchart of POP [SUPPLEMENT]



2.8 Program: Stack Using Array

[SUPPLEMENT] The slide "Program to implement Stack using Array" has only a title (the code was shown live in class). The program below implements the slide algorithms exactly.

Problem statement: Write a menu-driven C program to implement a stack using an array with push, pop, peek and display operations.

Program: stack_array.c

```
/* Stack using array */
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

int isEmpty(void)
{
    return (top == -1);
}

int isFull(void)
{
```

```
    return (top == MAX - 1);
}

void push(int value)
{
    if (isFull())
    {
        printf("Stack Overflow\n");
        return;
    }
    top = top + 1;
    stack[top] = value;
    printf("%d pushed\n", value);
}

int pop(void)
{
    int value;
    if (isEmpty())
    {
        printf("Stack Underflow\n");
        return -1;
    }
    value = stack[top];
    top = top - 1;
    return value;
}

int peek(void)
{
    if (isEmpty())
    {
        printf("Stack is empty\n");
        return -1;
    }
    return stack[top];
}

void display(void)
{
    int i;
    if (isEmpty())
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (top to bottom): ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main(void)
{
    int choice, value;
    do
    {
        printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice)
        {
        case 1:
            printf("Enter value: ");
            scanf("%d", &value);
            push(value);
            break;
        case 2:
            value = pop();
            if (value != -1)
                printf("Popped %d\n", value);
            break;
        case 3:
            value = peek();
```

```
            if (value != -1)
                printf("Top element is %d\n", value);
            break;
        case 4:
            display();
            break;
        case 5:
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 5);
    return 0;
}
```

Code explanation:

- #define MAX 5 / int stack[MAX] / int top = -1 : the array holds the elements; top = -1 means empty. Globals are used so every function sees the same stack.
- isEmpty() returns 1 when top == -1; isFull() returns 1 when top == MAX - 1. In C a comparison already gives 1 or 0.
- push(): checks overflow FIRST, then increments top, then stores. The order matters: storing before incrementing would overwrite the current top element.
- pop(): checks underflow, copies stack[top] into value, then decrements top. The element is not erased from the array; it simply becomes unreachable and will be overwritten by the next push.
- peek(): returns stack[top] but does not change top.
- display(): prints from top down to index 0 (top of stack first).
- main(): a do-while menu loop. scanf returning something other than 1 (for example end of input) ends the loop safely.
- Returning -1 for "empty" is a simple convention used in your notes; it means -1 cannot be stored as real data. An exam answer may mention this limitation.

Sample run (MAX = 5) - actual output:

```
Push 10, Push 20, Push 30   -> 10 pushed / 20 pushed / 30 pushed
Display                     -> Stack (top to bottom): 30 20 10
Peek                        -> Top element is 30
Pop, Pop, Pop               -> Popped 30 / Popped 20 / Popped 10
Pop                         -> Stack Underflow
Push 1,2,3,4,5              -> 1 pushed ... 5 pushed
Push 6                      -> Stack Overflow
Display                     -> Stack (top to bottom): 5 4 3 2 1
```

Dry run of variables for: push 10, push 20, pop, push 40

Operation	top	stack[0..4]	Output
start	-1	_ _ _ _ _	
push 10	0	10 _ _ _ _	10 pushed
push 20	1	10 20 _ _ _	20 pushed
pop	0	10 (20) _ _ _	Popped 20
push 40	1	10 40 _ _ _	40 pushed

(20) means the value is still in memory but no longer part of the stack.

Complexity:

```
push, pop, peek, isEmpty, isFull : O(1) time - a fixed number of steps,
                                   no loops.
display                          : O(n) time.
Space: O(MAX) for the array, fixed at compile time; O(1) auxiliary per
operation.
```

Exam-ready explanation (short):

The stack is an array with an index TOP initialised to -1. PUSH checks TOP = MAX-1 for overflow, increments TOP and stores the value. POP checks TOP = -1 for underflow, returns STACK[TOP] and decrements TOP. PEEK returns STACK[TOP] without changing TOP. All three take O(1) time.

2.9 Linked Representation of Stack (Linked Stack)

[SOURCE]

- **Every node has two parts:** DATA and the address of the NEXT node.
- The START pointer of the linked list is used as TOP. All insertions and deletions are done at the node pointed to by TOP.
- If TOP = NULL, the stack is empty.
- There is no fixed size, so no "stack full" condition except when malloc fails (memory exhausted).

```
 TOP
  |
  v
[10 | -]--> [18 | -]--> [34 | X]          (10 is the top element)
```

2.9.1 PUSH algorithm (linked) [SOURCE]

```
Step 1: Allocate memory for the new node and name it NEWNODE
Step 2: SET NEWNODE->DATA = VAL
Step 3: IF TOP = NULL
           SET NEWNODE->NEXT = NULL
           SET TOP = NEWNODE
        ELSE
           SET NEWNODE->NEXT = TOP
           SET TOP = NEWNODE
Step 4: END
Note: both branches can be merged into "NEWNODE->NEXT = TOP; TOP =
NEWNODE", because when TOP is NULL this already sets NEXT to NULL.
```

```
Push 5 onto  TOP->[10]->[18]->[34]->X :
   NEWNODE [5| ]
   NEWNODE->NEXT = TOP     [5| ]->[10]->[18]->[34]->X
   TOP = NEWNODE           TOP->[5]->[10]->[18]->[34]->X
```

2.9.2 POP algorithm (linked) [SOURCE]

```
Step 1: IF TOP = NULL
           PRINT "Stack Underflow" and go to Step 5
Step 2: SET PTR = TOP
Step 3: SET TOP = TOP->NEXT
Step 4: FREE PTR
Step 5: END
[SUPPLEMENT] To return the popped value, save PTR->DATA before FREE.
```

```
Pop from TOP->[10]->[18]->[34]->X :
   PTR = TOP            PTR->[10]
   TOP = TOP->NEXT      TOP->[18]->[34]->X
   free(PTR)            node 10 released; returns 10
```

2.10 Program: Stack Using Linked List

[SUPPLEMENT] The notes give the algorithms; this program implements them.

Program: stack_linked.c

```
/* Stack using singly linked list */
```

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *top = NULL;

void push(int value)
{
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    if (newNode == NULL)
    {
        printf("Memory not available (Overflow)\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;      /* works for empty stack too: top is NULL */
    top = newNode;
    printf("%d pushed\n", value);
}

int pop(void)
{
    struct node *ptr;
    int value;
    if (top == NULL)
    {
        printf("Stack Underflow\n");
        return -1;
    }
    ptr = top;
    value = ptr->data;
    top = top->next;
    free(ptr);
    return value;
}

int peek(void)
{
    if (top == NULL)
    {
        printf("Stack is empty\n");
        return -1;
    }
    return top->data;
}

void display(void)
{
    struct node *temp = top;
    if (temp == NULL)
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (top to bottom): ");
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main(void)
{
    int choice, value;
    do
    {
        printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit\nEnter choice: ");
```

```
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice)
        {
        case 1:
            printf("Enter value: ");
            scanf("%d", &value);
            push(value);
            break;
        case 2:
            value = pop();
            if (value != -1)
                printf("Popped %d\n", value);
            break;
        case 3:
            value = peek();
            if (value != -1)
                printf("Top element is %d\n", value);
            break;
        case 4:
            display();
            break;
        case 5:
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 5);
    while (top != NULL)          /* release remaining nodes */
        pop();
    return 0;
}
```

Code explanation:

- struct node holds data and next. top is the START pointer of the list.
- push(): malloc a node; if malloc returns NULL, memory is exhausted (that is the only "overflow" for a linked stack). newNode->next = top links the new node in front; top = newNode makes it the new top.
- pop(): ptr remembers the old top so it can be freed AFTER top moves forward. Freeing first and then reading top->next would read freed memory (a serious bug).
- display(): walks from top to NULL.
- At exit, remaining nodes are popped so all memory is freed.

Sample run - actual output:

```
Push 10, Push 20 -> 10 pushed / 20 pushed
Display          -> Stack (top to bottom): 20 10
Peek             -> Top element is 20
Pop, Pop         -> Popped 20 / Popped 10
Pop              -> Stack Underflow
```

Complexity: push O(1), pop O(1), peek O(1), display O(n). Space O(n) for n nodes; each node needs extra memory for the next pointer.

2.11 Array Stack vs Linked Stack

Point	Array stack	Linked stack
Memory allocation	Static (compile time)	Dynamic (run time)
Size	Fixed, MAX	Grows/shrinks as needed
Overflow	When TOP = MAX-1	Only when malloc fails
Underflow	TOP = -1	TOP = NULL
Extra memory	None per element, but unused slots are wasted	One pointer per node
push / pop time	O(1)	O(1)
Implementation	Simpler	Needs pointers, malloc and free
Use when	Maximum size known	Maximum size unknown

2.12 Chapter 2 Exam Preparation

Quick Revision

- Stack = LIFO, one end (TOP). Restrictive linear structure.
- **Operations:** push, pop, peek, isFull, isEmpty, size.
- **Array:** top = -1 empty, top = MAX-1 full. Push: ++top then store. Pop: read then top--.
- **Linked:** top = NULL empty. Push at front; pop from front; free the node.
- All main operations O(1).

Important Definitions

- **Stack:** a linear data structure where insertion and deletion occur at one end called TOP; it follows LIFO.
- **Overflow:** attempt to insert into a full stack.
- **Underflow:** attempt to delete from an empty stack.
- **Peek:** returning the top element without removing it.

Complexity Summary

Operation	Array stack	Linked stack
push	O(1)	O(1)
pop	O(1)	O(1)
peek	O(1)	O(1)
isEmpty	O(1)	O(1)
display	O(n)	O(n)
space	O(MAX) fixed	O(n) + links

Common Mistakes

- Writing stack[++top] check-less: always test overflow before pushing.
- Using top++ instead of ++top in push (stores at the old index, then moves - with top starting at -1 that writes stack[-1]).
- Forgetting free() in linked pop (memory leak), or freeing before moving top (use-after-free).
- Saying a linked stack "never overflows": it can, if heap memory is full.
- Confusing pop (removes) with peek (does not remove).

Practice Questions

1. [S] Define stack. Explain the stack ADT operations.

   Ans: Definition 2.2 and the table in 2.3.

2. [S] Write algorithms for PUSH, POP and PEEK using an array.

   Ans: 2.7.1 - 2.7.3, with overflow/underflow checks and stopping.

3. [S] Show the stack after: Push(5), Push(7), Push(18), Push(15), pop(), Push(30), pop(), pop(), pop(). What is returned by each pop?

   Ans: pops return 15, 30, 18, 7; final stack [5] (see 2.4).

4. [S] Write algorithms for push and pop on a linked stack.

   Ans: 2.9.1 and 2.9.2.

5. [S] Write a C program to implement a stack using an array.

   Ans: Program 2.8.

6. [A] MAX = 3. Trace: push 1, push 2, push 3, push 4, pop, pop, pop, pop.

   Ans: push 4 -> Stack Overflow; pops return 3, 2, 1; 4th pop -> Stack Underflow; final top = -1.

7. [A] Differentiate array and linked implementation of stack.

   Ans: Table 2.11.

8. [A] Why is the time complexity of push O(1) in both implementations?

   Ans: Push only changes TOP and writes one element/node; no loop over the other elements is needed, so the work does not depend on n.

9. [A] What is printed? push(4); push(9); printf("%d ", pop()); push(2); printf("%d ", peek()); printf("%d", pop());

   Ans: 9 2 2

Chapter 3: Applications of Stack

Sources: *"Stack-2.pdf" (applications, notations, infix to postfix/prefix), "Stack-3.pdf" (postfix evaluation, parenthesis checker, function calls, recursion), programs "INFIXPOSTFIX.C" and "EVALPOST.C".*

3.1 List of Applications

[SOURCE]

1. Expression conversion: infix to postfix (and prefix) conversion
2. Evaluation of arithmetic expressions: postfix evaluation
3. Parentheses checker / well-formedness of parentheses / balanced parentheses
4. Keeping track of function calls
5. Recursion
6. Reversal of data (Module 1 intro also lists backtracking.)

Why a stack fits all of these: in each case the item opened/seen LAST must be handled FIRST - the most recent "(" must be closed first, the most recent function call must return first, the last pushed operand is the right-hand operand.

3.2 Notations of Arithmetic Expressions

[SOURCE] Three different but equivalent ways of writing expressions:

Notation	Operator position	a+b	a+b*c
Infix	between the operands	a+b	a+b*c
Postfix	after the operands (Reverse Polish Notation,RPN)	ab+	abc*+
Prefix	before the operands (Polish Notation)	+ab	+a*bc

Why postfix/prefix [SUPPLEMENT]**:**

- Infix needs precedence rules and brackets to be unambiguous.
- Postfix and prefix need NO brackets and NO precedence rules; the order of operators already fixes the order of evaluation.
- A computer can evaluate postfix in one left-to-right scan using a stack, so compilers convert infix to postfix internally.

Operator precedence used in the notes' program (highest first):

Priority	Operators	Associativity
3	^ (power)	right to left (a^b^c = a^(b^c))
2	* /	left to right
1	+ -	left to right
0	(	(lowest inside the stack, so nothing pops it except a matching ')')

3.3 Manual Conversion (Bracketing Method)

[SUPPLEMENT] **Quick way to check answers by hand:**

1. Fully bracket the expression according to precedence.
2. For postfix, move each operator just after its closing bracket; for prefix, just before its opening bracket.
3. Remove brackets.

[SOURCE] Exercise table from the slides (left blank in the notes - solved here):

Infix	Postfix	Prefix
(a - b) * (c + d)	ab-cd+*	*-ab+cd
(a + b) / (c + d) - (d*e)	ab+cd+/de*-	-/+ab+cd*de

Working for the second row:

```
Fully bracketed: ( ( (a+b) / (c+d) ) - (d*e) )
Postfix: (a+b)->ab+ ; (c+d)->cd+ ; X/Y -> ab+cd+/ ; (d*e)->de* ;
         whole -> ab+cd+/ de* -  = ab+cd+/de*-
Prefix : (a+b)->+ab ; (c+d)->+cd ; X/Y -> /+ab+cd ; (d*e)->*de ;
         whole -> - /+ab+cd *de  = -/+ab+cd*de
```

3.4 Algorithm: Infix to Postfix Using Stack

Purpose : convert an infix expression to postfix.

Input : infix string (operands are single letters/digits).

Output : postfix string.

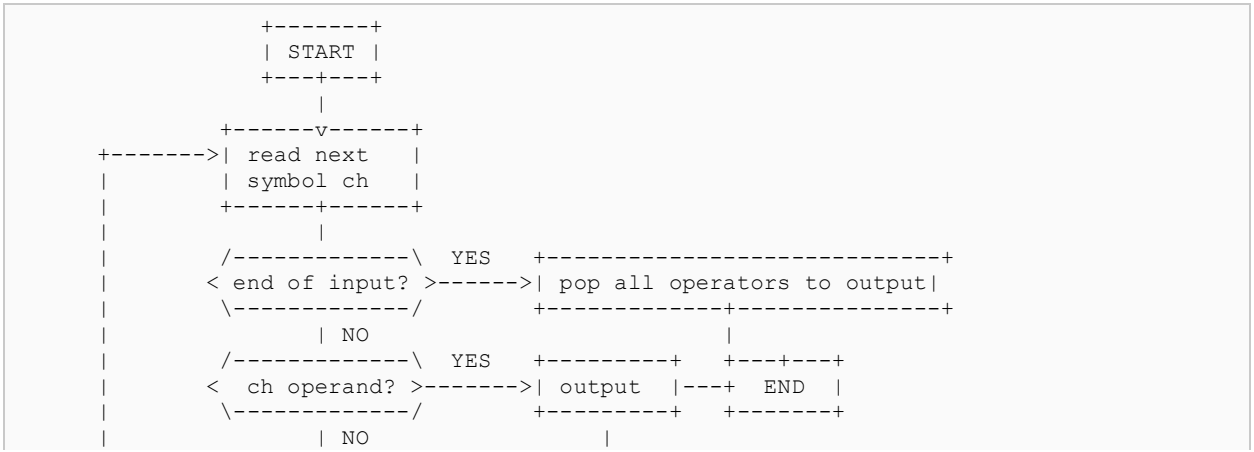
Data : a stack of characters for operators and '('.

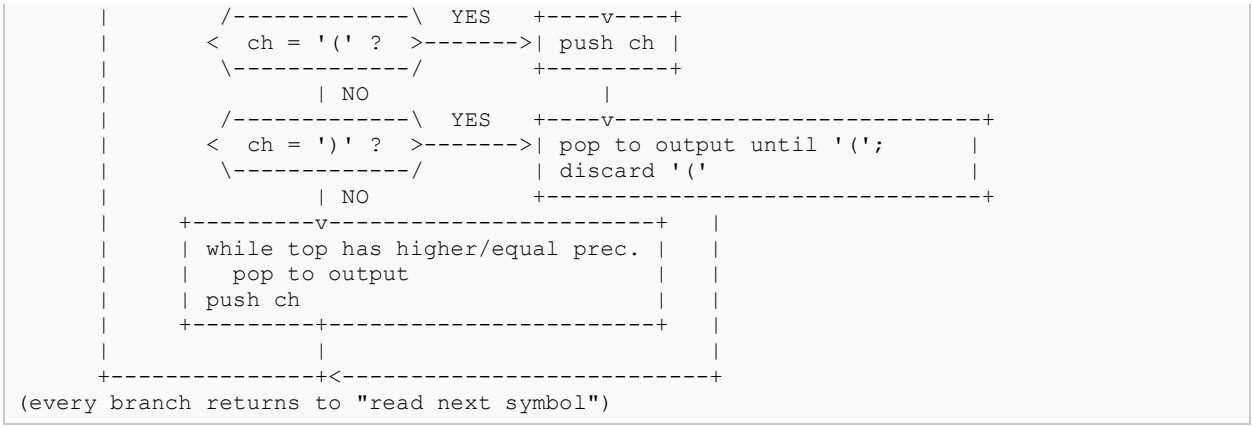
[SOURCE] **Algorithm (as in the notes, wording kept):**

```
Step 1: Scan the infix expression from left to right.
Step 2: Repeat until each character in the infix notation is scanned:
        IF '(' is encountered, push it on the stack.
        IF an operand (digit or character) is encountered, add it to the
           postfix expression.
        IF ')' is encountered, then
           a. Repeatedly pop from the stack and add to postfix until a '('
              is encountered.
           b. Discard the '(' (remove it from the stack, do not add it to
              postfix).
        IF an operator is encountered, then
           a. Repeatedly pop from the stack and add to postfix each operator
              which has HIGHER (or EQUAL) precedence than the scanned
              operator.
           b. Push the scanned operator on the stack.
Step 3: Repeatedly pop from the stack and add to postfix until the stack is
        empty.
Step 4: END
```

[CORRECTION/CLARIFICATION] The slide says "higher precedence" only. For left-associative operators (+ - * /) operators of EQUAL precedence must also be popped; otherwise a-b+c would become abc+- (meaning a-(b+c)), which is wrong. The notes' own program uses ">=", i.e. higher OR EQUAL. The only exception is '^' (right associative): for '^' pop only strictly higher.

Flowchart [SUPPLEMENT]**:**





3.5 Worked Examples (Dry Runs)

3.5.1 Example 1 [SOURCE slide 11]: (a - b) * (c + d)

Scan	Stack (bottom-> top)	Postfix	Action
(	(		push '('
a	(	a	operand -> output
-	(-	a	top is '(' (prio 0) -> push '-'
b	(-	ab	output
)	(empty)	ab-	pop '-' ; discard '('
*****	*	ab-	stack empty -> push
(	* (	ab-	push
c	* (	ab-c	output
+	* (+	ab-c	top '(' -> push
d	* (+	ab-cd	output
)	*	ab-cd+	pop '+', discard '('
end	(empty)	ab-cd+*	pop remaining '*'

Result: ab-cd+*

3.5.2 Example 2 [SOURCE slide 12]: (a + b) / (c + d) - (d * e)

Scan	Stack	Postfix	Action
(	(		push
a	(	a	output
+	(+	a	push
b	(+	ab	output
)		ab+	pop '+', discard '('
/	/	ab+	push
(	/ (	ab+	push
c	/ (	ab+c	output
+	/ (+	ab+c	top '(' -> push
d	/ (+	ab+cd	output
)	/	ab+cd+	pop '+', discard '('
-	-	ab+cd+/	'/'(2) >= '-'(1): pop '/'; push '-'
(	- (	ab+cd+/	push
d	- (	ab+cd+/d	output
*****	- (*	ab+cd+/d	top '(' -> push

Scan	Stack	Postfix	Action
e	- (*	ab+cd+/de	output
)	-	ab+cd+/de*	pop '*', discard '('
end		ab+cd+/de*-	pop '-'

Result: ab+cd+/de*-

3.5.3 Example 3 [SOURCE: sample output of INFIXPOSTFIX.C]: a-(b+c)*d-e

Scan	Stack	Postfix	Action
a		a	output
-	-	a	push
(	- (	a	push
b	- (	ab	output
+	- (+	ab	top '(' -> push
c	- (+	abc	output
)	-	abc+	pop '+', discard '('
*****	- *	abc+	'-'(1) < '*'(2): no pop; push '*'
d	- *	abc+d	output
-	-	abc+d*-	pop '*' (2>=1), pop '-' (1>=1); push '-'
e	-	abc+d*-e	output
end		abc+d*-e-	pop '-'

```
Result: abc+d*-e-   (matches the program output in your notes)
```

3.6 Program: Infix to Postfix (INFIXPOSTFIX.C, corrected)

Problem statement: Write a C program to convert an infix expression into a postfix expression using a stack.

Program: infix_postfix.c

```
/* Infix to Postfix conversion using stack */
#include <stdio.h>
#include <ctype.h>
#define MAX 50

char stack[MAX];
int top = -1;

int isEmpty(void)
{
    return (top == -1);
}

void push(char ch)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = ch;
}

char pop(void)
{
    if (isEmpty())
        return '\0';
    return stack[top--];
}
```

```
int priority(char ch)
{
    if (ch == '^')
        return 3;
    if (ch == '*' || ch == '/' || ch == '%')
        return 2;
    if (ch == '+' || ch == '-')
        return 1;
    return 0;                /* '(' and anything else */
}

void infixToPostfix(char infix[], char postfix[])
{
    int i, j = 0;
    char ch;
    for (i = 0; infix[i] != '\0'; i++)
    {
        ch = infix[i];
        if (ch == ' ' || ch == '\t' || ch == '\n')
            continue;                        /* ignore spaces */
        if (isalnum((unsigned char)ch))
            postfix[j++] = ch;               /* operand: output directly */
        else if (ch == '(')
            push(ch);
        else if (ch == ')')
        {
            while (!isEmpty() && stack[top] != '(')
                postfix[j++] = pop();
            pop();                           /* discard '(' */
        }
        else                                 /* operator */
        {
            /* '^' is right associative: pop only strictly higher */
            while (!isEmpty() &&
                   (priority(stack[top]) > priority(ch) ||
                    (priority(stack[top]) == priority(ch) && ch != '^')))
                postfix[j++] = pop();
            push(ch);
        }
    }
    while (!isEmpty())
        postfix[j++] = pop();
    postfix[j] = '\0';
}

int main(void)
{
    char infix[MAX], postfix[MAX];
    printf("Enter Infix Expression\n");
    if (fgets(infix, MAX, stdin) == NULL)
        return 0;
    infixToPostfix(infix, postfix);
    printf("Postfix Expression is %s\n", postfix);
    return 0;
}
```

Code explanation:

- stack[] holds operators and '('; top = -1 means empty.
- priority(): returns 3 for '^', 2 for * / %, 1 for + -, and 0 for '(' or anything else.
- Main loop over infix[i]:
  - spaces are skipped, so "(a - b) * (c + d)" works too;
  - isalnum(ch) (letter or digit) -> operand -> copied to postfix[j++];
  - '(' -> pushed;
  - ')' -> pop to output until '(' and then pop the '(' itself without copying it;
  - operator -> pop while the top has higher priority, or equal priority and the operator is not '^'; then push the operator.

- After the loop all remaining operators are popped.
- postfix[j] = '\0' terminates the string so printf("%s") works.

[CORRECTION] Differences from the notes' INFIXPOSTFIX.C and why:

1. <ctype.h> was missing; isalnum() needs it.
2. priority() had no return for other characters (undefined behaviour in C). Now it returns 0.
3. The original popped on ">=" for '^' as well, giving a^b^c -> ab^c^, which means (a^b)^c. Exponent is right associative, so the correct result is abc^^. Fixed. (For + - * / the original was correct.)
4. gets() replaced with fgets() (gets is unsafe and removed in C11).
5. The ')' loop also checks !isEmpty() so an unmatched ')' cannot read below the stack.
6. void main / clrscr / getch removed (Turbo C only).

Actual test outputs:

```
a-(b+c)*d-e              -> abc+d*-e-
(a-b)*(c+d)              -> ab-cd+*
(a+b)/(c+d)-(d*e)        -> ab+cd+/de*-
a+b*c                    -> abc*+
a^b^c                    -> abc^^
A+(B*C-(D/E^F)*G)*H      -> ABC*DEF^/G*-H*+
((a+b)*c-(d-e))^(f+g)    -> ab+c*de--fg+^
```

Complexity: each symbol is pushed at most once and popped at most once, so time is O(n) for an expression of length n. Auxiliary space O(n) for the stack (worst case, e.g. many '(').

3.7 Infix to Prefix Using Stack

[SOURCE] The slide "Convert Infix Expression into Prefix Expression using Stack" for (a - b) * (c + d) shows only the question. Standard method

[SUPPLEMENT]:

```
Step 1: Reverse the infix expression. While reversing, change every '('
        to ')' and every ')' to '('.
Step 2: Convert this reversed expression to postfix using the algorithm of
        3.4, with one change: for an incoming operator, pop only operators
        of STRICTLY higher precedence (equal ones stay; this keeps left
        associativity correct after the final reversal).
Step 3: Reverse the result. This is the prefix expression.
```

Example: (a - b) * (c + d)

```
Step 1: reversed with brackets swapped:  (d + c) * (b - a)
Step 2:
```

Scan	Stack	Output
(	(	
d	(	d
+	(+	d
c	(+	dc
)		dc+
*****	*	dc+
(	* (	dc+
b	* (	dc+b
-	* (-	dc+b
a	* (-	dc+ba
)	*	dc+ba-
end		dc+ba-*

```
Step 3: reverse "dc+ba-*"  ->  *-ab+cd      (matches 3.3)
```

Example 2: (a+b)/(c+d)-(d*e)

```
Reversed: (e*d)-(d+c)/(b+a)  -> postfix (strict rule): ed*dc+ba+/-
Reverse  -> -/+ab+cd*de                     (matches 3.3)
```

3.8 Evaluation of Postfix Expression

Purpose : compute the value of a postfix expression.

Input : postfix expression. Output: its numeric value.

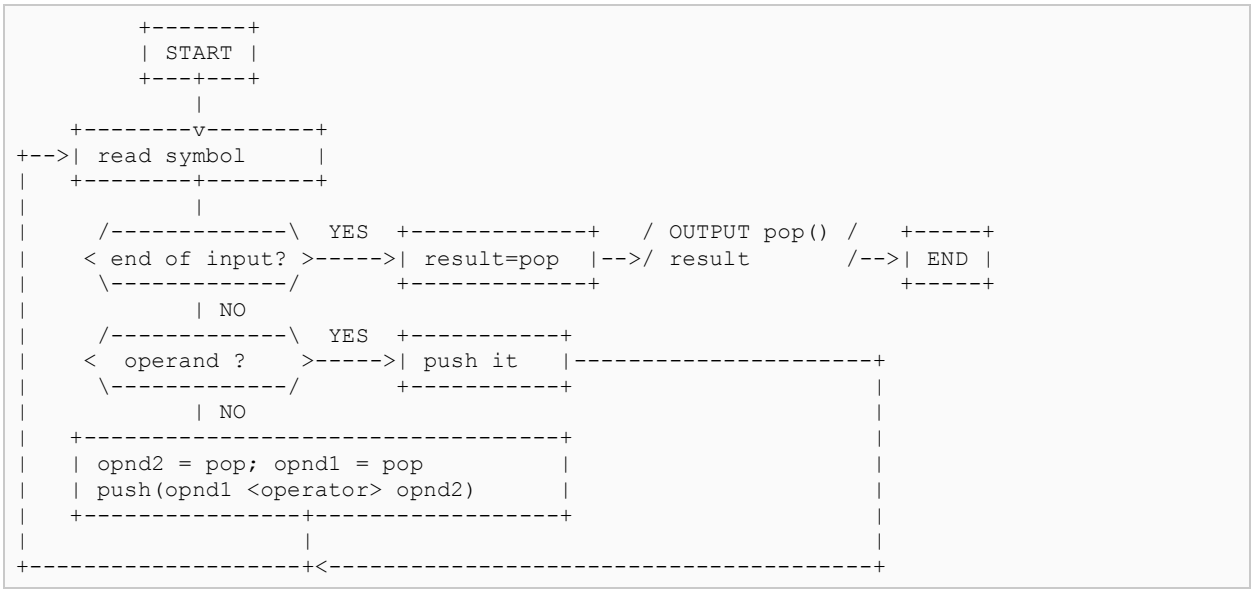
Data : a stack of operands (numbers).

[SOURCE] **Algorithm:**

```
Step 1: Scan the postfix expression from left to right.
Step 2: Repeat until each symbol is scanned:
        IF the symbol is an operand, push it on the stack.
        ELSE IF it is an operator:
            opnd2 = pop
            opnd1 = pop
            value = result of applying the symbol to opnd1 and opnd2
            push value on the stack
Step 3: Pop the result from the stack.
Step 4: END
```

Important: the FIRST pop gives the RIGHT operand (opnd2). For '-' and '/' the order matters: value = opnd1 - opnd2, opnd1 / opnd2.

Flowchart [SUPPLEMENT]:



3.8.1 Example 1 [SOURCE slide 5]: 8 9 5 - /

Symbol	Action	Stack
8	push 8	8
9	push 9	8 9
5	push 5	8 9 5
-	opnd2=5, opnd1=9, 9-5=4	8 4
/	opnd2=4, opnd1=8, 8/4=2	2
end	pop result	(empty)

Result = 2

3.8.2 Example 2 [SOURCE slide 6]: 6 2 3 + - 8 2 / +

Symb	opnd1	opnd2	value	opndstk
6				6
2				6, 2
3				6, 2, 3
+	2	3	5	6, 5
-	6	5	1	1
8				1, 8
2				1, 8, 2
/	8	2	4	1, 4
+	1	4	5	5

Result = 5

3.8.3 Example 3 [SOURCE: sample output of EVALPOST.C]: abc*+ with a=2, b=3, c=4

Push 2, push 3, push 4; '*' -> 3*4=12 -> stack 2 12; '+' -> 2+12=14. Result 14.

3.9 Program: Postfix Evaluation (EVALPOST.C, corrected)

Problem statement: Write a C program to evaluate a postfix expression whose operands are single digits or single-letter variables.

Program: eval_postfix.c

```
/* Evaluation of postfix expression using stack
   Operands: single digits (0-9) or single letters
   (the value of a letter is asked from the user) */
#include <stdio.h>
#include <ctype.h>
#define MAX 50

int stack[MAX];
int top = -1;

void push(int value)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = value;
}

int pop(void)
{
    if (top == -1)
    {
        printf("Stack Underflow: invalid expression\n");
        return 0;
    }
    return stack[top--];
}

int evalPostfix(char postfix[])
{
    int i, opnd1, opnd2, num;
    char ch;
    for (i = 0; postfix[i] != '\0'; i++)
    {
        ch = postfix[i];
        if (ch == ' ' || ch == '\t' || ch == '\n')
            continue;
        if (isdigit((unsigned char)ch))
            /* convert character digit to int */
            push(ch - '0');
        else if (isalpha((unsigned char)ch))
        {
            printf("Enter value of %c: ", ch);
            scanf("%d", &num);
            push(num);
        }
        else
        {
            /* first pop = right operand */
            opnd2 = pop();
            /* second pop = left operand */
            opnd1 = pop();
            switch (ch)
            {
            case '+': push(opnd1 + opnd2); break;
            case '-': push(opnd1 - opnd2); break;
            case '*': push(opnd1 * opnd2); break;
            case '/':
                if (opnd2 == 0)
                {
                    printf("Division by zero\n");
                    push(0);
                }
                else
                    push(opnd1 / opnd2);
                break;
            default:
                printf("Invalid operator %c\n", ch);
            }
        }
```

```
    }
    return pop();
}

int main(void)
{
    char postfix[MAX];
    printf("Enter Postfix Expression\n");
    if (fgets(postfix, MAX, stdin) == NULL)
        return 0;
    printf("Result %d\n", evalPostfix(postfix));
    return 0;
}
```

Code explanation:

- stack[] is an int stack (it stores numbers, not characters).
- isdigit(ch): ch - '0' converts the character '7' into the number 7 (character codes of '0'..'9' are consecutive).
- isalpha(ch): the program asks the value of that variable and pushes it.
- **Operator:** opnd2 = pop() FIRST, then opnd1 = pop(); result opnd1 op opnd2 is pushed.
- At the end the single value left on the stack is the answer.
- Integer division is used for '/' (8/3 gives 2).

[CORRECTION] **Changes from the notes' EVALPOST.C:**

1. Added #include <ctype.h> (needed for isdigit/isalpha).
2. gets() replaced by fgets(); spaces and the newline are skipped.
3. Overflow, underflow, division-by-zero and invalid-operator checks were added (the original had none; an invalid expression would read stack[-1]). Limitation kept from the original: operands must be single digits; "12" would be read as 1 and 2.

Actual test outputs:

```
895-/           -> Result 2
623+-82/+       -> Result 5
53+82-*         -> Result 48     ((5+3)*(8-2))
abc*+ (2,3,4)   -> Result 14
```

Complexity: O(n) time (one scan, each symbol pushed/popped a constant number of times); O(n) auxiliary space for the operand stack.

3.10 Parenthesis Checker (Balanced Parentheses)

> **Definition:** An expression is well-formed (balanced) if every opening bracket '(' '{' '[' has a matching closing bracket of the same type, and brackets close in the reverse order of opening (properly nested).

[SOURCE] **Algorithm:**

```
Step 1: Scan the expression from left to right.
Step 2: Set flag = 1
Step 3: Repeat until each symbol is scanned:
        IF symbol is '(' or '{' or '[', push it on the stack.
        IF symbol is ')' or '}' or ']', then
           IF stack is empty, set flag = 0 (and GOTO Step 5)
           ELSE pop the top into temp:
              IF symbol is ')' and temp is '{' or '[' -> flag = 0, GOTO 5
              IF symbol is '}' and temp is '(' or '[' -> flag = 0, GOTO 5
              IF symbol is ']' and temp is '(' or '{' -> flag = 0, GOTO 5
Step 4: IF stack is not empty, set flag = 0
Step 5: IF flag = 1, PRINT "Valid expression"
        ELSE PRINT "Invalid expression"
```

[CORRECTION] The slide does not write "GOTO Step 5" for the empty-stack case and has no ELSE in Step 5. Without them the algorithm keeps scanning after it already knows the answer and never prints "Invalid". Both are added above.

Three ways an expression can be invalid:

1. A closing bracket arrives when the stack is empty e.g. a+b)
2. Closing bracket type does not match the popped one e.g. (])
3. Opening brackets remain at the end e.g. ((a+b)

Dry run 1: {[()]}

```
{  push        stack: {
[  push        stack: { [
(  push        stack: { [ (
)  pop '(' ok  stack: { [
]  pop '[' ok  stack: {
}  pop '{' ok  stack: empty
end: stack empty, flag = 1 -> Valid expression
```

Dry run 2: (])

```
(  push        stack: (
[  push        stack: ( [
)  pop '[' -> ')' does not match '[' -> flag = 0 -> Invalid expression
```

Dry run 3: ((a+b)

((pushed; a + b ignored;) pops one '('. End: stack still has '(' -> Invalid expression

Program [SUPPLEMENT - the notes give only the algorithm]:

Program: paren_check.c

```
/* Parenthesis checker (balanced brackets) using stack */
#include <stdio.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
    if (top < MAX - 1)
        stack[++top] = ch;
}

char pop(void)
{
    return stack[top--];
}

int isMatchingPair(char open, char close)
{
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

int checkBalanced(char exp[])
{
    int i;
    char temp;
    for (i = 0; exp[i] != '\0'; i++)
    {
        if (exp[i] == '(' || exp[i] == '{' || exp[i] == '[')
            push(exp[i]);
        else if (exp[i] == ')' || exp[i] == '}' || exp[i] == ']')
        {
            if (top == -1)
                /* closing bracket with no opener */
```

```
                return 0;
            temp = pop();
            if (!isMatchingPair(temp, exp[i]))
                return 0;                    /* wrong type of bracket */
        }
    }
    /* leftover openers => invalid */
    return (top == -1);
}

int main(void)
{
    char exp[MAX];
    printf("Enter expression: ");
    if (fgets(exp, MAX, stdin) == NULL)
        return 0;
    if (checkBalanced(exp))
        printf("Valid expression\n");
    else
        printf("Invalid expression\n");
    return 0;
}
```

Code explanation:

- Opening brackets are pushed.
- **For a closing bracket:** if the stack is empty, return 0 (case 1); pop and compare using isMatchingPair(); a mismatch returns 0 (case 2).
- After the loop, return (top == -1): leftover openers mean invalid (case 3).
- Other characters (letters, operators) are ignored.

Actual test outputs: {[()]} Valid; ([)] Invalid; ((a+b) Invalid; a+b) Invalid; {a*(b+c)} Valid.

Complexity: O(n) time, O(n) auxiliary space (worst case all openers).

3.11 System Stack in Function Calls

[SOURCE] When a function is called, the system pushes its record (return address, local variables) on the system stack. When it finishes, the record is popped and control returns to the caller. The most recently called function always finishes first - LIFO.

Example program from the notes:

```
main() { A(); printf("Bye"); }
A()    { B(); printf("Inside A"); }
B()    { C(); printf("Inside B"); }
C()    { D(); printf("Inside C"); }
D()    { printf("Hello"); }
```

Stack growth (bottom -> top) and output:

```
                                                 | D  |
                                        | C  |   | C  |
                            | B  |      | B  |   | B  |
                  | A  |    | A  |      | A  |   | A  |
      | main |    |main|    |main|      |main|   |main|
Empty  Push main  Push A    Push B      Push C    Push D -> prints Hello

Pop D  -> back in C, prints "Inside C"
Pop C  -> back in B, prints "Inside B"
Pop B  -> back in A, prints "Inside A"
Pop A  -> back in main, prints "Bye"
Pop main -> stack empty, program ends
```

Output order: Hello, Inside C, Inside B, Inside A, Bye. (The slide's printf calls have no "\n", so on screen they would appear on one line: HelloInside CInside BInside ABye.)

3.12 Recursion

> **Definition:** Recursion is a technique in which a function calls itself (directly or indirectly) to solve a smaller instance of the same problem. It must have a BASE CASE that stops the calls.

Recursion uses the system stack: each call waits on the stack until the smaller call returns.

[SOURCE] **Factorial program:**

Program: factorial.c

```
/* Recursion: factorial (uses the system stack) */
#include <stdio.h>

int factorial(int n)
{
    int f;
    if (n == 0)
        return 1;                 /* base case */
    else
        f = n * factorial(n - 1); /* recursive case */
    return f;
}

int main(void)
{
    int fact, n;
    printf("Enter number: ");
    scanf("%d", &n);
    fact = factorial(n);
    printf("Factorial of %d = %d\n", n, fact);
    return 0;
}
```

[CORRECTION] The slide's printf("Factorial of %d = %d", fact) has two %d but only one argument; it must be printf("Factorial of %d = %d", n, fact). The slide also uses void main(); standard C uses int main.

Dry run for factorial(4) [SOURCE slide 10]:

```
Calls (pushed)                          Returns (popped)
factorial(4) = 4 * factorial(3)         = 4 * 6 = 24
factorial(3) = 3 * factorial(2)         = 3 * 2 = 6
factorial(2) = 2 * factorial(1)         = 2 * 1 = 2
factorial(1) = 1 * factorial(0)         = 1 * 1 = 1
factorial(0) = 1   (base case)          = 1

System stack at the deepest point (top first):
   | factorial(0) |
   | factorial(1) |
   | factorial(2) |
   | factorial(3) |
   | factorial(4) |
   |    main      |
```

Actual output: Enter number: 4 -> Factorial of 4 = 24; for 0 -> 1.

Complexity: O(n) time (n+1 calls); O(n) auxiliary space on the system stack. Note: int overflows for n > 12 (13! > 2^31 - 1). A negative n never reaches the base case and crashes (stack overflow) - check n >= 0 in a real program.

3.13 Reversal of Data [SUPPLEMENT]

Push every character of a string, then pop them all: they come out in reverse order because of LIFO.

"DATA" -> push D, A, T, A -> pop gives A, T, A, D -> "ATAD".

3.14 Chapter 3 Exam Preparation

Quick Revision

- Postfix = RPN (operator after); prefix = Polish (operator before).
- **Infix->postfix:** operand -> output; '(' -> push; ')' -> pop till '('; operator -> pop higher/equal precedence, then push; end -> pop all.
- **Infix->prefix:** reverse (swap brackets), postfix (strict pop), reverse.
- **Postfix evaluation:** operand push; operator: op2=pop, op1=pop, push(op1 op op2); answer = final pop.
- **Parenthesis checker:** push openers; on closer pop and match; at end stack must be empty.
- Function calls and recursion use the system stack (LIFO).

Important Definitions

- **Infix:** operator between operands (a+b).
- **Postfix / Reverse Polish Notation:** operator after operands (ab+).
- **Prefix / Polish Notation:** operator before operands (+ab).
- **Balanced parentheses:** every opener has a matching closer of the same type in the correct nesting order.
- **Recursion:** a function calling itself with a base case to stop.

Complexity Summary

Algorithm	Time	Auxiliary space
Infix to postfix	O(n)	O(n)
Infix to prefix	O(n)	O(n)
Postfix evaluation	O(n)	O(n)
Parenthesis checker	O(n)	O(n)
Recursive factorial	O(n)	O(n) call stack

Common Mistakes

- **Swapping operands in evaluation:** for "95-" the answer is 9-5 = 4, not 5-9. First pop = opnd2.
- Writing '(' or ')' into postfix. Brackets never appear in postfix/prefix.
- Forgetting to pop the remaining operators at the end.
- Popping '(' when an operator arrives - '(' has the lowest priority and is removed only by ')'.
- Treating '^' as left associative (a^b^c is abc^^).
- In the bracket checker, forgetting the final "stack empty" test.

Practice Questions

1. [S] Explain infix, postfix and prefix notation with examples.
   Ans: 3.2 table; mention RPN/Polish names and no brackets needed.
2. [S] Convert (a-b)*(c+d) and (a+b)/(c+d)-(d*e) to postfix and prefix.
   Ans: ab-cd+*, *-ab+cd ; ab+cd+/de*-, -/+ab+cd*de.
3. [S] Write the algorithm to convert infix to postfix and trace it for (a+b)/(c+d)-(d*e).
   Ans: 3.4 and table 3.5.2.
4. [S] Write the algorithm to evaluate a postfix expression. Evaluate 6 2 3 + - 8 2 / +. Ans: 3.8 ; result 5 (table 3.8.2).

5. [S] Evaluate 8 9 5 - /. Ans: 2.
6. [S] Write an algorithm to check well-formedness of parentheses.

   Ans: 3.10 with the corrections.
7. [S] Show the system stack for the main/A/B/C/D program and give the output. Ans: 3.11; Hello, Inside C, Inside B, Inside A, Bye.
8. [S] Explain recursion using factorial(4) and the stack. Ans: 3.12.
9. [A] Convert A+(B*C-(D/E^F)*G)*H to postfix.

   Ans: ABC*DEF^/G*-H*+ (verified by the program).
10. [A] Convert a*b+c*d-e to postfix and prefix.

    Ans: Postfix ab*cd*+e- ; Prefix -+*ab*cde.
11. [A] Evaluate the postfix 5 3 + 8 2 - *. Ans: (5+3)*(8-2) = 48.
12. [A] Evaluate the postfix 2 3 1 * + 9 -. Ans: 2 + 3*1 = 5; 5 - 9 = -4.
13. [A] Is "{(a+b]*c}" balanced? Show the stack.

    Ans: { pushed, (pushed, ']' pops '(' -> mismatch -> Invalid.
14. [A] What does the original INFIXPOSTFIX.C print for a^b^c, and why is it wrong? Ans: ab^c^; it pops on equal precedence for '^', but '^' is right associative, so the correct answer is abc^^.
15. [A] What is the time complexity of infix to postfix conversion? Justify.

    Ans: O(n): each symbol is read once and each operator is pushed and popped at most once.

Chapter 4: Queue

Sources: *"Queue-1.pdf", programs "LinearQueue_Array.c" and "Linked QUEUE.C" (Module 2 LL Prog folder).*

4.1 Definition and Concept

Definition (exam-ready):

> **A queue is a linear data structure (and an Abstract Data Type) in which elements are inserted at one end called the REAR and deleted from the other end called the FRONT. The element inserted first is removed first, so a queue follows the First-In-First-Out (FIFO) principle.**

Simple explanation: a line at a ticket counter. New people join at the back (rear); the person at the front is served and leaves first.

[SOURCE] **Real-world examples:** queue at the airport, queue (of vehicles) at a signal. A queue is a restrictive data structure because operations are limited to the front and rear ends.

```
    front                rear
      |                    |
      v                    v
    [10]  [20]  [30]  (next element enters here)
<- deletion                        insertion <-
```

4.2 Queue ADT

[SOURCE]

Operation	Description
enqueue(x)	Insert x at the rear end of the queue
dequeue()	Delete one element from the front of the queue
front()	Read the element at the front (without deleting)
isFull()	Check whether the queue is full
isEmpty()	Check whether the queue is empty
size()	Number of elements present in the queue

OVERFLOW : enqueue on a full queue. UNDERFLOW : dequeue on an empty one.

4.3 Implementation Choices

[SOURCE] Like stacks, queues are implemented with

1. Arrays (static) - when the maximum size is known ahead of time.
2. Linked lists (dynamic) - when the maximum size is not known.

4.4 Array Representation of Queue

[SOURCE]

- FRONT holds the index of the front element; REAR the index of the last element; MAX is the capacity.

```
#define MAX 20
int queue[MAX], front = -1, rear = -1;
```

- **Empty:** FRONT = -1 (or FRONT > REAR after deletions).

- **Full :** REAR = MAX - 1.

Enqueue sequence [SOURCE slides 9-10], int queue[5]:

```
Initial:  front = rear = -1          [  ][  ][  ][  ][  ]
Insert 10 (empty queue):              [10][  ][  ][  ][  ]   front=0 rear=0
Insert 20:                            [10][20][  ][  ][  ]   front=0 rear=1
Insert 30:                            [10][20][30][  ][  ]   front=0 rear=2
Insert 40:                            [10][20][30][40][  ]   front=0 rear=3
Insert 50:                            [10][20][30][40][50]   front=0 rear=4
Insert 60: rear = MAX-1 = 4 -> OVERFLOW
```

Dequeue sequence [SOURCE slide 12], starting from [10][20][30], front=0, rear=2:

> **Delete -> 10 removed:** [ ][20][30][ ][ ] front=1 rear=2 Delete -> 20 removed: [ ][ ][30][ ][ ] front=2 rear=2 (front=rear) Delete -> 30 removed: [ ][ ][ ][ ][ ] front=3 rear=2 (front>rear) -> queue is empty now (front > rear).

4.5 Algorithms (Array Queue)

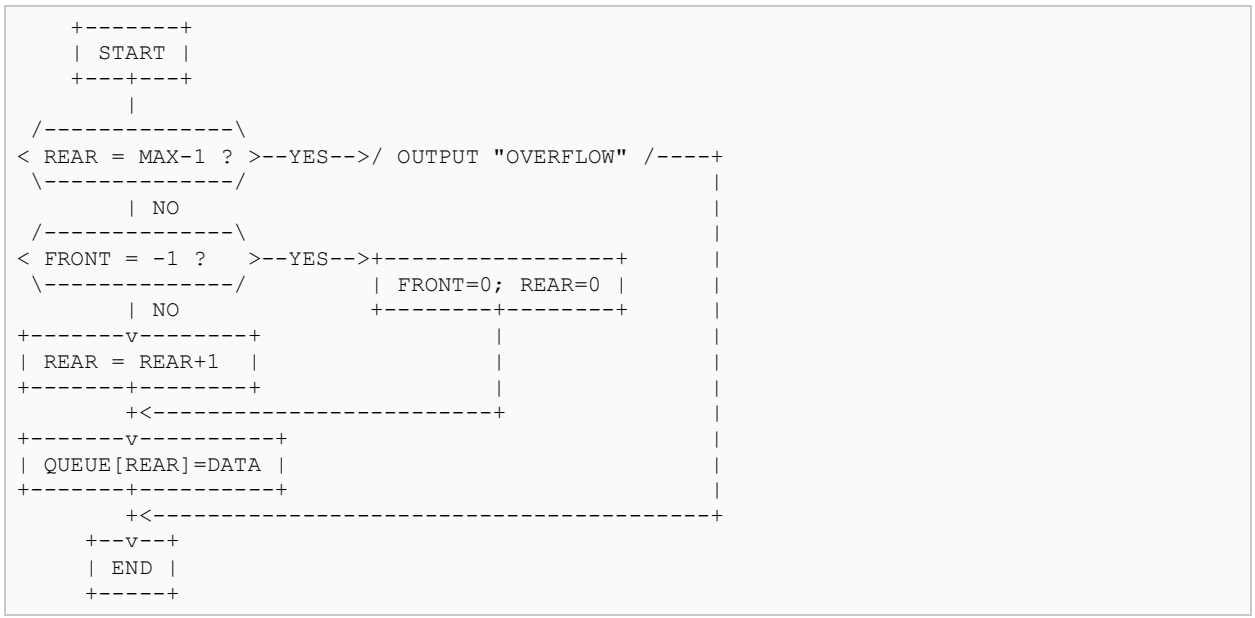
4.5.1 Enqueue [SOURCE]

```
Step 1: IF REAR = MAX-1 THEN PRINT "OVERFLOW" and STOP
Step 2: IF FRONT = -1 AND REAR = -1
           SET FRONT = REAR = 0
        ELSE
           SET REAR = REAR + 1
Step 3: SET QUEUE[REAR] = DATA
Step 4: END
```

4.5.2 Dequeue [SOURCE]

```
Step 1: IF FRONT = -1 OR FRONT > REAR THEN PRINT "UNDERFLOW" and STOP
Step 2: SET DATA = QUEUE[FRONT]
Step 3: SET FRONT = FRONT + 1
Step 4: RETURN DATA
Step 5: END
```

4.5.3 Flowchart of ENQUEUE [SUPPLEMENT]



4.6 Program: Linear Queue Using Array (LinearQueue_Array.c)

Problem statement: Write a menu-driven C program to implement a linear queue using an array.

Program: linear_queue.c

```
/* Linear queue using array */
#include <stdio.h>
#define MAX 3

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)          /* first insertion */
        front = 0;
    rear = rear + 1;
    queue[rear] = value;
}

int dequeue(void)
{
    int value;
    if (front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
        return -1;
    }
    value = queue[front];
    front = front + 1;
    return value;
}

void display(void)
{
    int i;
    if (front == -1 || front > rear)
    {
        printf("Queue Empty\n");
        return;
    }
    for (i = front; i <= rear; i++)
        printf("%d\t", queue[i]);
    printf("\n");
}

int main(void)
{
    int choice, num, temp;
    do
    {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit");
        printf("\nEnter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice)
        {
        case 1:
            printf("Enter element to insert: ");
            scanf("%d", &num);
            enqueue(num);
            break;
        case 2:
            temp = dequeue();
            if (temp != -1)
                printf("Element deleted=%d\n", temp);
            break;
        case 3:
```

```
            display();
            break;
        }
    } while (choice != 4);
    return 0;
}
```

Code explanation:

- MAX = 3 (as in your program) so overflow is easy to test.
- enqueue(): overflow if rear == MAX-1. On the very first insertion front changes from -1 to 0. Then rear++ and store.
- dequeue(): underflow if front == -1 (never used) or front > rear (all elements deleted). Otherwise read queue[front] and front++.
- display(): prints queue[front..rear].
- **Change from the notes:** int main(void) instead of void main(), and the menu loop ends on choice 4 instead of asking "Do u wish to continue?". The queue logic is identical to your program.

Actual sample run (MAX = 3):

```
Enqueue 10, 20, 30          -> stored
Enqueue 40                  -> Queue Overflow
Display                     -> 10 20 30
Dequeue, Dequeue            -> Element deleted=10 / Element deleted=20
Display                     -> 30
Enqueue 50                  -> Queue Overflow   <-- although 2 slots
                                                    are free!
Dequeue                     -> Element deleted=30
Dequeue                     -> Queue Underflow
Enqueue 60                  -> Queue Overflow   (rear is still 2)
```

Dry run table:

Operation	front	rear	queue[0..2]	Output
start	-1	-1	_ _ _	
enq 10	0	0	10 _ _	
enq 20	0	1	10 20 _	
enq 30	0	2	10 20 30	
enq 40	0	2	10 20 30	Queue Overflow
deq	1	2	(10) 20 30	Element deleted=10
deq	2	2	(10)(20) 30	Element deleted=20
enq 50	2	2	unchanged	Queue Overflow (drawback)

This demonstrates the DRAWBACK OF A LINEAR QUEUE (Chapter 5): freed slots at the front cannot be reused because REAR only moves forward.

Complexity: enqueue O(1), dequeue O(1), display O(n). Space O(MAX).

4.7 Linked Representation of Queue

[SOURCE]

- Each node has DATA and the address of the NEXT node.
- FRONT points to the first node; REAR points to the last node.
- Insertion at REAR, deletion at FRONT.
- FRONT = REAR = NULL means the queue is empty.

[SOURCE] Initialisation:

```
struct node { int data; struct node *next; };
typedef struct node Node;
Node *front, *rear;
front = rear = NULL;

Enqueue(10) on an empty queue:
   newNode = (Node*)malloc(sizeof(Node));
   newNode->data = 10; newNode->next = NULL;
   if (front == NULL && rear == NULL) front = rear = newNode;

front -> [10 | X] <- rear
```

[CORRECTION] The slide writes malloc(sizeof(Node); - the closing bracket is missing; it must be malloc(sizeof(Node)); "Struct" and "If" must be lowercase in C.

4.7.1 ENQUEUE (linked) [SOURCE]

```
Step 1: IF FRONT = NULL AND REAR = NULL
           SET FRONT = REAR = newNode
        ELSE
           SET REAR->NEXT = newNode
           SET REAR = newNode
Step 2: END
(newNode is allocated with DATA set and NEXT = NULL before Step 1.)
```

```
front->[10|X]<-rear   insert 20:   front->[10]->[20|X]<-rear
insert 30:            front->[10]->[20]->[30|X]<-rear
(The slide labels the third node "20" by mistake; it should be 30.)
```

4.7.2 DEQUEUE (linked) [SOURCE with corrections]

```
Step 1: IF FRONT = NULL THEN PRINT "UNDERFLOW" and STOP
Step 2: SET TEMP = FRONT, SET VALUE = FRONT->DATA
Step 3: SET FRONT = FRONT->NEXT
Step 4: IF FRONT = NULL THEN SET REAR = NULL
Step 5: FREE TEMP
Step 6: RETURN VALUE
```

[CORRECTION] The slide's version (a) frees "front" in the single-node case but never frees TEMP in the general case (memory leak), and (b) jumps from Step 2 to Step 4. The version above handles both cases with one set of steps.

```
front->[10]->[20]->[30|X]<-rear
delete: returns 10      front->[20]->[30|X]<-rear
delete: returns 20      front->[30|X]<-rear
delete: returns 30      front = rear = NULL (empty)
```

4.8 Program: Queue Using Linked List (Linked QUEUE.C, corrected)

Program: linked_queue.c

```
/* Queue using singly linked list */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};
typedef struct node Node;

Node *front = NULL, *rear = NULL;

int isEmpty(void)
{
```

```
    return (front == NULL);
}

void enqueue(int value)
{
    /* sizeof(Node), not sizeof(Node*) */
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL)
    {
        printf("Memory not available\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;
    if (isEmpty())
        front = rear = newNode;
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
}

int dequeue(void)
{
    Node *temp;
    int value;
    if (isEmpty())
    {
        printf("Queue Underflow\n");
        return -1;
    }
    temp = front;
    value = temp->data;
    front = front->next;
    if (front == NULL)        /* queue became empty */
        rear = NULL;
    free(temp);
    return value;
}

void display(void)
{
    Node *t = front;
    if (isEmpty())
    {
        printf("Queue Empty\n");
        return;
    }
    printf("Queue Elements: ");
    while (t != NULL)
    {
        printf("%d\t", t->data);
        t = t->next;
    }
    printf("\n");
}

int main(void)
{
    int ch, value;
    do
    {
        printf("\n1.Insert 2.Remove 3.Display 4.Exit\nEnter choice: ");
        if (scanf("%d", &ch) != 1)
            break;
        switch (ch)
        {
        case 1:
            printf("Enter the data: ");
            scanf("%d", &value);
            enqueue(value);
            break;
        case 2:
```

```
            value = dequeue();
            if (value != -1)
                printf("Element Removed = %d\n", value);
            break;
        case 3:
            display();
            break;
        }
    } while (ch != 4);
    while (!isEmpty())
        dequeue();
    return 0;
}
```

Code explanation:

- front and rear are global Node pointers, both NULL initially.
- enqueue(): creates a node with next = NULL. If the queue is empty both front and rear point to it; else it is linked after rear and becomes the new rear. O(1) because we never traverse.
- dequeue(): saves front in temp, moves front forward, resets rear to NULL when the queue becomes empty, frees temp.
- display(): walks from front to NULL.

[CORRECTION] Bugs in the notes' "Linked QUEUE.C":

1. Node *t = (Node*)malloc(sizeof(Node*)); allocates the size of a POINTER (8 bytes on 64-bit), not the size of a node (16 bytes). Writing t->next then writes outside the allocated block. AddressSanitizer reports "heap-buffer-overflow" for the original. Fix: sizeof(Node).
2. <stdlib.h> was not included for malloc/free.
3. rear was not reset to NULL when the last node was deleted (harmless in that program because isEmpty() checks only f, but unsafe in general).
4. The menu printed "3.Display Stack" - it is a queue.

Actual sample run (matches the notes' output):

```
Insert 5, 6, 7 -> Display -> Queue Elements: 5 6 7
Remove         -> Element Removed = 5
Display        -> 6 7
Insert 8       -> Display -> 6 7 8
Remove x3      -> 6, 7, 8 removed; next Remove -> Queue Underflow
```

Complexity: enqueue O(1) (thanks to the rear pointer), dequeue O(1), display O(n). Space O(n) plus one pointer per node.

4.9 Stack vs Queue

[SOURCE + SUPPLEMENT]

Point	Stack	Queue
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Ends used	One end: TOP	Two ends: REAR (insert) and FRONT (delete)
Insert / delete	push / pop	enqueue / dequeue
Pointers/indices	top	front and rear
Empty (array)	top = -1	front = -1 or front > rear
Full (array)	top = MAX-1	rear = MAX-1 (linear)
Example	pile of plates	line at a counter
Applications	expression conversion and evaluation, recursion, function calls, undo	printer spooling, CPU scheduling, buffers
Time of insert/del	O(1)	O(1)

4.10 Chapter 4 Exam Preparation

Quick Revision

- Queue = FIFO; insert at REAR, delete at FRONT.
- **Array:** front = rear = -1 initially; first insert sets both to 0. Full: rear = MAX-1. Empty: front = -1 or front > rear.
- **Linked:** front = rear = NULL empty; enqueue at rear; dequeue at front; reset rear when the queue becomes empty; free the deleted node.
- Linear array queue wastes front slots - fixed by circular queue.

Important Definitions

- **Queue:** linear data structure with insertion at rear and deletion at front, following FIFO.
- **Enqueue:** insert at rear. Dequeue: delete from front.
- **Queue overflow:** insert into a full queue. Underflow: delete from an empty queue.

Complexity Summary

Operation	Array (linear)	Linked (with rear)
enqueue	O(1)	O(1)
dequeue	O(1)	O(1)
front()	O(1)	O(1)
display	O(n)	O(n)

Common Mistakes

- Checking overflow with front instead of rear.
- Forgetting to set front = 0 on the first insertion (front stays -1).
- Using only "front == -1" as the empty test; after deletions the queue is empty when front > rear.
- malloc(sizeof(Node*)) instead of malloc(sizeof(Node)).
- **Linked queue:** not setting rear = NULL when the last node is removed.

Practice Questions

1. [S] Define queue. Explain the queue ADT. Ans: 4.1, 4.2.
2. [S] Write enqueue and dequeue algorithms for an array queue. Ans: 4.5.

3. [S] For int queue[5], show front, rear and contents after inserting 10, 20, 30, 40, 50 and then deleting twice.

Ans: after inserts front=0 rear=4; after two deletes front=2 rear=4, elements 30 40 50.

4. [S] Write algorithms for enqueue and dequeue on a linked queue.

Ans: 4.7.1, 4.7.2.

5. [S] Write a C program to implement a queue using a linked list.

Ans: Program 4.8.

6. [S] Differentiate stack and queue. Ans: Table 4.9.

7. [A] MAX = 3. Trace: enq 1, enq 2, deq, enq 3, enq 4, deq, deq, deq.

Ans: enq4 -> Overflow (rear=2); deq returns 1, then 2, then 3; last deq -> Underflow. Final front=3, rear=2.

8. [A] What is wrong with `Node *t = malloc(sizeof(Node*));` ?

Ans: it reserves memory for a pointer, not a node; writing t->next overflows the block. Use sizeof(Node).

9. [A] Why is a rear pointer needed in a linked queue?

Ans: without it every enqueue must traverse to the last node - O(n); with it enqueue is O(1).

Chapter 5: Types of Queues and Applications

Sources: *"Queue-2.pdf" (types, circular queue, priority queue), "Queue-3.pdf" (deque, applications), programs "CircularQueue.txt", "priorityQueue.docx", "DequeProgramUsingArray.pdf".*

5.1 Types of Queues

[SOURCE]

1. Linear queue (Chapter 4)
2. Circular queue
3. Priority queue
4. Double-ended queue (deque)

5.2 Drawback of Linear Queue

[SOURCE] The full condition of a linear queue is REAR = MAX-1.

```
int queue[5]; insert 10, 20, 30, 40, 50:
     [10][20][30][40][50]     front = 0, rear = 4
delete 2 elements:
     [  ][  ][30][40][50]     front = 2, rear = 4
Now REAR = MAX-1, so the queue reports FULL even though index 0 and 1 are
free. "We cannot insert the next element even if there is space in front
of the queue." Memory is wasted.
```

5.3 Circular Queue

Definition (exam-ready):

> **A circular queue is a queue in which the last position is logically connected back to the first position, so that after index MAX-1 the next position is index 0. This allows the empty slots at the front (left by deletions) to be reused.**

Working principle: indices advance with the modulo operator:

```
   rear  = (rear + 1)  % SIZE
   front = (front + 1) % SIZE
When rear = SIZE-1, (rear+1) % SIZE = 0, so it "wraps around".
```

[SOURCE] **Using the space at the front (SIZE 5):**

```
[  ][  ][30][40][50]  front=2 rear=4
insert 60 -> rear = (4+1)%5 = 0
[60][  ][30][40][50]  front=2 rear=0

Drawn as a circle:
              [0]=60
        [4]=50     [1]=empty
        [3]=40     [2]=30 <- front
(rear = 0)
```

[SOURCE] **Operations example from the slide (SIZE 6):**

```
After inserting 18, 7, 42, 67:
   index: 0   1   2   3   4   5
         [18][ 7][42][67][  ][  ]    front=0, rear=3
After removing 18, 7:
         [  ][  ][42][67][  ][  ]    front=2, rear=3
After inserting 30, 47, 14:
         [14][  ][42][67][30][47]    front=2, rear=0
  (30 -> index 4, 47 -> index 5, 14 -> (5+1)%6 = index 0)
```

5.3.1 Initialisation and conditions [SOURCE]

```
   #define SIZE 5
   int queue[SIZE];
   int count = 0;     /* number of elements */
   int front = 0;
   int rear  = -1;
Empty: count == 0         Full: count == SIZE
Using a count variable is the simplest way to tell "full" from "empty",
because in both cases front and rear can be at the same relative
positions.
```

[SUPPLEMENT] **Another common method without count:** front = rear = -1 initially; full when (rear+1) % SIZE == front; empty when front == -1. Know that it exists, but the notes use the count method.

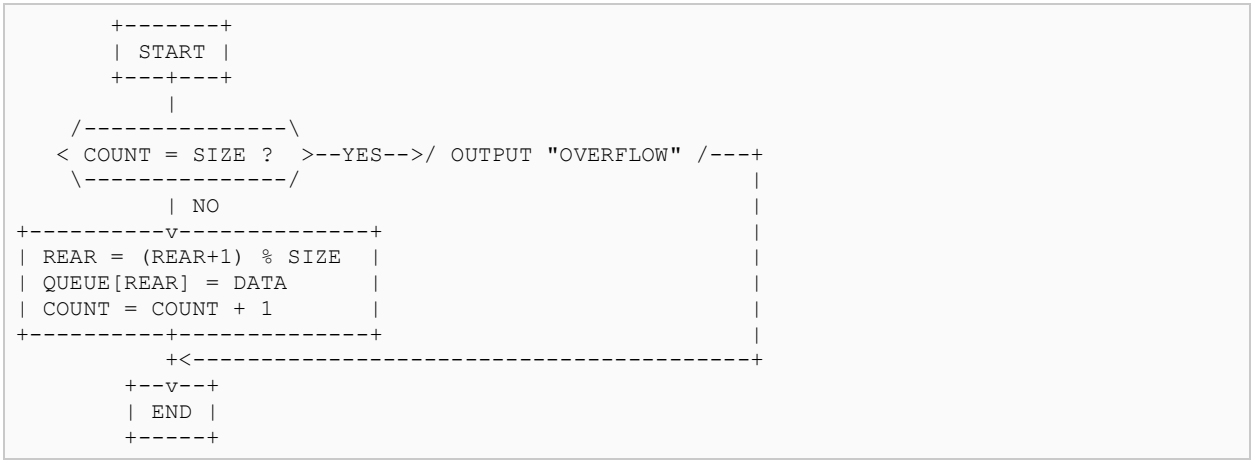
5.3.2 ENQUEUE algorithm [SOURCE]

```
Step 1: IF COUNT = SIZE THEN PRINT "OVERFLOW" and STOP
Step 2: SET REAR = (REAR + 1) % SIZE
Step 3: SET QUEUE[REAR] = DATA
Step 4: SET COUNT = COUNT + 1
Step 5: END
```

5.3.3 DEQUEUE algorithm [SOURCE]

```
Step 1: IF COUNT = 0 THEN PRINT "UNDERFLOW" and STOP
Step 2: SET DATA = QUEUE[FRONT]
Step 3: SET FRONT = (FRONT + 1) % SIZE
Step 4: SET COUNT = COUNT - 1
Step 5: RETURN DATA
```

5.3.4 Flowchart of circular ENQUEUE [SUPPLEMENT]



5.4 Program: Circular Queue Using Array (CircularQueue.txt)

Program: circular_queue.c

```
/* Circular queue using array (count method) */
#include <stdio.h>
#define MAX 3

int cqueue[MAX];
int front = 0, rear = -1, count = 0;

void enqueue(int value)
{
    if (count == MAX)
    {
        printf("Queue Overflow\n");
        return;
```

```
    }
    rear = (rear + 1) % MAX;      /* wrap around to index 0 after MAX-1 */
    cqueue[rear] = value;
    count++;
}

int dequeue(void)
{
    int value;
    if (count == 0)
    {
        printf("Queue Underflow\n");
        return -1;
    }
    value = cqueue[front];
    front = (front + 1) % MAX;
    count--;
    return value;
}

void display(void)
{
    int i, k;
    if (count == 0)
    {
        printf("Queue Empty\n");
        return;
    }
    k = front;
    for (i = 0; i < count; i++)
    {
        printf("%d\t", cqueue[k]);
        k = (k + 1) % MAX;
    }
    printf("\n");
}

int main(void)
{
    int choice, num, temp;
    do
    {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit");
        printf("\nEnter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice)
        {
        case 1:
            printf("Enter element to insert: ");
            scanf("%d", &num);
            enqueue(num);
            break;
        case 2:
            temp = dequeue();
            if (temp != -1)
                printf("Element deleted=%d\n", temp);
            break;
        case 3:
            display();
            break;
        }
    } while (choice != 4);
    return 0;
}
```

Code explanation:

- front = 0, rear = -1, count = 0 as in the slides. (Your .txt program starts with front = -1 and sets it to 0 on the first insert; both work. This version follows the slide initialisation, so no special first- insert case is needed.)
- enqueue(): full test uses count; rear moves with % MAX.

- dequeue(): empty test uses count; front moves with % MAX.
- display(): starts at front and prints count elements, wrapping k with % MAX.

Actual sample run (MAX = 3):

```
enq 10, 20, 30 ; enq 40    -> Queue Overflow
deq, deq                   -> Element deleted=10 / 20
enq 40, enq 50             -> stored at index 0 and 1 (wrap-around)
display                    -> 30 40 50
deq x3                     -> 30, 40, 50 ; next deq -> Queue Underflow
```

Dry run:

Op	front	rear	count	cqueue[0..2]	Note
start	0	-1	0	_ _ _	
enq 10	0	0	1	10 _ _	rear=(-1+1)%3=0
enq 20	0	1	2	10 20 _	
enq 30	0	2	3	10 20 30	
enq 40	0	2	3	10 20 30	count=3=MAX: Overflow
deq	1	2	2	-- 20 30	returns 10
deq	2	2	1	-- -- 30	returns 20
enq 40	2	0	2	40 -- 30	rear=(2+1)%3=0 WRAP
enq 50	2	1	3	40 50 30	full again
display					k=2,0,1 -> 30 40 50

Complexity: enqueue O(1), dequeue O(1), display O(n). Space O(SIZE).

Advantages: reuses freed space; all operations still O(1).

Limitation: size still fixed at compile time.

5.5 Linear Queue vs Circular Queue

Point	Linear queue	Circular queue
Arrangement	Straight line; ends separate	Last index connected to first
Index update	rear = rear + 1	rear = (rear+1) % SIZE
Full condition	rear = MAX-1	count = SIZE (or (rear+1)%SIZE = front)
Memory use	Front slots wasted after deletions	Freed slots reused
Complexity	O(1) enqueue/dequeue	O(1) enqueue/dequeue

5.6 Priority Queue

Definition [SOURCE]:

> **A priority queue is a data structure in which each element is assigned a priority, and the priority decides the order in which elements are processed.**

Rules [SOURCE]:

1. An element with higher priority is processed before an element with lower priority.
2. Two elements with the same priority are processed on a first-come- first-served (FCFS) basis.

Types [SOURCE]:

- **Ascending priority queue:** elements can be inserted in any order, but the SMALLEST element is always deleted first.

- **Descending priority queue:** elements can be inserted in any order, but the LARGEST element is always deleted first.

Example (ascending): insert 30, 10, 50, 20 -> deletions give 10, 20, 30, 50.

Implementation ideas [SUPPLEMENT]:

Method	Insert	Delete highest priority
Unsorted array	O(1) at rear	O(n) search + shift
Sorted array (notes' way)	O(n) shift	O(1) from front
Sorted linked list	O(n) find place	O(1) from front

Applications [SUPPLEMENT]: CPU scheduling by priority, handling interrupts, emergency room triage, shortest-path algorithms.

5.7 Program: Ascending Priority Queue Using Array (corrected)

(source: priorityQueue.docx)

[SOURCE idea] The notes' program inserts an element at the rear and then SORTS the whole array, so the smallest element is always at the front and is deleted first.

[CORRECTION] The notes' program does not compile and its sort is wrong:

1. main() is missing its closing brace (the while loop's brace closes, the function's does not).
2. c is used in while(c != 4) before it is given a value.
3. Bubble-sort bound: for (j = f; j < (r - i - 1); j++) does one comparison too few. With 2 elements (f=0, r=1) the inner loop never runs, so the queue is not sorted. When f > 0 the bounds are also wrong. (An insertion-sort version is commented out in the file.)
4. On underflow it still prints "Element deleted is -1".
5. It calls sort() even after an overflow.

The corrected program keeps the same idea (array kept in ascending order, delete from the front) but inserts each new element directly into its sorted position, which is simpler and O(n) per insert instead of O(n^2).

Program: priority_queue.c

```
/* Ascending priority queue using a sorted array
   (smallest value = highest priority, deleted first) */
#include <stdio.h>
#define MAX 20

int q[MAX];
int f = 0, r = -1;

/* Insert so that q[f..r] always stays in ascending order */
void insertq(int e)
{
    int j;
    if (r == MAX - 1)
    {
        printf("Overflow\n");
        return;
    }
    j = r;
    /* shift larger elements one place right */
    while (j >= f && q[j] > e)
    {
        q[j + 1] = q[j];
        j--;
    }
    q[j + 1] = e;
    r++;
}
```

```
int deleteq(void)
{
    if (f > r)
    {
        printf("Underflow\n");
        return -1;
    }
    return q[f++];                /* front always holds the smallest */
}

void display(void)
{
    int i;
    if (f > r)
    {
        printf("Queue Empty\n");
        return;
    }
    printf("Sorted Queue is: ");
    for (i = f; i <= r; i++)
        printf("%d ", q[i]);
    printf("\n");
}

int main(void)
{
    int c = 0, n, val;
    while (c != 4)
    {
        printf("\n1:insert 2:delete 3:display 4:exit\nEnter choice: ");
        if (scanf("%d", &c) != 1)
            break;
        switch (c)
        {
        case 1:
            printf("Enter element: ");
            scanf("%d", &n);
            insertq(n);
            break;
        case 2:
            val = deleteq();
            if (val != -1)
                printf("Element deleted is %d\n", val);
            break;
        case 3:
            display();
            break;
        }
    }
    return 0;
}
```

Code explanation:

- q[f..r] is always sorted in ascending order; f = front, r = rear.
- insertq(e): start at j = r and move every element larger than e one place right, then put e in the gap at j+1. Using ">" (not ">=") puts a new element AFTER equal ones, so equal priorities stay FCFS.
- deleteq(): returns q[f] (the smallest) and moves f forward.
- This is an ascending priority queue. For descending, change q[j] > e to q[j] < e.

Dry run of insert 30, 10, 50, 20:

```
insert 30: q = [30]                    f=0 r=0
insert 10: 30 > 10 -> shift; q = [10, 30]              r=1
insert 50: 30 > 50? no -> q = [10, 30, 50]             r=2
insert 20: 50 shifts, 30 shifts, 10 > 20? no -> q = [10, 20, 30, 50]
delete -> 10, delete -> 20, remaining [30, 50]
```

Actual sample run: insert 30,10,50,20,20 -> "Sorted Queue is: 10 20 20 30 50"; delete -> 10; delete -> 20; insert 5 -> "5 20 30 50"; deletes give 5, 20, 30, 50.

Complexity: insert O(n) (shifting), delete O(1), space O(MAX).

Limitation: like the linear queue, slots before f are not reused.

5.8 Program: Priority Queue Using Linked List [SUPPLEMENT]

The Queue-2 learning objective says "Develop program to implement Circular Queue ADT and Priority Queue ADT using array and linked list". The notes have no linked priority queue program, so here is one. Each node stores a data value and a priority number; a smaller number means higher priority.

Program: pq_linked.c

```
/* Priority queue using a sorted linked list.
   Lower priority number = higher priority (served first).
   Equal priorities are served first-come-first-served. */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    int priority;
    struct node *next;
};

struct node *front = NULL;

void insertPQ(int data, int priority)
{
    struct node *ptr, *temp;
    ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr == NULL)
    {
        printf("Overflow\n");
        return;
    }
    ptr->data = data;
    ptr->priority = priority;
    if (front == NULL || priority < front->priority)
    {
        /* new highest priority goes to the front */
        ptr->next = front;
        front = ptr;
    }
    else
    {
        temp = front;
        /* skip nodes with priority <= new one (keeps FCFS) */
        while (temp->next != NULL && temp->next->priority <= priority)
            temp = temp->next;
        ptr->next = temp->next;
        temp->next = ptr;
    }
}

int deletePQ(void)
{
    struct node *temp;
    int data;
    if (front == NULL)
    {
        printf("Underflow\n");
        return -1;
    }
    temp = front;
    data = temp->data;
    front = front->next;
```

```
    free(temp);
    return data;
}

void display(void)
{
    struct node *temp = front;
    if (temp == NULL)
    {
        printf("Queue is empty\n");
        return;
    }
    printf("Data(Priority): ");
    while (temp != NULL)
    {
        printf("%d(%d) ", temp->data, temp->priority);
        temp = temp->next;
    }
    printf("\n");
}

int main(void)
{
    insertPQ(100, 2);
    insertPQ(200, 1);
    insertPQ(300, 3);
    insertPQ(400, 2);
    display();
    printf("Deleted %d\n", deletePQ());
    printf("Deleted %d\n", deletePQ());
    display();
    while (front != NULL)
        deletePQ();
    return 0;
}
```

Code explanation:

- The list is kept sorted by priority, so the front node is always the next to be served.
- insertPQ(): if the list is empty or the new priority is better than the front's, insert at the front. Otherwise move temp forward while the NEXT node's priority is <= the new priority, and insert after temp. The "<=" places the new node after all equal-priority nodes (FCFS).
- deletePQ(): removes the front node like a normal queue.

Actual output:

```
Data(Priority): 200(1) 100(2) 400(2) 300(3)
Deleted 200
Deleted 100            (100 and 400 have equal priority; 100 came first)
Data(Priority): 400(2) 300(3)
```

Complexity: insert O(n), delete O(1), space O(n).

5.9 Double-Ended Queue (Deque)

Definition [SOURCE]:

> **A deque (double-ended queue) is a linear list in which insertion and deletion can be performed at EITHER end - front or rear. The notes add that it is "also known as a head-tail linked list".**

```
insertion ->                              <- insertion
            [  ][  ][  ] ... [  ][  ]
<- deletion  front                 rear  deletion ->
```

Types [SOURCE]:

1. Input-restricted deque: insertion at ONLY ONE end (rear), deletion from BOTH ends. Operations: add at rear, delete from front, delete from rear.
2. Output-restricted deque: deletion at ONLY ONE end (front), insertion at BOTH ends. Operations: add at rear, add at front, delete from front.

[SOURCE] Example figure:

```
Input restricted deque:
                                insertion
                                   |
                                   v
       [34][12][53][61][ 9]
         ^                  ^
       front              rear
         |                  |
      deletion          deletion

Output restricted deque:
    insertion                       insertion
        |                               |
        v                               v
       [34][12][53][61][ 9]
         ^ front           rear
         |
      deletion
```

[SOURCE] Conditions:

- Inserting into a full deque -> OVERFLOW.
- Deleting from an empty deque -> UNDERFLOW.

Note [SUPPLEMENT]: a deque can behave as a stack (insert and delete at the same end) or as a queue (insert at rear, delete at front), so it generalises both.

5.10 Program: Deque Using Circular Array (corrected)

(source: DequeProgramUsingArray.pdf)

Program: deque.c

```
/* Double ended queue (deque) using circular array */
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = 0, rear = -1;
int count = 0;

int isEmpty(void)
{
    return (count == 0);
}

int isFull(void)
{
    return (count == SIZE);
}

void insertFront(int data)
{
    /* first element: front and rear meet */
    if (isEmpty())
    {
        front = 0;
        rear = 0;
    }
    else
```

```
        /* +SIZE avoids a negative index */
        front = (front - 1 + SIZE) % SIZE;
    queue[front] = data;
    count++;
}

void insertRear(int data)
{
    if (isEmpty())
    {
        front = 0;
        rear = 0;
    }
    else
        rear = (rear + 1) % SIZE;
    queue[rear] = data;
    count++;
}

int deleteFront(void)
{
    int data = queue[front];
    front = (front + 1) % SIZE;
    count--;
    return data;
}

int deleteRear(void)
{
    int data = queue[rear];
    rear = (rear - 1 + SIZE) % SIZE;
    count--;
    return data;
}

void display(void)
{
    int i, p;
    printf("Elements in Deque: ");
    p = front;
    for (i = 1; i <= count; i++)
    {
        printf("%d ", queue[p]);
        p = (p + 1) % SIZE;
    }
    printf("\n");
}

int main(void)
{
    int choice, num;
    do
    {
        printf("\n1.Insert Front 2.Delete Front 3.Insert Rear");
        printf("\n4.Delete Rear 5.Display 6.Exit\nEnter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice)
        {
        case 1:
        case 3:
            if (isFull())
                printf("Queue Overflow\n");
            else
            {
                printf("Enter element to insert: ");
                scanf("%d", &num);
                if (choice == 1)
                    insertFront(num);
                else
                    insertRear(num);
                printf("Element inserted successfully\n");
            }
            break;
```

```
        case 2:
        case 4:
            if (isEmpty())
                printf("Queue Underflow\n");
            else if (choice == 2)
                printf("Element deleted=%d\n", deleteFront());
            else
                printf("Element deleted=%d\n", deleteRear());
            break;
        case 5:
            if (isEmpty())
                printf("Queue Empty\n");
            else
                display();
            break;
        }
    } while (choice != 6);
    return 0;
}
```

Code explanation:

- The array is used circularly, with count for full/empty tests, like the circular queue.
- insertFront(): moves front one step BACKWARD: (front - 1 + SIZE) % SIZE and stores there.
- insertRear(): moves rear one step FORWARD: (rear + 1) % SIZE.
- deleteFront(): reads queue[front], front moves forward.
- deleteRear(): reads queue[rear], rear moves backward: (rear - 1 + SIZE) % SIZE.
- When the deque is empty, the first insertion (from either end) sets front = rear = 0, so both ends refer to the single element.
- main() checks isFull()/isEmpty() before calling the functions, exactly as your program does.

[CORRECTION] **Critical bug in the notes' program:**

```
  front = (front - 1) % SIZE;    and    rear = (rear - 1) % SIZE;
In C, the % of a negative number is negative: (0 - 1) % 5 = -1. So the
first InsertFront writes queue[-1], outside the array (undefined
behaviour). Running the original under a checker reports "index -1 out
of bounds". The printed output "5 10 20 30" in the notes appears only
by accident: InsertFront writes queue[-1] and then queue[-2] (memory
outside the array), and display() happens to read those same places
back. Undefined behaviour can crash or corrupt other variables on
another compiler, so the code is still wrong. Fix: add SIZE before taking %:
  front = (front - 1 + SIZE) % SIZE;
Second fix: when the deque is empty, insertFront and insertRear must make
front and rear refer to the same slot; the original starts with front=0,
rear=-1 and handles this inconsistently.
```

Actual sample run (same inputs as the notes' output):

```
Insert Front 10 ; Insert Rear 20 ; Insert Front 5 ; Insert Rear 30
Display        -> Elements in Deque: 5 10 20 30
Delete Front   -> Element deleted=5
Delete Rear    -> Element deleted=30
Display        -> Elements in Deque: 10 20
```

Dry run (SIZE = 5):

Operation	front	rear	count	queue[0..4]	Note
start	0	-1	0	_ _ _ _ _	
InsFront 10	0	0	1	10 _ _ _ _	empty case
InsRear 20	0	1	2	10 20 _ _ _	
InsFront 5	4	1	3	10 20 _ _ 5	(0-1+5)%5 = 4
InsRear 30	4	2	4	10 20 30 _ 5	
Display	4	2	4	unchanged	prints from index 4: 5 10 20 30
DelFront	0	2	3	10 20 30 _ --	returns 5
DelRear	0	1	2	10 20 -- _ --	returns 30
Display	0	1	2	unchanged	prints 10 20

Complexity: all four insert/delete operations O(1); display O(n).

5.11 Comparison of Queue Types

Type	Insert at	Delete from	Key feature
Linear queue	rear	front	FIFO; wastes front space
Circular queue	rear (wraps)	front (wraps)	reuses space
Priority queue	any order (placed by priority)	highest-priority element	order by priority
Deque	front or rear	front or rear	both ends
Input-restricted deque	rear only	front or rear	
Output-restrict. deque	front or rear	front only	

5.12 Applications of Queue

[SOURCE] **Applications:** serving requests (printer spooler), CPU time-sharing systems, buffers, business online applications.

5.12.1 Serving requests (printer spooler) [SOURCE]

Many processes often need a single shared resource. When several people share a printer and print requests arrive faster than the printer can finish them, the requests are placed in a queue to preserve their order. New requests join at the end; when the printer finishes a job it takes the request at the front. Large shared disks are served the same way.

5.12.2 Round-robin scheduling (CPU time sharing) [SOURCE]

Every process gets a fixed amount of CPU time called a TIME QUANTUM or TIME SLICE. If a process is still running when its quantum runs out, it is suspended and put at the END of the queue.

```
CPU <- [P1][P2][P3][P4][P5]      P1 runs for one quantum
CPU <- [P2][P3][P4][P5][P1]      P1 unfinished, moved to the end
                                 (a circular queue is ideal here)
```

5.12.3 Buffers [SOURCE]

Queues store data transferred ASYNCHRONOUSLY between two processes (sender and receiver are not synchronised - the receiver may not be ready at the speed the sender sends). The sender

enqueues whenever it likes; the receiver dequeues whenever it likes. Such a queue is called a buffer (for example keyboard buffer, I/O buffers).

5.13 Chapter 5 Exam Preparation

Quick Revision

- **Linear queue drawback:** rear = MAX-1 means "full" even with free front slots.
- **Circular queue:** rear = (rear+1)%SIZE; front = (front+1)%SIZE; count decides full (count = SIZE) / empty (count = 0).
- **Priority queue:** higher priority first; equal priority FCFS. Ascending = smallest first; descending = largest first.
- **Deque:** insert/delete at both ends. Input-restricted: insert one end. Output-restricted: delete one end.
- **Backward move in a circular array:** (i - 1 + SIZE) % SIZE.
- **Applications:** printer spooler, round robin, buffers.

Important Definitions

- **Circular queue:** a queue whose last position is connected to the first so freed positions can be reused.
- **Priority queue:** each element has a priority; higher priority processed first, equal priority FCFS.
- **Deque:** a linear list where insertion and deletion are allowed at both ends.
- **Time quantum:** fixed CPU time given to each process in round robin.
- **Buffer:** a queue between two asynchronous processes.

Complexity Summary

Structure	Insert	Delete
Circular queue	O(1)	O(1)
Priority q. (sorted array / sorted list)	O(n)	O(1)
Priority q. (source program: sort after every insert)	O(n^2) per insert	O(1)
Deque (circular arr.)	O(1) both ends	O(1) both ends

Common Mistakes

- Writing rear = rear + 1 % SIZE (precedence: that is rear + (1 % SIZE)). Use brackets: (rear + 1) % SIZE.
- Using (i - 1) % SIZE to move backwards - gives -1 in C.
- Mixing up input-restricted and output-restricted. Remember: the RESTRICTED operation is the one named (input = insertion restricted).
- Saying a priority queue is FIFO. It is FIFO only among equal priorities.

Practice Questions

1. [S] What is the drawback of a linear queue? How does a circular queue overcome it? Ans: 5.2 and 5.3 with the diagrams.
2. [S] Write enqueue and dequeue algorithms for a circular queue.

   Ans: 5.3.2, 5.3.3.

3. [S] SIZE = 6. Show the circular queue after inserting 18, 7, 42, 67, deleting 2 elements and inserting 30, 47, 14.

Ans: [14][ ][42][67][30][47], front = 2, rear = 0 (see 5.3).

4. [S] What is a priority queue? Explain its types. Ans: 5.6.

5. [S] Explain deque and its types with diagrams. Ans: 5.9.

6. [S] Explain any three applications of queue. Ans: 5.12.

7. [A] SIZE = 4, count method, front = 0, rear = -1. Trace: enq A, enq B, enq C, deq, deq, enq D, enq E, enq F, enq G. Give front, rear, count.

Ans: after A,B,C: f=0 r=2 c=3; two deq: f=2 c=1; D -> r=3; E -> r=0; F -> r=1 (c=4, full); G -> Overflow. Final front=2, rear=1, count=4, array [E][F][C][D] with order C D E F.

8. [A] Why does the original deque program fail on its first InsertFront?

Ans: (0-1)%5 is -1 in C, so it writes queue[-1]; use (front-1+SIZE)%SIZE.

9. [A] Insert 40, 15, 25, 15 into an ascending priority queue and show the order of deletion. Ans: 15, 15 (first-inserted 15 first), 25, 40.

10. [A] Can a deque be used as a stack? As a queue?

Ans: Yes - stack: insert and delete at the same end; queue: insert at rear, delete at front.

Chapter 6: Pointers and Array vs Linked List

Sources: *Module 2 "Lecture 3 Pointers & Array vs Linked List.pdf" (= slides 16-25 of "Module 2 Linked List.pptx"). Module 2 course outcome (CO2): "Construct linked list structures using dynamic memory, ensuring system safety."*

6.1 What Is a Pointer

[SOURCE] Every variable is a memory location, and every memory location has an address, which is obtained with the ampersand (&) operator. A pointer stores this address.

Definition (exam-ready):

> **A pointer is a variable whose value is the address of another variable, i.e. the direct address of a memory location.**

Key operators [SUPPLEMENT]:

```
&x   "address of x"
*p   "value at the address stored in p" (dereference / indirection)

int x = 10;
int *p = &x;        /* p holds the address of x */

  x (at address 1000)          p (at address 2000)
  +--------+                   +--------+
  |   10   | <---------------- |  1000  |
  +--------+                   +--------+
*p is 10;  *p = 50 changes x to 50.
```

Why pointers matter for this module: linked lists, dynamic memory (malloc), call by reference and returning several results all depend on pointers.

6.2 Types of Pointers

[SOURCE] **The slide lists:**

Type	Example / meaning [SUPPLEMENT]
Integer pointer	int *p; points to an int
Float pointer	float *p; points to a float
Double pointer	double *p; points to a double (not to be confused with pointer-to-pointer int **pp)
Character pointer	char *s; points to a char / string
Structure pointer	struct node *ptr; used for linked list nodes; members accessed with ptr->data
Null pointer	int *p = NULL; points nowhere
Void pointer	void *vp; generic pointer, can hold any address

6.3 Pointer Concepts in Detail

[SOURCE table "Pointer in Details"]

1. Pointer arithmetic: four arithmetic operators can be used - ++, --, +, -. Pointer comparison operators ==, <, >, <=, >= are also supported. [SUPPLEMENT] p + 1 moves by sizeof(type) bytes, not by 1 byte: if p is an int* at 1000 and int is 4 bytes, p + 1 is 1004.
2. Array of pointers: an array can hold several pointers, e.g. char *names[3] = {"Ram", "Sita", "Amit"};
3. Pointer to pointer: C allows a pointer to a pointer and so on: int **pp = &p; then **pp is x.

4. Passing pointers to functions: passing an argument by reference/address lets the called function change the variable of the calling function.
5. Returning a pointer from a function: C allows a function to return a pointer to a static variable or to dynamically allocated memory. [CORRECTION] The slide also says "to the local variable". Returning the address of an ordinary (automatic) local variable is WRONG: the local variable is destroyed when the function returns, so the pointer "dangles". Only static variables or malloc'd memory may be returned.

6.4 Call by Value and Call by Reference

[SOURCE title; the slide content is an animation that does not appear in the PDF - SUPPLEMENT below]

Point	Call by value	Call by reference
What is passed	A copy of the value	The address (&x)
Parameter type	normal variable (int a)	pointer (int *a)
Effect on caller	Changes are NOT seen by the caller	Changes ARE seen by the caller
Memory	New copy created	No copy of the data
Example call	swap(x, y)	swap(&x, &y)

(C itself always passes by value; "call by reference" in C means passing the address by value and using * inside the function.)

Demonstration program (covers 6.1 - 6.5):

Program: pointers.c

```
/* Pointer basics, call by value vs call by reference,
   void and NULL pointers */
#include <stdio.h>

void swapByValue(int a, int b)
{
    int t = a;
    a = b;
    b = t;                      /* only the local copies are swapped */
}

void swapByReference(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;                     /* the caller's variables are swapped */
}

int main(void)
{
    int x = 10, y = 20;
    int *p = &x;                /* p stores the address of x */
    int arr[3] = {5, 15, 25};
    int *q = arr;               /* array name gives address of arr[0] */
    void *vp;
    int *np = NULL;             /* null pointer: points nowhere */
    float f = 2.5f;

    printf("x = %d, *p = %d\n", x, *p);
    *p = 50;                    /* change x through the pointer */
    printf("After *p = 50, x = %d\n", x);

    printf("*q = %d, *(q+1) = %d, *(q+2) = %d\n", *q, *(q + 1), *(q + 2));

    x = 10;
    swapByValue(x, y);
    printf("After swapByValue    : x = %d, y = %d\n", x, y);
    swapByReference(&x, &y);
    printf("After swapByReference: x = %d, y = %d\n", x, y);
```

```
    vp = &x;                    /* void pointer can hold any address */
    printf("void pointer to int  : %d\n", *(int *)vp);
    vp = &f;
    printf("void pointer to float: %.1f\n", *(float *)vp);

    if (np == NULL)
        printf("np is a NULL pointer\n");
    return 0;
}
```

Actual output:

```
x = 10, *p = 10
After *p = 50, x = 50
*q = 5, *(q+1) = 15, *(q+2) = 25
After swapByValue    : x = 10, y = 20
After swapByReference: x = 20, y = 10
void pointer to int  : 20
void pointer to float: 2.5
np is a NULL pointer
```

Explanation:

- swapByValue swaps its own copies a and b; x and y in main are untouched.
- swapByReference receives &x and &y; *a and *b ARE x and y, so they are really swapped.
- *(q+1) is the same as arr[1]: pointer arithmetic moves by one int.
- **A void pointer must be cast before dereferencing:** *(int *)vp.

6.5 Void Pointer and Null Pointer

[SOURCE] **Void pointer and heterogeneous linked list:**

- A heterogeneous linked list is a linked list that can store data of different data types.
- A void pointer is used in such lists because we are not sure which type of data will be stored in a node.

```
struct hnode { void *data; struct hnode *next; };   [SUPPLEMENT]
```

[SOURCE] **Null pointer vs void pointer:**

- A NULL pointer is a special reserved VALUE of a pointer. Every pointer type (int *, char *, ...) has its own null value. A pointer holding NULL does not point anywhere.
- A void pointer is a specific pointer TYPE - void * - that points to some data location which has no specific type.
- "Null pointer is a value, while void pointer is a type."

Point	NULL pointer	void pointer
Nature	A value	A type (void *)
Points to	Nothing	Any data (type unknown)
Dereference	Never allowed (crash)	Only after a cast
Use	Marks end of a linked list, empty list, failed malloc	Generic code, malloc's return type, hetero- geneous lists
Example	struct node *start = NULL;	void *vp = &x;

(A void pointer can also hold NULL - the two ideas are independent.)

6.6 Static vs Dynamic Data Structures

[SOURCE learning outcome: "Compare static and dynamic data structures"]

Point	Static (e.g. array)	Dynamic (e.g. linked list)
Size	Fixed when declared	Changes at run time
Allocation	Compile time	Run time (malloc/calloc)
Wastage	Unused slots wasted; overflow if more needed	Exactly as many nodes as needed (+ pointer cost)
Access	Fast, direct by index	Usually sequential
Example	int a[100];	nodes created by malloc

6.7 Array vs Linked List

[SOURCE: six points from three slides, plus SUPPLEMENT rows marked *]

Point	Array	Linked list
Definition	A collection of elements, each identified by an index	A linear collection of elements whose order is NOT given by their location in memory
Access	Random access: any element directly by index, a[i]	Sequential access: must go node by node to reach an element
Storage	Contiguous memory locations	Anywhere in memory
Size	Must be specified at declaration	No need to specify size
Memory allocation	Compile time (static)	Run time (dynamic)
Element relation	Elements independent of each other	Each node points to the next (or next and prev)
***Insert/delete in middle**	O(n): elements must be shifted	O(1) once the position is known (change links)
***Access i-th**	O(1)	O(n)
***Extra memory**	None per element	One pointer per node (two for doubly)
***Wastage**	Unused declared slots	None (allocate on demand)
***Locality**	Good (contiguous)	Poor (scattered)

Memory picture [SUPPLEMENT]:

```
Array int a[4] = {10,20,30,40}; (4-byte int)
   address: 1000 1004 1008 1012
            [10] [20] [30] [40]          a[2] = *(a + 2) -> 1008 directly

Linked list 10 -> 20 -> 30 -> 40:
   START=3020
   3020:[10|5000] 5000:[20|1240] 1240:[30|8800] 8800:[40|NULL]
   To reach 30 you must read 10's link, then 20's link.
```

6.8 Chapter 6 Exam Preparation

Quick Revision

- Pointer = variable holding an address. & gives address, * dereferences.
- **Pointer types:** int, float, double, char, structure, NULL, void.
- **Pointer arithmetic:** ++ -- + - and comparisons; steps by sizeof(type).
- Call by reference = pass address; changes visible to caller.
- NULL is a value; void * is a type.
- **Array:** contiguous, fixed size, random access. Linked list: scattered, dynamic size, sequential access.

Important Definitions

- **Pointer:** a variable whose value is the address of another variable.

- **Null pointer:** a pointer holding the reserved value NULL; points nowhere.
- **Void pointer:** a pointer of type void * that can point to data of any type.
- **Heterogeneous linked list:** a linked list storing data of different types (uses void pointers).
- **Dangling pointer** [SUPPLEMENT]**:** a pointer that still holds the address of memory that has been freed or has gone out of scope.

Common Mistakes

- Dereferencing NULL or an uninitialised pointer (crash).
- Returning the address of a local variable from a function.
- Thinking p+1 adds one byte.
- Writing "array allocation is dynamic" - an ordinary C array is static.
- Dereferencing a void pointer without a cast.

Practice Questions

1. [S] What is a pointer? List its types. Ans: 6.1, 6.2.
2. [S] Explain pointer arithmetic, array of pointers, pointer to pointer.
   Ans: 6.3.
3. [S] Differentiate NULL pointer and void pointer. Ans: 6.5 table.
4. [S] What is a heterogeneous linked list? Why is a void pointer used?
   Ans: 6.5.
5. [S] Differentiate array and linked list (at least 6 points).
   Ans: 6.7 table.
6. [S] Differentiate call by value and call by reference with a swap example. Ans: 6.4.
7. [A] Output? int a[] = {2,4,6,8}; int *p = a + 1;

```
printf("%d %d", *p, *(p+2));
```

   Ans: 4 8
8. [A] Output? int x = 5, *p = &x, **pp = &p; **pp = 9; printf("%d", x);
   Ans: 9
9. [A] Why is this function wrong? int *f() { int v = 3; return &v; }
   Ans: v is destroyed when f returns; the returned pointer dangles.

Chapter 7: Dynamic Memory Management

Source: *Module 2 "Lecture 5 Dynamic Memory Management.pdf" (= pptx slides 38-44). Lecture outcomes: "Demonstrate use of dynamic memory functions in C" and "Allocate and release memory effectively to avoid leaks."*

7.1 Dynamic Memory Allocation

Definition [SOURCE]:

> **Dynamic memory allocation is the procedure by which memory for a data structure is obtained (and its size changed) during RUN TIME instead of compile time.**

[SOURCE] C provides library functions for this, declared in <stdlib.h>:

```
malloc(), calloc(), free()
[SUPPLEMENT] A fourth function, realloc(ptr, newSize), resizes an
existing block. It is not in your notes but is useful to know.
```

Where the memory comes from [SUPPLEMENT]: a region called the HEAP. Memory on the heap stays allocated until the program calls free() (it is not released automatically when a function ends, unlike local variables on the stack).

> **Why linked lists need it:** we do not know in advance how many nodes will be needed, so each node is created with malloc() when required and released with free() when deleted.

7.2 malloc()

[SOURCE]

- "malloc" = memory allocation.
- Allocates a SINGLE block of memory of the specified size in bytes.
- Returns a pointer of type void *, which can be cast to any pointer type.
- **The memory is NOT initialised:** it contains garbage values.
- If space is insufficient, allocation fails and it returns NULL.

```
Syntax : ptr = (cast-type *) malloc(byte-size);
Example: ptr = (struct node *) malloc(sizeof(struct node));
         int *ptr = (int *) malloc(5 * sizeof(int));
```

> [CORRECTION] The slide writes ptr = struct node* malloc(sizeof(struct node)). The cast must be in brackets: (struct node *) malloc(...).

```
Diagram [SOURCE]:  int *ptr = (int*) malloc(5 * sizeof(int));
     ptr --> [ one block of 20 bytes, garbage values ]
              <--------- 20 bytes ---------->
     (assuming int is 4 bytes)
```

7.3 calloc()

[SOURCE]

- "calloc" = contiguous allocation.
- Allocates the specified NUMBER of blocks, each of the specified size.
- Initialises every byte to 0.
- Returns NULL if space is insufficient.

```
Syntax : ptr = (cast-type *) calloc(n, element-size);
Example: ptr = (struct node *) calloc(4, sizeof(struct node));
         int *ptr = (int *) calloc(5, sizeof(int));

Diagram [SOURCE]:  int *ptr = (int*) calloc(5, sizeof(int));
     ptr --> [ 0 | 0 | 0 | 0 | 0 ]   5 blocks of 4 bytes each
              <-4->
              <------ 20 bytes ----->
```

7.4 free()

[SOURCE]

- free() de-allocates memory obtained with malloc() or calloc().
- Such memory is not released on its own, so free() must be used whenever dynamic allocation is done. It reduces wastage of memory.
- **Syntax:** free(ptr);

```
Diagram [SOURCE]:
   ptr = calloc(5, sizeof(int))  -> [ | | | | ]   (20 bytes)
   ... operations on ptr ...
   free(ptr)                     -> the memory of ptr is released
```

Rules [SUPPLEMENT]**:**

1. Free only memory that came from malloc/calloc/realloc.
2. Free each block exactly once (double free crashes or corrupts memory).
3. Do not use memory after freeing it (use-after-free). Set the pointer to NULL after free to avoid a DANGLING POINTER.
4. free(NULL) is allowed and does nothing.
5. Forgetting to free memory you no longer need is a MEMORY LEAK.

7.5 malloc vs calloc

Point	malloc()	calloc()
Full form	Memory allocation	Contiguous allocation
Arguments	One: total bytes, malloc(n * size)	Two: number of blocks and size of each, calloc(n,size)
Blocks	Single large block	n blocks of given size (also contiguous)
Initial value	Garbage (uninitialised)	All bytes set to 0
Speed [SUPP.]	Slightly faster (no zeroing)	Slightly slower (zeroes)
Failure	Returns NULL	Returns NULL
Example	(int*)malloc(5*sizeof(int))	(int*)calloc(5,sizeof(int))

7.6 Program: malloc, calloc and free [SUPPLEMENT]

Program: dma.c

```
/* malloc, calloc and free */
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int i, n = 5;
    int *a, *b;

    /* one block of 20 bytes (if int = 4 bytes) */
    a = (int *)malloc(n * sizeof(int));
    /* 5 blocks of 4 bytes, all set to 0 */
```

```
    b = (int *)calloc(n, sizeof(int));
    if (a == NULL || b == NULL)
    {
        printf("Memory allocation failed\n");
        free(a);
        free(b);
        return 1;
    }

    for (i = 0; i < n; i++)
        /* malloc memory must be initialised by us */
        a[i] = (i + 1) * 10;

    printf("malloc array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\ncalloc array: ");
    for (i = 0; i < n; i++)
        printf("%d ", b[i]);               /* prints 0 0 0 0 0 */
    printf("\n");

    free(a);                               /* return memory to the heap */
    free(b);
    a = b = NULL;                          /* avoid dangling pointers */
    return 0;
}
```

Actual output:

malloc array: 10 20 30 40 50
calloc array: 0 0 0 0 0

Explanation:

- Both pointers are checked for NULL before use - allocation can fail.
- malloc memory is filled by the loop; reading it before that would give garbage.
- calloc memory is already zero.
- free() releases both blocks; setting the pointers to NULL prevents accidental use later.

7.7 Using malloc for a Linked List Node

```
struct node *ptr;
ptr = (struct node *) malloc(sizeof(struct node));
if (ptr == NULL)
    printf("Memory not available");      /* always check */
else
{
    ptr->data = 10;
    ptr->next = NULL;
}
...
free(ptr);          /* when the node is deleted */
```

sizeof(struct node) is used so the program stays correct on any machine (an int and a pointer can have different sizes on different systems). Never use sizeof(struct node *) here - that is only the size of a pointer.

7.8 Chapter 7 Exam Preparation

Quick Revision

- Dynamic allocation = run time, from the heap, functions in <stdlib.h>.
- malloc(bytes): one block, garbage. calloc(n, size): n blocks, zeros.
- Both return void * (cast it) and NULL on failure.

- free(ptr) releases memory; never use or free it again afterwards.

Important Definitions

- **Dynamic memory allocation:** allocating memory at run time.
- malloc: allocates a single block of given bytes, uninitialised.
- calloc: allocates n contiguous blocks of given size, initialised to 0.
- free: releases dynamically allocated memory.
- **Memory leak:** allocated memory that is never freed and no longer reachable.

Common Mistakes

- Not checking for NULL after malloc/calloc.
- Writing malloc(sizeof(node *)) instead of malloc(sizeof(node)).
- Expecting malloc memory to be zero.
- Using a node after free(), or freeing it twice.
- Losing the only pointer to a block (leak), e.g. start = start->next without freeing the old first node.

Practice Questions

1. [S] What is dynamic memory allocation? Name the functions used.

   Ans: 7.1.
2. [S] Explain malloc() with syntax and example. Ans: 7.2.
3. [S] Explain calloc() with syntax and example. Ans: 7.3.
4. [S] Explain free(). Ans: 7.4.
5. [S] Differentiate malloc and calloc. Ans: 7.5 table.
6. [A] How many bytes does calloc(4, sizeof(struct node)) allocate if struct node has an int (4 bytes) and a pointer (8 bytes)?

   Ans: sizeof(struct node) is 16 on a typical 64-bit system (4 + 4 bytes padding + 8), so 4 x 16 = 64 bytes. (On 16-bit Turbo C, int and near pointer are 2 bytes each: 4 x 4 = 16 bytes.)
7. [A] What is a dangling pointer and how do you avoid it after free()?

   Ans: a pointer to released memory; set it to NULL after free.

Chapter 8: Linked List and Singly Linked List Operations

Sources: *Module 2 "Lecture 4 Types of Linked List and Linked List Representation.pdf", "Lecture 6/7/8 Singly Link List Operations - 1/2/3.pdf", and slides 26-37 and 45-66 of "Module 2 Linked List.pptx" (which contain the full code of every operation).*

8.1 Introduction to Linked List

Definition (exam-ready):

> **A linked list is a linear data structure consisting of a sequence of nodes, where each node contains data and a link (pointer) to the next node. The linear order is given by the links, not by the physical placement of the nodes in memory.**

[SOURCE]

- "A linked list is a sequence of data structures which are connected together via links."
- "Linked list is the second most-used data structure after array."
- **Important terms:**
  Data - stores the element.
  Next - the link to the next node.
  Head - the connection link to the first node (called START in the code).

Why it is needed: arrays have a fixed size and need shifting for insertion and deletion. A linked list grows and shrinks at run time (using malloc and free) and inserts/deletes by changing links.

8.2 Representation of a Linked List

[SOURCE]

```
struct node
{
    int data;
    struct node *next;
};

A node:         +------+------+
                | data | next |
                +------+------+

[SOURCE figure] head -> [H| ]-> [E| ]-> [A| ]-> [P| ]-> null
```

- struct node *next is a SELF-REFERENTIAL pointer: a structure containing a pointer to the same structure type.
- The last node's next is NULL - this marks the end of the list.
- An empty list has start = NULL.
- In the program the list is accessed through a global pointer:

```
struct node *start = NULL;
```

8.3 Types of Linked List

[SOURCE] Singly linked list, circular linked list, doubly linked list.

1. Singly linked list - each node points to the next; last points to NULL.

```
START -> [A| ]-> [B| ]-> [C| ]-> [D|X]
```

2. Circular linked list - the last node points back to the first node.

```
Head -> [12| ]-> [56| ]-> [2| ]-> [11| ]--+
         ^                                |
         +--------------------------------+
```

3. Doubly linked list - each node has prev and next pointers.

```
START -> X<-[ |A| ]<=>[ |B| ]<=>[ |C| ]<=>[ |D| ]->X
           prev pointer        next pointer
```

[SUPPLEMENT] 4. Circular doubly linked list - doubly linked, and the first and last nodes are linked to each other in both directions (11.5).

Lecture outcome [SOURCE]**:** "Differentiate types of linked lists based on their structure and analyse suitable applications for each type." See the comparison table in 11.7.

8.4 Basic Operations Pictured (Lecture 4)

[SOURCE] **The basic operations on almost all linked lists:** Add First, Traversing, Add Last, Insert After, Insert Before, Deletion. The slides show them on the list H -> E -> A -> P.

8.4.1 Traversing

```
tmp starts at head and follows next until NULL:
   head
    |
   [H]-->[E]-->[A]-->[P]--> null
    ^     ^     ^     ^
   tmp   tmp   tmp   tmp      (tmp moves one node per step)
```

8.4.2 Add First

```
new node C:  C.next = head ; head = C
         +---+
   head->| C |-->[H]-->[E]-->[A]-->[P]--> null
         +---+
(the old "head -> H" link is replaced)
```

8.4.3 Add Last

```
tmp walks to the last node P; P.next (was null) = S; S.next = null
   head->[H]-->[E]-->[A]-->[P]--> [S]--> null
                           ^tmp
```

8.4.4 Inserting After (a node pointed to by tmp, here E)

```
new.next = E.next (A) ; E.next = new
   head->[H]-->[E]   [A]-->[P]--> null
               |      ^
               +->[new]
```

8.4.5 Inserting Before (node cur = A, with prev = E)

```
Need the PREVIOUS node: new.next = cur ; prev.next = new
   head->[H]-->[E]   [A]-->[P]--> null
              prev   cur
               |      ^
               +->[new]
```

8.4.6 Delete (node cur = A, with prev = E)

```
prev.next = cur.next ; free(cur)
   head->[H]-->[E]   [A]   [P]--> null
                |           ^
                +-----------+
```

Key insight: to insert BEFORE or delete a node in a singly linked list you must stop at the node BEFORE it, because you cannot go backwards.

8.5 Operations on Singly Linked List (from the notes)

[SOURCE list]

```
Create                          Count number of nodes
Insert at beginning             Delete at beginning
Insert at end                   Delete at end
Insert after an element         Delete after an element
Insert before an element        Delete before an element
Insert at a particular position Delete at a particular position
Display linked list             Reverse a linked list
Sort the linked list            Search in a linked list
(plus Copy a linked list, slides 64-65)
```

All functions below are from the corrected menu program in 8.22. Common setup used by every function:

```
#include <stdio.h>
#include <stdlib.h>
struct node { int data; struct node *next; };
struct node *start = NULL;    /* main list */
struct node *start1 = NULL;   /* second list for reverse/copy */
```

Running example for dry runs: START -> 10 -> 20 -> 30 -> 40 -> NULL

8.6 Display (Traversal)

Algorithm:

```
Step 1: SET TEMP = START
Step 2: IF TEMP = NULL, PRINT "List is empty" and go to Step 5
Step 3: Repeat while TEMP != NULL
           PRINT TEMP->DATA
           SET TEMP = TEMP->NEXT
Step 4: (end of loop)
Step 5: END
```

Function: print() [from sll.c]

```
/* ---------- Display ---------- */
void print(void)
{
    struct node *temp = start;
    if (temp == NULL)
        printf("\nList is empty.");
    else
    {
        printf("\n");
        while (temp != NULL)
        {
            printf("%d\t", temp->data);
            temp = temp->next;
        }
    }
    printf("\n");
}
```

Dry run: temp = 10 -> print 10; temp = 20 -> print 20; ... temp = NULL -> stop. Output: 10 20 30 40.

Complexity: O(n) time, O(1) auxiliary space.

Common mistake: writing while(temp->next != NULL) - that skips printing the last node.

8.7 Create

[SOURCE] Values are read until -1 is entered; each value is appended at the end.

Algorithm:

```
Step 1: READ VAL
Step 2: Repeat while VAL != -1
          a. Allocate PTR; PTR->DATA = VAL; PTR->NEXT = NULL
          b. IF START = NULL, SET START = PTR
             ELSE walk TEMP from START to the last node; TEMP->NEXT = PTR
          c. READ VAL
Step 3: END
```

Function: create() [from sll.c]

```
/* ---------- Create (append values until -1) ---------- */
void create(void)
{
    struct node *ptr, *temp;
    int val;
    printf("\nTo stop creation of linked list enter -1.");
    printf("\nEnter value:");
    scanf("%d", &val);
    while (val != -1)
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            ptr->data = val;
            ptr->next = NULL;
            if (start == NULL)
                start = ptr;
            else
            {
                temp = start;
                while (temp->next != NULL)
                    temp = temp->next;
                temp->next = ptr;
            }
        }
        printf("Enter value:");
        scanf("%d", &val);
    }
}
```

Dry run (input 10 20 30 -1):

```
val=10: list empty -> start = [10]
val=20: temp walks to 10 -> 10.next = [20]
val=30: temp walks 10 -> 20 -> 20.next = [30]
val=-1: stop. List: 10 -> 20 -> 30 -> NULL
```

Complexity: O(n^2) for n values, because every append walks the whole list (1 + 2 + ... + n-1 steps). Keeping a "last" pointer makes it O(n).

[SOURCE note] The slide sets next = NULL separately in its two branches (start->next = ptr->next = NULL and ptr->next = NULL). That is correct; the version above simply sets ptr->next = NULL once for every new node.

8.8 Count Number of Nodes

Function: count() [from sll.c]

```
/* ---------- Count ---------- */
int count(void)
{
    struct node *temp = start;
    int pos = 0;
    while (temp != NULL)
    {
        temp = temp->next;
        pos++;
    }
    return pos;
}
```

Dry run: pos 0 -> 1 (10) -> 2 (20) -> 3 (30) -> 4 (40) -> temp NULL -> return 4. Complexity: O(n) time, O(1) space.

8.9 Search a Particular Element

[SOURCE] search(key) returns:

```
  -1  if the list is empty,
  -2  if the key is not found,
  otherwise the INDEX of the node, counted from 0 (first node = 0).
This 0-based index is what insert_after, insert_before, del_after and
del_before use to move to the right node.
```

Function: search() [from sll.c]

```
int search(int key)
{
    struct node *temp = start;
    int pos = -1;
    if (temp == NULL)
        return pos;
    while (temp != NULL)
    {
        if (temp->data == key)
            return ++pos;
        temp = temp->next;
        pos++;
    }
    return -2;
}
```

Dry run search(30):

```
temp=10 pos=-1: no -> temp=20, pos=0
temp=20: no -> temp=30, pos=1
temp=30: match -> return ++pos = 2      (30 is at index 2, position 3)
```

search(99): runs off the end -> return -2.

Complexity: best O(1) (first node), worst/average O(n). Space O(1).

Note: a linked list cannot use binary search efficiently because there is no random access to the middle node.

8.10 Insert at Beginning

Algorithm:

```
Step 1: Allocate PTR; IF PTR = NULL, PRINT "OVERFLOW" and stop
Step 2: READ PTR->DATA
Step 3: SET PTR->NEXT = START
Step 4: SET START = PTR
Step 5: END
```

Function: insert_beg() [from sll.c]

```
/* ---------- Insert at beginning ---------- */
void insert_beg(void)
{
    struct node *ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr != NULL)
    {
        printf("\nEnter value to be inserted:");
        scanf("%d", &ptr->data);
        ptr->next = start;     /* NULL if list was empty */
        start = ptr;
    }
}
```

Before: START -> 10 -> 20 -> 30 -> 40 -> NULL ; insert 5

```
ptr->next = start :  [5]-> 10 -> 20 -> 30 -> 40
start = ptr       :  START -> 5 -> 10 -> 20 -> 30 -> 40 -> NULL
```

Complexity: O(1).

Order matters: if you do start = ptr first, the old list is lost.

[SOURCE note] The slide version has two branches (empty / not empty); both do the same thing, so they are merged here.

8.11 Insert at End

Algorithm:

```
Step 1: Allocate PTR; READ PTR->DATA; SET PTR->NEXT = NULL
Step 2: IF START = NULL, SET START = PTR and go to Step 5
Step 3: SET TEMP = START; Repeat while TEMP->NEXT != NULL:
           TEMP = TEMP->NEXT
Step 4: SET TEMP->NEXT = PTR
Step 5: END
```

Function: insert_end() [from sll.c]

```
/* ---------- Insert at end ---------- */
void insert_end(void)
{
    struct node *ptr, *temp = start;
    ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr != NULL)
    {
        printf("\nEnter value to be inserted:");
        scanf("%d", &ptr->data);
        ptr->next = NULL;
        if (temp == NULL)
            start = ptr;
        else
        {
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = ptr;
        }
    }
}
```

Insert 50: temp stops at 40 (40->next is NULL); 40->next = [50].

Result: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

Complexity: O(n) (walk to the end); O(1) if a last pointer is kept.

8.12 Insert After an Element

Algorithm:

```
Step 1: READ KEY; POS = search(KEY)
Step 2: IF POS < 0, PRINT "Value not present" and go to Step 6
Step 3: Allocate PTR and READ its data
Step 4: Move TEMP from START forward POS times (TEMP = node with KEY)
Step 5: PTR->NEXT = TEMP->NEXT ; TEMP->NEXT = PTR
Step 6: Display updated list; END
```

Function: insert_after() [from sll.c]

```
/* ---------- Insert after a given value ---------- */
void insert_after(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter after which value you want to insert:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    else
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            printf("\nEnter value to be inserted:");
            scanf("%d", &ptr->data);
            /* move to the node holding the key */
            for (i = 0; i < key; i++)
                temp = temp->next;
            ptr->next = temp->next;
            temp->next = ptr;
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run: insert 25 after 20. search(20) = 1. Loop runs once: temp = 20.

```
ptr->next = 20->next (30)  ;  20->next = ptr
Result: 10 -> 20 -> 25 -> 30 -> 40
```

Insert after the last value (40) works too: temp = 40, ptr->next = NULL.

Complexity: O(n) (search + walk).

8.13 Insert Before an Element

Algorithm:

```
Step 1: READ KEY; POS = search(KEY)
Step 2: IF POS < 0, PRINT "Value not present" and stop
Step 3: Allocate PTR and READ its data
Step 4: IF POS = 0 (key is in the first node):
           PTR->NEXT = START ; START = PTR
        ELSE
           Move TEMP forward POS-1 times (TEMP = node before KEY)
           PTR->NEXT = TEMP->NEXT ; TEMP->NEXT = PTR
Step 5: END
```

Function: insert_before() [from sll.c]

```
/* ---------- Insert before a given value ---------- */
void insert_before(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter before which value you want to insert:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    else
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            printf("\nEnter value to be inserted:");
            scanf("%d", &ptr->data);
            if (key == 0)                   /* key is in the first node */
            {
                ptr->next = start;
                start = ptr;
            }
            else
            {
                /* stop at node BEFORE the key */
                for (i = 0; i < key - 1; i++)
                    temp = temp->next;
                ptr->next = temp->next;
                temp->next = ptr;
            }
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run: insert 15 before 20. search(20) = 1 -> loop runs 0 times, temp = 10

```
ptr->next = 10->next (20) ; 10->next = ptr
Result: 10 -> 15 -> 20 -> 30 -> 40
```

Insert 5 before 10: pos = 0 -> new first node.

[SOURCE note] The slide tests "if (temp->data == i)" (first node holds the key) - this is the same as key == 0.

Complexity: O(n).

8.14 Insert at a Particular Position

Positions are counted from 1.

Algorithm:

```
Step 1: READ POS; while POS <= 0 read again
Step 2: IF POS > COUNT + 1, PRINT "exceeds length" and stop
Step 3: Allocate PTR and READ its data
Step 4: IF POS = 1: PTR->NEXT = START ; START = PTR
        ELSE move TEMP forward POS-2 times (TEMP = node at POS-1)
             PTR->NEXT = TEMP->NEXT ; TEMP->NEXT = PTR
Step 5: END
```

Function: insert_at() [from sll.c]

```
/* ---------- Insert at a position (1-based) ---------- */
void insert_at(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter location at which you want to insert:");
```

```
    scanf("%d", &key);
    while (key <= 0)
    {
        printf("\nEnter any other location at which you want to insert:");
        scanf("%d", &key);
    }
    if (key > count() + 1)        /* count()+1 means "append at end" */
        printf("\nSorry!!!You exceed the linked list length.");
    else
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            printf("\nEnter value to be inserted:");
            scanf("%d", &ptr->data);
            if (key == 1)
            {
                ptr->next = start;
                start = ptr;
            }
            else
            {
                /* reach node at position key-1 */
                for (i = 0; i < key - 2; i++)
                    temp = temp->next;
                ptr->next = temp->next;
                temp->next = ptr;
            }
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run: insert 99 at position 3 in 10 -> 20 -> 30 -> 40.

```
Loop i < 1: runs once -> temp = 20 (position 2)
ptr->next = 30 ; 20->next = ptr
Result: 10 -> 20 -> 99 -> 30 -> 40   (99 is now 3rd)
```

[CORRECTION] **The slide version:**

1. Tests key > count(), so you cannot insert at position count()+1 (the end) and cannot insert into an empty list at position 1. Corrected to key > count() + 1.
2. Re-asks only when key == 0; a negative position slips through and inserts at a wrong place. Corrected to key <= 0.
3. Has a typo "Else`".

Complexity: O(n).

8.15 Delete at Beginning

Algorithm:

```
Step 1: IF START = NULL, PRINT "UNDERFLOW / list empty" and stop
Step 2: SET TEMP = START
Step 3: SET START = START->NEXT
Step 4: FREE TEMP
Step 5: END
```

Function: del_beg() [from sll.c]

```
/* ---------- Delete at beginning ---------- */
void del_beg(void)
{
    struct node *temp = start;
    if (temp == NULL)
        printf("\nLinked list is empty.");
    else
    {
        printf("\nDeleted value from link list is:%d.", start->data);
        start = start->next;
        free(temp);
    }
}
```

10 -> 20 -> 30 -> 40 : temp = 10 ; start = 20 ; free 10

Result: 20 -> 30 -> 40. Complexity: O(1).

8.16 Delete at End

Algorithm:

```
Step 1: IF START = NULL, PRINT "list empty" and stop
Step 2: IF START->NEXT = NULL (one node): FREE START; START = NULL; stop
Step 3: SET TEMP = START; Repeat while TEMP->NEXT->NEXT != NULL:
           TEMP = TEMP->NEXT          (TEMP stops at second-last node)
Step 4: FREE TEMP->NEXT ; TEMP->NEXT = NULL
Step 5: END
```

Function: del_end() [from sll.c]

```
/* ---------- Delete at end ---------- */
void del_end(void)
{
    struct node *temp = start;
    if (temp == NULL)
        printf("\nLinked list is empty.");
    else
    {
        if (temp->next == NULL)            /* only one node */
        {
            printf("\nDeleted value from link list is: %d", temp->data);
            start = NULL;
            free(temp);
        }
        else
        {
            /* stop at second-last node */
            while (temp->next->next != NULL)
                temp = temp->next;
            printf("\nDeleted value from link list is:%d.",
                   temp->next->data);
            free(temp->next);
            temp->next = NULL;
        }
        printf("\nUpdated list is:");
        print();
    }
}
```

10 -> 20 -> 30 -> 40 : temp stops at 30 (30->next->next is NULL); free 40 ; 30->next = NULL. Result 10 -> 20 -> 30.

[CORRECTION] The slide's del_end is missing the function's final closing brace.

Why the one-node case is separate: with one node, temp->next is NULL, so temp->next->next would dereference NULL and crash.

Complexity: O(n).

8.17 Delete After an Element

Function: del_after() [from sll.c]

```
/* ---------- Delete the node after a given value ---------- */
void del_after(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter after which value you want to delete:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    else
    {
        for (i = 0; i < key; i++)
            temp = temp->next;
        ptr = temp->next;
        if (ptr == NULL)
            printf("\nNo value available to delete.");
        else
        {
            printf("\nDeleted value from link list is:%d.", ptr->data);
            temp->next = ptr->next;
            free(ptr);
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run: delete after 20: pos = 1 -> temp = 20 ; ptr = 30 ; 20->next = 40 ; free 30. Result 10 -> 20 -> 40.

Delete after 40 (last): ptr = NULL -> "No value available to delete".

Complexity: O(n).

8.18 Delete Before an Element

Cases (pos = index of key):

```
pos = 0 : key is the first node -> nothing before it.
pos = 1 : the node before key is the FIRST node -> delete at beginning.
pos >= 2: move temp pos-2 times (to the node two places before key);
          ptr = temp->next is the node to delete; temp->next = ptr->next.
```

Function: del_before() [from sll.c]

```
/* ---------- Delete the node before a given value ---------- */
void del_before(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter before which value you want to delete:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    /* key in first node: nothing before it */
    else if (key == 0)
        printf("\nNo value present before:%d to delete.", start->data);
    else if (key == 1)                    /* delete the first node */
    {
        start = temp->next;
        printf("\nDeleted value from link list is:%d.", temp->data);
        free(temp);
    }
    else
    {
        /* reach node two places before key */
```

```
        for (i = 0; i < key - 2; i++)
            temp = temp->next;
        ptr = temp->next;
        printf("\nDeleted value from link list is:%d.", ptr->data);
        temp->next = ptr->next;
        free(ptr);
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run: delete before 30 in 10 -> 20 -> 30 -> 40. pos = 2 -> loop runs 0 times, temp = 10, ptr = 20 ; 10->next = 30 ; free 20. Result 10 -> 30 -> 40.

Delete before 20: pos = 1 -> first node 10 deleted, start = 20.

[SOURCE note] The slide checks "temp->next->data == i" for the pos = 1 case; the corrected code uses pos directly (same logic, easier to read).

Complexity: O(n).

8.19 Delete at a Particular Position

Function: del_at() [from sll.c]

```
/* ---------- Delete at a position (1-based) ---------- */
void del_at(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter location at which you want to delete:");
    scanf("%d", &key);
    while (key <= 0)
    {
        printf("\nEnter another location:");
        scanf("%d", &key);
    }
    if (key > count())
        printf("\nSorry!!!You exceed the linked list length.");
    else
    {
        if (key == 1)
        {
            start = start->next;
            printf("\nDeleted value from link list is:%d.", temp->data);
            free(temp);
        }
        else
        {
            for (i = 0; i < key - 2; i++)
                temp = temp->next;
            ptr = temp->next;
            printf("\nDeleted value from link list is:%d.", ptr->data);
            temp->next = ptr->next;
            free(ptr);
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run: delete position 3 in 10 -> 20 -> 30 -> 40: temp moves once to 20; ptr = 30 ; 20->next = 40 ; free 30. Result 10 -> 20 -> 40. Position 1 -> same as delete at beginning. Position > count -> error.

[CORRECTION] Like insert_at, only key == 0 was rejected; negative positions are now rejected too.

Complexity: O(n).

8.20 Reverse a Linked List

8.20.1 Method 1 - in-place reversal with three pointers [SUPPLEMENT; this is

the standard exam method and is also the idea behind REVERSEL.doc]

Algorithm:

```
Step 1: PREV = NULL, CURR = START
Step 2: Repeat while CURR != NULL
            NEXT = CURR->NEXT       (save the rest)
            CURR->NEXT = PREV       (reverse the link)
            PREV = CURR             (advance prev)
            CURR = NEXT             (advance curr)
Step 3: START = PREV
Step 4: END
```

Function: reverse() [from sll.c]

```
/* ---------- Reverse (in place, three pointers) ---------- */
void reverse(void)
{
    struct node *prev = NULL, *curr = start, *nextNode;
    while (curr != NULL)
    {
        nextNode = curr->next;   /* remember the rest of the list */
        curr->next = prev;       /* reverse this link */
        prev = curr;             /* move prev one step */
        curr = nextNode;         /* move curr one step */
    }
    start = prev;
    printf("\nReversed list is:");
    print();
}
```

Dry run on 10 -> 20 -> 30:

Iter	prev	curr	nextNode	Links after the step
init	NULL	10	-	10->20->30->NULL
1	10	20	20	10->NULL ; 20->30->NULL
2	20	30	30	20->10->NULL ; 30->NULL
3	30	NULL	NULL	30->20->10->NULL

start = prev = 30 -> List: 30 -> 20 -> 10 -> NULL

Pictures:

```
start: [10]->[20]->[30]->X
iter1: X<-[10]   [20]->[30]->X        prev=10 curr=20
iter2: X<-[10]<-[20]   [30]->X        prev=20 curr=30
iter3: X<-[10]<-[20]<-[30]            prev=30 curr=NULL -> start=30
```

Complexity: O(n) time, O(1) auxiliary space (no new nodes).

8.20.2 Method 2 - the slide method (rev1/rev2) [SOURCE, slide 62]

```
rev1() deletes the first node of the list and returns its value;
rev2(val) inserts that value at the BEGINNING of a second list start1.
Repeating this count() times moves 10, 20, 30 into start1 as 30, 20, 10.
Finally start = start1.
```

Function: rev1() [from sll.c]

```
int rev1(void)
{
    struct node *temp = start;
    int val = 0;
    if (temp == NULL)
        printf("\nLinked list is empty.");
    else
    {
        val = start->data;
        start = start->next;
        free(temp);
    }
    return val;
}
```

Function: rev2() [from sll.c]

```
void rev2(int val)
{
    struct node *ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr != NULL)
    {
        ptr->data = val;
        ptr->next = start1;
        start1 = ptr;
    }
}
```

Function: reverse_by_copy() [from sll.c]

```
void reverse_by_copy(void)
{
    int i, j;
    start1 = NULL;
    j = count();
    for (i = 0; i < j; i++)
        rev2(rev1());
    start = start1;
    start1 = NULL;
    printf("\nReversed list is:");
    print();
}
```

Dry run: list 10 20 30.

```
rev1 -> 10, rev2(10): start1 = 10
rev1 -> 20, rev2(20): start1 = 20 -> 10
rev1 -> 30, rev2(30): start1 = 30 -> 20 -> 10 ; start = start1
```

Complexity: O(n) time; it frees and allocates n nodes (slower than method 1, and needs extra malloc calls).

[CORRECTION] The slide's rev1 returns an uninitialised val when the list is empty, and reverse() declares unused variables (char ch). Fixed.

8.21 Sort and Copy

8.21.1 Sort (ascending / descending) [SOURCE slide 63]

temp1 takes each node in turn; temp2 scans all nodes after it; whenever temp1->data > temp2->data the DATA values are swapped (nodes are not moved). This is selection-exchange sort. For descending order use <.

Function: sort() [from sll.c]

```
/* ---------- Sort ascending (swap data, selection-style) ---------- */
void sort(void)
{
    struct node *temp1 = start, *temp2;
    int tmp;
```

```
    if (temp1 == NULL)
        printf("\nLinked list is empty.");
    else
    {
        while (temp1->next != NULL)
        {
            temp2 = temp1->next;
            while (temp2 != NULL)
            {
                if (temp1->data > temp2->data)    /* use < for descending */
                {
                    tmp = temp1->data;
                    temp1->data = temp2->data;
                    temp2->data = tmp;
                }
                temp2 = temp2->next;
            }
            temp1 = temp1->next;
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

Dry run on 30 -> 10 -> 40 -> 20:

temp1=30: vs 10 swap -> 10 30 40 20 ; vs 40 no ; vs 20 no (10<20) (temp1 now holds 10)

```
temp1=2nd node(30): vs 40 no ; vs 20 swap -> 10 20 40 30
temp1=3rd node(40): vs 30 swap -> 10 20 30 40
temp1=4th node: next is NULL -> stop. Sorted: 10 20 30 40
```

Complexity: O(n^2) comparisons in all cases; O(1) auxiliary space.

8.21.2 Copy a linked list [SOURCE slides 64-65]

Creates a new list start1 with the same data in new nodes, then prints both lists side by side with node addresses to show that the copy uses DIFFERENT memory.

Function: copy() [from sll.c]

```
/* ---------- Copy list into start1 and show both ---------- */
void copy(void)
{
    struct node *ptr, *temp = start, *last = NULL;
    start1 = NULL;
    while (temp != NULL)
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            ptr->data = temp->data;
            ptr->next = NULL;
            if (last == NULL)
                start1 = ptr;
            else
                last->next = ptr;
            /* keep a tail pointer: no re-traversal needed */
            last = ptr;
        }
        temp = temp->next;
    }
    printf("\nOriginal\tCopy\n");
    temp = start;
    ptr = start1;
    while (temp != NULL && ptr != NULL)
    {
        printf("%d (%p)\t%d (%p)\n", temp->data, (void *)temp,
               ptr->data, (void *)ptr);
        temp = temp->next;
        ptr = ptr->next;
    }
    while (start1 != NULL)       /* free the copy */
```

```
    {
        ptr = start1;
        start1 = start1->next;
        free(ptr);
    }
}
```

Actual output (addresses differ every run):

```
Original        Copy
7 (0x502000000230)  7 (0x502000000250)
20 (0x502000000210) 20 (0x502000000270)
...
```

[CORRECTION] The slide prints addresses with %u; the correct format for a pointer is %p (with a (void *) cast). The slide's inner while loop on temp1 works (temp1 stays at the last copied node, so it moves one step at a time); an explicit "last" pointer is clearer. The slide also never frees the copy (the corrected version does).

Complexity: O(n) time, O(n) extra space (n new nodes).

8.22 Complete Menu-Driven Program: Singly Linked List

Problem statement: Write a menu-driven C program to perform all the singly linked list operations listed in 8.5.

Program: sll.c

```
/* Singly linked list: all operations from the Module 2
   lecture slides (corrected) */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *start = NULL;     /* first node of the main list */
/* first node of the second list (used by reverse/copy) */
struct node *start1 = NULL;

/* ---------- Display ---------- */
void print(void)
{
    struct node *temp = start;
    if (temp == NULL)
        printf("\nList is empty.");
    else
    {
        printf("\n");
        while (temp != NULL)
        {
            printf("%d\t", temp->data);
            temp = temp->next;
        }
    }
    printf("\n");
}

/* ---------- Create (append values until -1) ---------- */
void create(void)
{
    struct node *ptr, *temp;
    int val;
    printf("\nTo stop creation of linked list enter -1.");
    printf("\nEnter value:");
    scanf("%d", &val);
    while (val != -1)
    {
```

```
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            ptr->data = val;
            ptr->next = NULL;
            if (start == NULL)
                start = ptr;
            else
            {
                temp = start;
                while (temp->next != NULL)
                    temp = temp->next;
                temp->next = ptr;
            }
        }
        printf("Enter value:");
        scanf("%d", &val);
    }
}

/* ---------- Count ---------- */
int count(void)
{
    struct node *temp = start;
    int pos = 0;
    while (temp != NULL)
    {
        temp = temp->next;
        pos++;
    }
    return pos;
}

/* ---------- Search ----------
   returns 0-based index of key, -1 if list empty, -2 if not found */
int search(int key)
{
    struct node *temp = start;
    int pos = -1;
    if (temp == NULL)
        return pos;
    while (temp != NULL)
    {
        if (temp->data == key)
            return ++pos;
        temp = temp->next;
        pos++;
    }
    return -2;
}

/* ---------- Insert at beginning ---------- */
void insert_beg(void)
{
    struct node *ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr != NULL)
    {
        printf("\nEnter value to be inserted:");
        scanf("%d", &ptr->data);
        ptr->next = start;     /* NULL if list was empty */
        start = ptr;
    }
}

/* ---------- Insert at end ---------- */
void insert_end(void)
{
    struct node *ptr, *temp = start;
    ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr != NULL)
    {
        printf("\nEnter value to be inserted:");
        scanf("%d", &ptr->data);
        ptr->next = NULL;
```

```
        if (temp == NULL)
            start = ptr;
        else
        {
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = ptr;
        }
    }
}

/* ---------- Insert after a given value ---------- */
void insert_after(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter after which value you want to insert:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    else
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            printf("\nEnter value to be inserted:");
            scanf("%d", &ptr->data);
            /* move to the node holding the key */
            for (i = 0; i < key; i++)
                temp = temp->next;
            ptr->next = temp->next;
            temp->next = ptr;
        }
    }
    printf("\nUpdated list is:");
    print();
}

/* ---------- Insert before a given value ---------- */
void insert_before(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter before which value you want to insert:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    else
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            printf("\nEnter value to be inserted:");
            scanf("%d", &ptr->data);
            if (key == 0)                   /* key is in the first node */
            {
                ptr->next = start;
                start = ptr;
            }
            else
            {
                /* stop at node BEFORE the key */
                for (i = 0; i < key - 1; i++)
                    temp = temp->next;
                ptr->next = temp->next;
                temp->next = ptr;
            }
        }
    }
    printf("\nUpdated list is:");
    print();
}
```

```
/* ---------- Insert at a position (1-based) ---------- */
void insert_at(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter location at which you want to insert:");
    scanf("%d", &key);
    while (key <= 0)
    {
        printf("\nEnter any other location at which you want to insert:");
        scanf("%d", &key);
    }
    if (key > count() + 1)        /* count()+1 means "append at end" */
        printf("\nSorry!!!You exceed the linked list length.");
    else
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            printf("\nEnter value to be inserted:");
            scanf("%d", &ptr->data);
            if (key == 1)
            {
                ptr->next = start;
                start = ptr;
            }
            else
            {
                /* reach node at position key-1 */
                for (i = 0; i < key - 2; i++)
                    temp = temp->next;
                ptr->next = temp->next;
                temp->next = ptr;
            }
        }
    }
    printf("\nUpdated list is:");
    print();
}

/* ---------- Delete at beginning ---------- */
void del_beg(void)
{
    struct node *temp = start;
    if (temp == NULL)
        printf("\nLinked list is empty.");
    else
    {
        printf("\nDeleted value from link list is:%d.", start->data);
        start = start->next;
        free(temp);
    }
}

/* ---------- Delete at end ---------- */
void del_end(void)
{
    struct node *temp = start;
    if (temp == NULL)
        printf("\nLinked list is empty.");
    else
    {
        if (temp->next == NULL)            /* only one node */
        {
            printf("\nDeleted value from link list is: %d", temp->data);
            start = NULL;
            free(temp);
        }
        else
        {
            /* stop at second-last node */
            while (temp->next->next != NULL)
                temp = temp->next;
```

```
            printf("\nDeleted value from link list is:%d.",
                   temp->next->data);
            free(temp->next);
            temp->next = NULL;
        }
        printf("\nUpdated list is:");
        print();
    }
}

/* ---------- Delete the node after a given value ---------- */
void del_after(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter after which value you want to delete:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    else
    {
        for (i = 0; i < key; i++)
            temp = temp->next;
        ptr = temp->next;
        if (ptr == NULL)
            printf("\nNo value available to delete.");
        else
        {
            printf("\nDeleted value from link list is:%d.", ptr->data);
            temp->next = ptr->next;
            free(ptr);
        }
    }
    printf("\nUpdated list is:");
    print();
}

/* ---------- Delete the node before a given value ---------- */
void del_before(void)
{
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter before which value you want to delete:");
    scanf("%d", &key);
    key = search(key);
    if (key == -1 || key == -2)
        printf("\nSorry!!!Value not present in list.");
    /* key in first node: nothing before it */
    else if (key == 0)
        printf("\nNo value present before:%d to delete.", start->data);
    else if (key == 1)                      /* delete the first node */
    {
        start = temp->next;
        printf("\nDeleted value from link list is:%d.", temp->data);
        free(temp);
    }
    else
    {
        /* reach node two places before key */
        for (i = 0; i < key - 2; i++)
            temp = temp->next;
        ptr = temp->next;
        printf("\nDeleted value from link list is:%d.", ptr->data);
        temp->next = ptr->next;
        free(ptr);
    }
    printf("\nUpdated list is:");
    print();
}

/* ---------- Delete at a position (1-based) ---------- */
void del_at(void)
{
```

```
    struct node *ptr, *temp = start;
    int key, i;
    printf("\nEnter location at which you want to delete:");
    scanf("%d", &key);
    while (key <= 0)
    {
        printf("\nEnter another location:");
        scanf("%d", &key);
    }
    if (key > count())
        printf("\nSorry!!!You exceed the linked list length.");
    else
    {
        if (key == 1)
        {
            start = start->next;
            printf("\nDeleted value from link list is:%d.", temp->data);
            free(temp);
        }
        else
        {
            for (i = 0; i < key - 2; i++)
                temp = temp->next;
            ptr = temp->next;
            printf("\nDeleted value from link list is:%d.", ptr->data);
            temp->next = ptr->next;
            free(ptr);
        }
    }
    printf("\nUpdated list is:");
    print();
}

/* ---------- Reverse (in place, three pointers) ---------- */
void reverse(void)
{
    struct node *prev = NULL, *curr = start, *nextNode;
    while (curr != NULL)
    {
        nextNode = curr->next;   /* remember the rest of the list */
        curr->next = prev;       /* reverse this link */
        prev = curr;             /* move prev one step */
        curr = nextNode;         /* move curr one step */
    }
    start = prev;
    printf("\nReversed list is:");
    print();
}

/* ---------- Reverse (slide method) ----------
   delete from front of list 1, insert at front of list 2 */
int rev1(void)
{
    struct node *temp = start;
    int val = 0;
    if (temp == NULL)
        printf("\nLinked list is empty.");
    else
    {
        val = start->data;
        start = start->next;
        free(temp);
    }
    return val;
}

void rev2(int val)
{
    struct node *ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr != NULL)
    {
        ptr->data = val;
        ptr->next = start1;
        start1 = ptr;
```

```
    }
}

void reverse_by_copy(void)
{
    int i, j;
    start1 = NULL;
    j = count();
    for (i = 0; i < j; i++)
        rev2(rev1());
    start = start1;
    start1 = NULL;
    printf("\nReversed list is:");
    print();
}

/* ---------- Sort ascending (swap data, selection-style) ---------- */
void sort(void)
{
    struct node *temp1 = start, *temp2;
    int tmp;
    if (temp1 == NULL)
        printf("\nLinked list is empty.");
    else
    {
        while (temp1->next != NULL)
        {
            temp2 = temp1->next;
            while (temp2 != NULL)
            {
                if (temp1->data > temp2->data)   /* use < for descending */
                {
                    tmp = temp1->data;
                    temp1->data = temp2->data;
                    temp2->data = tmp;
                }
                temp2 = temp2->next;
            }
            temp1 = temp1->next;
        }
    }
    printf("\nUpdated list is:");
    print();
}

/* ---------- Copy list into start1 and show both ---------- */
void copy(void)
{
    struct node *ptr, *temp = start, *last = NULL;
    start1 = NULL;
    while (temp != NULL)
    {
        ptr = (struct node *)malloc(sizeof(struct node));
        if (ptr != NULL)
        {
            ptr->data = temp->data;
            ptr->next = NULL;
            if (last == NULL)
                start1 = ptr;
            else
                last->next = ptr;
            /* keep a tail pointer: no re-traversal needed */
            last = ptr;
        }
        temp = temp->next;
    }
    printf("\nOriginal\tCopy\n");
    temp = start;
    ptr = start1;
    while (temp != NULL && ptr != NULL)
    {
        printf("%d (%p)\t%d (%p)\n", temp->data, (void *)temp,
               ptr->data, (void *)ptr);
        temp = temp->next;
```

```
        ptr = ptr->next;
    }
    while (start1 != NULL)       /* free the copy */
    {
        ptr = start1;
        start1 = start1->next;
        free(ptr);
    }
}

int main(void)
{
    int ch, key, pos;
    do
    {
        printf("\n 1.Create 2.Display 3.InsBeg 4.InsEnd 5.InsAfter");
        printf("\n 6.InsBefore 7.InsAt 8.DelBeg 9.DelEnd 10.DelAfter");
        printf("\n 11.DelBefore 12.DelAt 13.Search 14.Count 15.Reverse");
        printf("\n 16.ReverseByCopy 17.Sort 18.Copy 0.Exit\nEnter choice:");
        if (scanf("%d", &ch) != 1)
            break;
        switch (ch)
        {
        case 1: create(); break;
        case 2: print(); break;
        case 3: insert_beg(); break;
        case 4: insert_end(); break;
        case 5: insert_after(); break;
        case 6: insert_before(); break;
        case 7: insert_at(); break;
        case 8: del_beg(); break;
        case 9: del_end(); break;
        case 10: del_after(); break;
        case 11: del_before(); break;
        case 12: del_at(); break;
        case 13:
            printf("\nEnter value to search:");
            scanf("%d", &key);
            pos = search(key);
            if (pos < 0)
                printf("\nValue not found.\n");
            else
                printf("\nValue found at position %d (1-based)\n", pos + 1);
            break;
        case 14: printf("\nNumber of nodes = %d\n", count()); break;
        case 15: reverse(); break;
        case 16: reverse_by_copy(); break;
        case 17: sort(); break;
        case 18: copy(); break;
        }
    } while (ch != 0);
    while (start != NULL)
        del_beg();
    return 0;
}
```

Tested input sequence and actual results:

```
Create 10 20 30 40 -1          -> 10 20 30 40
Insert at beginning 5          -> 5 10 20 30 40
Insert at end 50               -> 5 10 20 30 40 50
Insert after 20 value 25       -> 5 10 20 25 30 40 50
Insert before 10 value 7       -> 5 7 10 20 25 30 40 50
Insert before 30 value 27      -> 5 7 10 20 25 27 30 40 50
Insert at position 1 value 1   -> 1 5 7 10 20 25 27 30 40 50
Insert at position 10 value 99 -> 1 5 7 10 20 25 27 30 40 99 50
Insert at position 9 value 60  -> 1 5 7 10 20 25 27 30 60 40 99 50
Count                          -> Number of nodes = 12
Search 27                      -> found at position 7 (1-based)
Search 100                     -> Value not found.
Delete at beginning            -> deleted 1
Delete at end                  -> deleted 50
Delete after 25                -> deleted 27
Delete before 7                -> deleted 5
Delete before 30               -> deleted 25
Delete at position 2           -> deleted 10
Delete at position 100         -> exceed the linked list length
Reverse                        -> 99 40 60 30 20 7
Reverse by copy (slide method) -> 7 20 30 60 40 99
Sort                           -> 7 20 30 40 60 99
```

All operations ran without memory errors under AddressSanitizer.

8.23 Chapter 8 Exam Preparation

Quick Revision

- Node = data + next. start = NULL means empty. Last node's next = NULL.
- **Insert at beginning:** ptr->next = start; start = ptr. O(1).
- **Insert at end:** walk to node whose next is NULL; link. O(n).
- **Insert after X:** ptr->next = X->next; X->next = ptr.
- **Insert before X / delete X:** stop at the PREVIOUS node.
- **Delete at beginning:** temp = start; start = start->next; free(temp). O(1).
- **Delete at end:** stop at second-last; free last; second-last->next = NULL.
- **Always handle:** empty list, one-node list, first node, last node.
- **Reverse:** prev/curr/next, O(n) time, O(1) space.

Important Definitions

- **Linked list:** a linear collection of nodes where each node holds data and a pointer to the next node; order is given by links, not memory position.
- **Self-referential structure:** a structure containing a pointer to a structure of the same type.
- **Head/START:** pointer to the first node.
- **Traversal:** visiting every node once from START to NULL.

COMPLEXITY SUMMARY (singly linked list, only START pointer)

Operation	Time	Auxiliary space
Traverse / display / count	O(n)	O(1)
Search	O(1) best, O(n) worst	O(1)
Insert at beginning	O(1)	O(1)
Insert at end	O(n)	O(1)
Insert after/before value	O(n)	O(1)
Insert at position k	O(k) / O(n)	O(1)
Delete at beginning	O(1)	O(1)
Delete at end	O(n)	O(1)
Delete after/before/at	O(n)	O(1)
Reverse (3 pointers)	O(n)	O(1)
Sort (slide method)	O(n^2)	O(1)
Copy	O(n)	O(n)
Create n nodes (append each)	O(n^2)	O(n) for nodes

Common Mistakes

- **Losing the list:** start = ptr before ptr->next = start.
- Not handling the empty list (temp->next when temp is NULL -> crash).
- **Delete at end with one node:** temp->next->next crashes.
- Forgetting free() after unlinking (memory leak).
- Using the node after free().
- Traversal with while(temp->next != NULL) when the last node must also be processed.
- Mixing 0-based index (search) with 1-based position (insert_at).

Practice Questions

1. [S] What is a linked list? Explain its representation in C.

   Ans: 8.1, 8.2 with struct node and diagram.
2. [S] List and explain the types of linked list with diagrams. Ans: 8.3.
3. [S] Write C functions to insert a node at the beginning and at the end of a singly linked list. Ans: 8.10, 8.11.
4. [S] Write a function to delete a node at the end. Why is the single-node case handled separately? Ans: 8.16.
5. [S] Write functions to insert after and before a given element.

   Ans: 8.12, 8.13.
6. [S] Write functions to insert and delete at a given position.

   Ans: 8.14, 8.19.
7. [S] Write a function to reverse a singly linked list and trace it.

   Ans: 8.20.1 table.
8. [S] Write functions to count nodes and search an element. Ans: 8.8, 8.9.
9. [S] Write a function to sort a linked list. Ans: 8.21.1.
10. [A] List is 5 -> 15 -> 25. Show the list after: insert 1 at beginning, insert 30 at end, delete after 15, insert 20 before 30, delete at position 2.

    Ans: 1 5 15 25 -> 1 5 15 25 30 -> 1 5 15 30 -> 1 5 15 20 30 -> delete position 2 (5) -> 1 15 20 30.

11. [A] What does this print for list 1 -> 2 -> 3 -> 4? struct node *t = start; while (t != NULL && t->next != NULL) { printf("%d ", t->data); t = t->next->next; }

Ans: 1 3

12. [A] Give the time complexity of inserting at the end with and without a last pointer, and explain.

Ans: Without: O(n) - must walk to the last node. With: O(1) - last->next = ptr; last = ptr.

Chapter 9: Linked List Lab Programs (LL Prog Folder)

Sources: *Module 2 "LL Prog" folder - "create List & Display.docx", "Count Number of nodes.docx", "sum and Avg.docx", "SEARCH element.doc", "REVERSEL.doc", "EMP.docx". (CIR_SLL.C is in Chapter 10 and Linked QUEUE.C in Chapter 4.)*

These programs use a different style from the lecture slides:

- typedef struct node Node; so "Node" can be written instead of "struct node".
- The list pointer l is passed to and returned from functions (no global start). This is the style most textbooks use.
- createNode(d) allocates and fills one node; createList(l) keeps asking "Do u wish to insert(1/0)".

9.1 Common Building Blocks: createNode and createList

Function: createNode() [from ll_prog.c]

```
Node *createNode(int d)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL)
    {
        printf("Memory not available\n");
        exit(1);
    }
    newNode->data = d;
    newNode->next = NULL;
    return newNode;
}
```

Function: createList() [from ll_prog.c]

```
Node *createList(Node *l)
{
    int ch, d;
    Node *newNode, *t;
    do
    {
        printf("Enter the data: ");
        scanf("%d", &d);
        newNode = createNode(d);
        if (l == NULL)
            l = newNode;
        else
        {
            t = l;
            while (t->next != NULL)
                t = t->next;
            t->next = newNode;
        }
        printf("Do u wish to insert(1/0): ");
        scanf("%d", &ch);
    } while (ch == 1);
    return l;
}
```

Explanation:

- createNode(): malloc one node, store d, set next = NULL, return its address. [CORRECTION] The originals do not check for NULL; the corrected version exits with a message if malloc fails.
- createList(l): do-while so at least one node is created. If the list is empty the new node becomes the first node (l = newNode); otherwise t walks to the last node and links the new node. Finally the (possibly new) first-node pointer is returned, so main must write l = createList(l);

- Why return l? Because l is passed BY VALUE: changing l inside the function does not change main's l. Returning it (or passing &l) is required. This is a favourite viva question.

Dry run (input 10,1,20,1,30,0):

```
d=10: l NULL -> l = [10]
d=20: t = 10 -> 10->next = [20]
d=30: t = 10 -> 20 -> 20->next = [30]
ch=0: stop, return l (points to 10)
```

9.2 Display

Function: display() [from ll_prog.c]

```
void display(Node *l)
{
    Node *t = l;
    while (t != NULL)
    {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}
```

Walk t from l to NULL printing each data. O(n).

9.3 Count Number of Nodes (Count Number of nodes.docx)

Function: countNodes() [from ll_prog.c]

```
void countNodes(Node *l)
{
    Node *t = l;
    int count = 0;
    while (t != NULL)
    {
        count++;
        t = t->next;
    }
    printf("No of Nodes = %d\n", count);
}
```

count increases once per node. For 10 20 30 40 -> "No of Nodes = 4". O(n).

9.4 Sum and Average of All Elements (sum and Avg.docx)

Function: sumAvg() [from ll_prog.c]

```
void sumAvg(Node *l)
{
    Node *t = l;
    int count = 0, sum = 0;
    float avg;
    while (t != NULL)
    {
        count++;
        sum = sum + t->data;
        t = t->next;
    }
    if (count == 0)
    {
        printf("List is empty\n");
        return;
    }
    avg = (float)sum / count;      /* cast avoids integer division */
    printf("Sum = %d\nAverage = %f\n", sum, avg);
}
```

Explanation:

- One traversal computes both count and sum.
- avg = (float)sum / count; the cast is essential. Without it, sum/count is integer division (e.g. 7/2 = 3) and only then converted to float.
- [CORRECTION] The original divides by count without checking for 0. Its do-while always creates at least one node, so it never happens there, but a general function must check (added).

Dry run 10 20 30 40: count 4, sum 100 -> Sum = 100, Average = 25.000000.

Complexity: O(n) time, O(1) space.

9.5 Search an Element (SEARCH element.doc)

Function: search() [from ll_prog.c]

```
int search(Node *l, int skey)
{
    Node *t = l;
    int position = 1;
    while (t != NULL)
    {
        if (t->data == skey)
            return position;
        position++;
        t = t->next;
    }
    return -1;
}
```

- Returns the 1-based POSITION (first node = 1), or -1 if not found. (Note the difference from the slide search() in 8.9, which returns a 0-based index and uses -1/-2 codes.)

Dry run search 30 in 10 20 30 40: position 1 (10), 2 (20), 3 (30) match -> "Element found at position 3".

Complexity: best O(1), worst O(n).

9.6 Reverse a Linked List (REVERSEL.doc)

Original code from the notes:

```
Node *reverse(Node *l)
{
    Node *p = l->next, *q = p->next;
    l->next = NULL;
    while (p != NULL)
    {
        p->next = l;
        l = p;
        p = q;
        q = p->next;     /* <-- crashes when p becomes NULL */
    }
    return l;
}
```

[CORRECTION] **Problems:**

1. In the last iteration p becomes NULL and then q = p->next dereferences NULL - the program crashes (verified: the original exits with a segmentation fault for a 3-node list).
2. For a one-node list, q = p->next runs with p = NULL at the very start -> crash. For an empty list, l->next crashes. The idea (p walks forward, each node is pointed back to l) is correct. The fix is to read the next node at the START of each iteration:

Function: reverse() [from ll_prog.c]

```
Node *reverse(Node *l)
{
    Node *prev = NULL, *curr = l, *nextNode;
    while (curr != NULL)
    {
        nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;                    /* new first node */
}
```

Mapping: original l ~ prev, p ~ curr, q ~ nextNode.

Dry run 10 20 30 40 -> returns 40; display gives 40 30 20 10 (see 8.20.1 for a step table). Works for 0, 1 or many nodes.

Complexity: O(n) time, O(1) space.

9.7 Combined Program (all of the above)

The separate .docx files each repeat createNode/createList with one extra function. Here they are combined into one tested program. To write a single lab program, keep struct node, createNode, createList, the one function you need, and a main that calls it.

Program: ll_prog.c

```
/* Linked list lab programs: create, display, count,
   sum & average, search, reverse */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};
typedef struct node Node;

Node *createNode(int d)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL)
    {
        printf("Memory not available\n");
        exit(1);
    }
    newNode->data = d;
    newNode->next = NULL;
    return newNode;
}

Node *createList(Node *l)
{
    int ch, d;
    Node *newNode, *t;
    do
    {
        printf("Enter the data: ");
        scanf("%d", &d);
        newNode = createNode(d);
        if (l == NULL)
            l = newNode;
        else
        {
            t = l;
            while (t->next != NULL)
                t = t->next;
            t->next = newNode;
```

```
        }
        printf("Do u wish to insert(1/0): ");
        scanf("%d", &ch);
    } while (ch == 1);
    return l;
}

void display(Node *l)
{
    Node *t = l;
    while (t != NULL)
    {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

void countNodes(Node *l)
{
    Node *t = l;
    int count = 0;
    while (t != NULL)
    {
        count++;
        t = t->next;
    }
    printf("No of Nodes = %d\n", count);
}

void sumAvg(Node *l)
{
    Node *t = l;
    int count = 0, sum = 0;
    float avg;
    while (t != NULL)
    {
        count++;
        sum = sum + t->data;
        t = t->next;
    }
    if (count == 0)
    {
        printf("List is empty\n");
        return;
    }
    avg = (float)sum / count;      /* cast avoids integer division */
    printf("Sum = %d\nAverage = %f\n", sum, avg);
}

int search(Node *l, int skey)
{
    Node *t = l;
    int position = 1;
    while (t != NULL)
    {
        if (t->data == skey)
            return position;
        position++;
        t = t->next;
    }
    return -1;
}

Node *reverse(Node *l)
{
    Node *prev = NULL, *curr = l, *nextNode;
    while (curr != NULL)
    {
        nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
```

```
    return prev;                    /* new first node */
}

void freeList(Node *l)
{
    Node *t;
    while (l != NULL)
    {
        t = l;
        l = l->next;
        free(t);
    }
}

int main(void)
{
    Node *l = NULL;
    int skey, position;
    l = createList(l);
    printf("\nList\n");
    display(l);
    countNodes(l);
    sumAvg(l);
    printf("Enter data to be searched: ");
    scanf("%d", &skey);
    position = search(l, skey);
    if (position == -1)
        printf("Element not found\n");
    else
        printf("Element found at position %d\n", position);
    l = reverse(l);
    printf("Reversed List\n");
    display(l);
    freeList(l);
    return 0;
}
```

Actual run (input: 10 20 30 40, search 30):

```
List
10 20 30 40
No of Nodes = 4
Sum = 100
Average = 25.000000
Element found at position 3
Reversed List
40 30 20 10
```

9.8 Employee List Program (EMP.docx)

Problem statement [SOURCE]: Consider a list that stores information about employees in an organisation, where each node contains Employee Id, Employee Name, Designation and Salary. Write a C program to display the details of all employees whose salary is above 5000.

Program: emp.c

```
/* Employee list: display employees whose salary is above 5000 */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int empid;
    char empName[20];
    char designation[15];
    int salary;
    struct node *next;
};
typedef struct node Node;
```

```
Node *createNode(void)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL)
    {
        printf("Memory not available\n");
        exit(1);
    }
    printf("Enter employee Number: ");
    scanf("%d", &newNode->empid);
    printf("Enter employee Name: ");
    /* width limit prevents overflow */
    scanf("%19s", newNode->empName);
    printf("Enter employee designation: ");
    scanf("%14s", newNode->designation);
    printf("Enter employee Salary: ");
    scanf("%d", &newNode->salary);
    newNode->next = NULL;
    return newNode;
}

Node *createList(Node *l)
{
    int ch;
    Node *newNode, *t;
    do
    {
        newNode = createNode();
        if (l == NULL)
            l = newNode;
        else
        {
            t = l;
            while (t->next != NULL)
                t = t->next;
            t->next = newNode;
        }
        printf("Do u wish to insert(1/0): ");
        scanf("%d", &ch);
    } while (ch == 1);
    return l;
}

void display(Node *l)
{
    Node *t = l;
    while (t != NULL)
    {
        if (t->salary > 5000)
            printf("%d    %s     %s     %d\n", t->empid, t->empName,
                   t->designation, t->salary);
        t = t->next;
    }
}

int main(void)
{
    Node *l = NULL, *t;
    l = createList(l);
    printf("\nEmployees having salary greater than 5000\n");
    display(l);
    while (l != NULL)
    {
        t = l;
        l = l->next;
        free(t);
    }
    return 0;
}
```

Explanation:

- The node holds several fields; next is still the link.
- createNode() reads all four fields. scanf("%19s") reads a word of at most 19 characters (leaving room for '\0'), protecting the 20-char array.
- display() traverses the whole list but prints only nodes where salary > 5000 - a traversal with a condition (filtering).
- [CORRECTION] The original heading says "salary greater than 50000" but the code tests > 5000. The heading is fixed to 5000 to match the problem statement. The original also uses scanf("%s") without a width (a long name overflows the array) and never frees the list.
- Names with spaces cannot be read by %s (it stops at a space).

Actual run:

```
Input: (1, Ravi, Manager, 8000) (2, Sita, Clerk, 4000)
       (3, Amit, Engineer, 12000)
Output:
   Employees having salary greater than 5000
   1    Ravi     Manager     8000
   3    Amit     Engineer     12000
```

Complexity: O(n) to create (with append walk O(n^2) overall) and O(n) to display.

9.9 Chapter 9 Exam Preparation

Quick Revision

- typedef struct node Node; createNode(d) returns a filled node.
- l = createList(l): the function must return the head because C passes pointers by value.
- Count / sum / average / search / filtered display are all single traversals: O(n).
- **Average:** (float)sum / count.
- Correct reversal reads next at the START of the loop.

Common Mistakes

- Calling createList(l) without assigning l = createList(l) -> l stays NULL in main.
- Integer division in the average.
- The REVERSEL crash (reading p->next after p became NULL).
- Message text not matching the condition (5000 vs 50000).

Practice Questions

1. [S] Write a C program to create a singly linked list and display it.
   Ans: 9.1, 9.2 + main.
2. [S] Write a C function to count the nodes of a list. Ans: 9.3.
3. [S] Write a C program to find the sum and average of all elements in a singly linked list. Ans: 9.4.
4. [S] Write a C function to search an element and return its position.
   Ans: 9.5.
5. [S] Write a C function to reverse a singly linked list. Ans: 9.6.
6. [S] Employee list: display employees with salary above 5000.
   Ans: 9.8.

7. [A] Find the bug: Node *reverse(Node *l){ Node *p=l->next,*q=p->next; … q = p->next; } for a 1-node list.

Ans: p = l->next is NULL, so q = p->next dereferences NULL.

8. [A] Modify sumAvg to also print the largest element.

Ans: keep max = l->data before the loop; inside, if (t->data > max) max = t->data; print max after the loop (check l != NULL first).

9. [A] Why does createList need to return the list pointer?

Ans: l is a copy inside the function; assigning l = newNode changes only the copy, so the new head must be returned (or passed as Node **).

Chapter 10: Circular Linked List

Sources: *pptx slides 67-74 (circular singly linked list functions); program "CIR_SLL.C" (Module 2 LL Prog). Lecture outcome: "Implement insertion and deletion operations on circular singly linked lists using dynamic memory allocation in C."*

10.1 Definition and Concept

Definition (exam-ready):

> **A circular linked list is a linked list in which the last node points back to the first node instead of containing NULL, so the nodes form a circle.**

[SOURCE] The node structure is the same as a singly linked list:

```
struct node { int data; struct node *next; };
```

[SOURCE figure]

```
Head --> [12| ]--> [56| ]--> [2| ]--> [11| ]--+
          ^                                   |
          +-----------------------------------+
```

Key properties:

- No node has next = NULL (except in an empty list, where start = NULL).
- From any node you can reach every other node.
- **A single node points to itself:** start->next == start.
- Traversal must stop when we come BACK to start (not at NULL), otherwise the loop never ends.

Why keep a LAST pointer: with only start, inserting at the beginning needs the last node (to update last->next), which takes O(n) to find. Keeping a pointer to the last node makes insert at beginning, insert at end and delete at beginning all O(1). (In fact last alone is enough, since last->next is the first node.)

Advantages [SUPPLEMENT]**:** any node can be a starting point; useful for round-robin scheduling and repeating playlists; going from the last node to the first is O(1).

Disadvantages: easy to create an infinite loop; reversing and some operations are a little more complex.

10.2 Traversal / Print

[SOURCE slide 69] uses a do-while loop:

```
   do { print temp->data; temp = temp->next; } while (temp != start);
A do-while is used because temp starts AT start; a while(temp != start)
loop would stop immediately without printing anything.
```

[CORRECTION] **The slide has an extra bracket:** while((temp!=start);

10.3 Algorithms (with START and LAST pointers)

10.3.1 Insert at beginning

```
Step 1: Allocate PTR, set its data
Step 2: IF START = NULL: START = LAST = PTR ; PTR->NEXT = PTR
        ELSE: PTR->NEXT = START ; START = PTR ; LAST->NEXT = START
Step 3: END
```

```
Before: START->[10]->[20]->[30]--+     LAST = 30
          ^----------------------+
Insert 5: [5]->[10]->[20]->[30]--+     START = 5, 30->next = 5
          ^----------------------+
```

10.3.2 Insert at end

```
Step 1: Allocate PTR, set its data
Step 2: IF START = NULL: START = LAST = PTR ; PTR->NEXT = PTR
        ELSE: LAST->NEXT = PTR ; PTR->NEXT = START ; LAST = PTR
Step 3: END
```

10.3.3 Delete at beginning

```
Step 1: IF START = NULL, PRINT "empty" and stop
Step 2: TEMP = START
Step 3: IF START = LAST (one node): START = LAST = NULL
        ELSE: START = START->NEXT ; LAST->NEXT = START
Step 4: FREE TEMP
```

10.3.4 Delete at end

```
Step 1: IF START = NULL, PRINT "empty" and stop
Step 2: IF START = LAST: FREE START ; START = LAST = NULL ; stop
Step 3: TEMP = START; while TEMP->NEXT != LAST: TEMP = TEMP->NEXT
Step 4: TEMP->NEXT = START ; FREE LAST ; LAST = TEMP
```

```
Before: [10]->[20]->[30]--+ (LAST = 30)
temp stops at 20 ; 20->next = 10 ; free 30 ; LAST = 20
After : [10]->[20]--+
         ^----------+
```

10.4 Program: Circular Singly Linked List (CIR_SLL.C, corrected)

Program: circular_sll.c

```
/* Circular singly linked list with first and last pointers
   (corrected CIR_SLL.C) */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int num;
    struct node *next;
};
typedef struct node NODE;

/* global so every function updates both */
NODE *first = NULL, *last = NULL;

void insertFirst(int item)
{
    NODE *p = (NODE *)malloc(sizeof(NODE));
    if (p == NULL)
        return;
    p->num = item;
    /* empty list: single node points to itself */
    if (first == NULL)
    {
        first = last = p;
        p->next = p;
    }
    else
    {
        p->next = first;
        first = p;
        last->next = first;         /* keep the circle closed */
    }
```

```
}

void insertLast(int item)
{
    NODE *p = (NODE *)malloc(sizeof(NODE));
    if (p == NULL)
        return;
    p->num = item;
    if (first == NULL)
    {
        first = last = p;
        p->next = p;
    }
    else
    {
        last->next = p;
        p->next = first;
        last = p;
    }
}

void deleteFirst(void)
{
    NODE *temp;
    if (first == NULL)
    {
        printf("list is empty\n");
        return;
    }
    temp = first;
    printf("Element deleted: %d\n", temp->num);
    if (first == last)              /* only one node */
        first = last = NULL;
    else
    {
        first = first->next;
        last->next = first;
    }
    free(temp);
}

void deleteLast(void)
{
    NODE *temp;
    if (first == NULL)
    {
        printf("list is empty\n");
        return;
    }
    printf("Element deleted: %d\n", last->num);
    if (first == last)
    {
        free(first);
        first = last = NULL;
    }
    else
    {
        temp = first;
        while (temp->next != last)  /* find the node before last */
            temp = temp->next;
        temp->next = first;
        free(last);
        last = temp;
    }
}

void display(void)
{
    NODE *temp = first;
    if (temp == NULL)
    {
        printf("List is empty\n");
        return;
    }
```

```
    printf("Elements of list are: ");
    do
    {
        printf("%d->", temp->num);
        temp = temp->next;
    } while (temp != first);        /* stop after one full round */
    printf("(back to %d)\n", first->num);
}

int main(void)
{
    int choice, item;
    do
    {
        printf("\n1.InsBegin 2.InsEnd 3.DelBegin 4.DelEnd 5.Exit");
        printf("\nEnter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice)
        {
        case 1:
            printf("Enter the number: ");
            scanf("%d", &item);
            insertFirst(item);
            display();
            break;
        case 2:
            printf("Enter the number: ");
            scanf("%d", &item);
            insertLast(item);
            display();
            break;
        case 3:
            deleteFirst();
            display();
            break;
        case 4:
            deleteLast();
            display();
            break;
        case 5:
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 5);
    while (first != NULL)
        deleteFirst();
    return 0;
}
```

Code explanation:

- first and last are GLOBAL, so every function updates the real pointers.
- insertFirst/insertLast: the empty case makes the node point to itself; otherwise last->next is always set back to first to keep the circle.
- deleteFirst: special case for one node; otherwise first moves forward and last->next is updated.
- deleteLast: finds the node before last by walking until temp->next == last.
- display: do-while from first until we return to first.

[CORRECTION] Bugs in the notes' CIR_SLL.C (each was confirmed by running the original):

1. insertfirst(first, last) changes only its local copy of last. main assigns only first = insertfirst(...), so main's last stays NULL. The second insertfirst then executes last->next = p with last = NULL -> segmentation fault (observed).
2. insertlast returns first when the list was empty but main stores the result in last, so main's first stays NULL; display then says "List is empty" although a node exists (observed).

3. On the first insertion p->next is never set (should point to itself).
4. deletefirst with a single node does not empty the list: first is set to first->next, which is garbage (or the freed node itself).
5. display stops when temp->next == last, so the LAST node is never printed; for a single node it reads an uninitialised pointer.
6. Missing #include <stdlib.h>; printf("\Invalid choice") has an invalid escape "\I". The fix keeps first and last as globals (alternatively pass NODE **).

[CORRECTION] **Bugs in the slide functions (pptx slides 70-74):**

- insert_end declares a LOCAL "last" that is uninitialised and hides the global one.
- create() keeps last as a local variable, so insert_beg/del_beg (which use a global last) do not see it.
- del_beg with one node sets start = start->next (itself) and frees it, leaving start dangling; it should set start = last = NULL.
- del_end is missing a closing brace.

Actual sample runs:

```
InsertBegin 10    -> Elements of list are: 10->(back to 10)
InsertBegin 20    -> 20->10->(back to 20)
InsertEnd 30      -> 20->10->30->(back to 20)
InsertEnd 40      -> 20->10->30->40->(back to 20)
(second run) InsertEnd 10 ; DeleteBegin -> Element deleted: 10 ; List is
empty ; DeleteEnd -> list is empty ; InsertBegin 5 ; DeleteEnd ->
Element deleted: 5 ; List is empty
```

Complexity (with first and last pointers):

Operation	Time
Insert at beginning	O(1)
Insert at end	O(1)
Delete at beginning	O(1)
Delete at end	O(n)
Display	O(n)

Delete at end is O(n) because the node before last must be found.

10.5 Singly vs Circular Linked List

Point	Singly linked list	Circular linked list
Last node's next	NULL	Points to first node
End of traversal	temp == NULL	temp == start (do-while)
First from last	Not possible	O(1): last->next
Infinite loop	Low risk	High if the stop condition is wrong
Insert at begin	O(1)	O(1) with last pointer
Insert at end	O(n) (O(1) with last ptr)	O(1) with last pointer
Uses	General lists, stacks, queues	Round robin, circular buffers, playlists

10.6 Chapter 10 Exam Preparation

Quick Revision

- Last node points to first. Single node points to itself.
- Traverse with do { ... } while (temp != start).

- After every insert/delete at an end, re-close the circle: last->next = first.
- Keep first and last global (or pass by address) so changes persist.

Important Definitions

- **Circular linked list:** a linked list whose last node points to the first node.

Common Mistakes

- while (temp != start) instead of do-while -> prints nothing.
- while (temp != NULL) on a circular list -> infinite loop.
- Forgetting to update last->next after changing first.
- Passing first/last by value and expecting the caller's pointers to change.
- Not handling the single-node case in delete.

Practice Questions

1. [S] What is a circular linked list? Draw it. Ans: 10.1.
2. [S] Write functions to insert at the beginning and end of a circular singly linked list. Ans: 10.3.1, 10.3.2, program 10.4.
3. [S] Write functions to delete at the beginning and end of a circular singly linked list. Ans: 10.3.3, 10.3.4.
4. [S] Write a function to display a circular linked list. Why is a do-while used? Ans: 10.2.
5. [A] Differentiate singly and circular linked lists. Ans: 10.5.
6. [A] List the bugs in the given CIR_SLL.C and correct them.

   Ans: 10.4 correction list.
7. [A] Circular list 1 -> 2 -> 3 (last = 3). Show first, last and links after: insert 0 at beginning, delete at end, insert 9 at end.

   Ans: 0 1 2 3 (first 0, 3->0) ; delete end -> 0 1 2 (last 2, 2->0) ; insert 9 -> 0 1 2 9 (last 9, 9->0).

Chapter 11: Doubly Linked List and Circular Doubly Linked List

Sources: *pptx slides 75-81. Lecture outcomes: "Implement basic operations on doubly linked lists using forward and backward pointers in C" and "Implement insertion and deletion operations on circular doubly linked lists maintaining bi-directional circular links in C" (slide 81 has only this outcome; its content below is* [SUPPLEMENT]*).*

11.1 Doubly Linked List: Definition and Representation

Definition (exam-ready):

> **A doubly linked list is a linked list in which each node contains three parts: a pointer to the previous node, the data, and a pointer to the next node, so the list can be traversed in both directions.**

[SOURCE]

```
struct node
{
    int data;
    struct node *prev;
    struct node *next;
};
```

[SOURCE figure]

```
Start                       next pointer
Pointer                         |
  |                             v
  +-->[X| A | ]<==>[ | B | ]<==>[ | C | ]<==>[ | D |X]
       ^                ^
     Null          prev pointer                    Null
```

- first node: prev = NULL. last node: next = NULL.
- **The programs keep two pointers:** start (first node) and last (last node).

Advantages [SUPPLEMENT]**:**

- Traversal in both directions (forward with next, backward with prev).
- Deleting a given node is easy - its previous node is node->prev, no search for the previous node is needed.
- With a last pointer, insert and delete at BOTH ends are O(1).

Disadvantages:

- Extra memory for the prev pointer in every node.
- Every insert/delete must update more pointers (more chance of error).

11.2 Algorithms

11.2.1 Insert at beginning

```
Step 1: Allocate PTR; PTR->DATA = VALUE; PTR->PREV = NULL
Step 2: PTR->NEXT = START
Step 3: IF START = NULL, LAST = PTR  ELSE START->PREV = PTR
Step 4: START = PTR
```

```
Before: START->[X|10| ]<=>[ |20|X]<-LAST
Insert 5: [X|5| ]<=>[ |10| ]<=>[ |20|X]    START = 5, 10->prev = 5
```

11.2.2 Insert at end

```
Step 1: Allocate LINK; LINK->DATA = VALUE; LINK->NEXT = NULL
Step 2: LINK->PREV = LAST
Step 3: IF START = NULL, START = LINK  ELSE LAST->NEXT = LINK
Step 4: LAST = LINK
```

11.2.3 Delete at beginning

```
Step 1: IF START = NULL, PRINT "empty" and stop
Step 2: TEMP = START ; START = START->NEXT
Step 3: IF START = NULL, LAST = NULL  ELSE START->PREV = NULL
Step 4: FREE TEMP
```

11.2.4 Delete at end

```
Step 1: IF LAST = NULL, PRINT "empty" and stop
Step 2: TEMP = LAST ; LAST = LAST->PREV
Step 3: IF LAST = NULL, START = NULL  ELSE LAST->NEXT = NULL
Step 4: FREE TEMP
```

```
Before: [X|10| ]<=>[ |20| ]<=>[ |30|X]  LAST = 30
After : [X|10| ]<=>[ |20|X]            LAST = 20 (via 30->prev), 30 freed
```

11.3 Program: Doubly Linked List

Program: dll.c

```
/* Doubly linked list: insert/delete at both ends,
   display forward and backward */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *start = NULL, *last = NULL;

void insert_beg(int value)
{
    struct node *ptr = (struct node *)malloc(sizeof(struct node));
    if (ptr == NULL)
        return;
    ptr->data = value;
    ptr->prev = NULL;
    ptr->next = start;
    if (start == NULL)
        last = ptr;            /* first node is also the last node */
    else
        start->prev = ptr;
    start = ptr;
}

void insert_end(int value)
{
    struct node *link = (struct node *)malloc(sizeof(struct node));
    if (link == NULL)
        return;
    link->data = value;
    link->next = NULL;
    link->prev = last;
    if (start == NULL)
        start = link;
    else
        last->next = link;
    last = link;
```

```
}

void delete_beg(void)
{
    struct node *temp = start;
    if (start == NULL)
    {
        printf("List is empty\n");
        return;
    }
    printf("Deleted Value: %d\n", temp->data);
    start = start->next;
    if (start == NULL)
        last = NULL;
    else
        start->prev = NULL;
    free(temp);
}

void delete_end(void)
{
    struct node *temp = last;
    if (last == NULL)
    {
        printf("List is empty\n");
        return;
    }
    printf("Deleted Value: %d\n", temp->data);
    last = last->prev;
    if (last == NULL)
        start = NULL;
    else
        last->next = NULL;
    free(temp);
}

void display(void)
{
    struct node *ptr = start;
    printf("Forward : ");
    while (ptr != NULL)
    {
        printf("%d\t", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

void display_reverse(void)
{
    struct node *ptr = last;
    printf("Backward: ");
    while (ptr != NULL)
    {
        printf("%d\t", ptr->data);
        ptr = ptr->prev;
    }
    printf("\n");
}

int main(void)
{
    int ch, value;
    do
    {
        printf("\n1.InsBeg 2.InsEnd 3.DelBeg 4.DelEnd 5.Display 6.Exit");
        printf("\nEnter choice: ");
        if (scanf("%d", &ch) != 1)
            break;
        switch (ch)
        {
        case 1:
            printf("Enter Data: ");
            scanf("%d", &value);
```

```
            insert_beg(value);
            break;
        case 2:
            printf("Enter Data: ");
            scanf("%d", &value);
            insert_end(value);
            break;
        case 3: delete_beg(); break;
        case 4: delete_end(); break;
        case 5: display(); display_reverse(); break;
        }
    } while (ch != 6);
    while (start != NULL)
        delete_beg();
    return 0;
}
```

Code explanation:

- insert_beg: the new node's prev is NULL and next is the old start; the old start's prev now points back to the new node.
- insert_end: uses last, so no traversal is needed - O(1).
- delete_beg / delete_end: after moving start/last, the new end node's outer pointer is set to NULL; if the list became empty, both start and last are NULL.
- display walks forward with next; display_reverse walks backward with prev starting from last - this proves both directions are linked correctly.

[CORRECTION] **Problems in the slide code (pptx slides 77-80):**

1. insert_beg never sets ptr->prev = NULL (garbage prev in the first node).
2. insert_end never sets link->next = NULL, and when the list is empty it sets last but never sets head/start, so the list stays "empty".
3. The slides mix the names start and head for the same pointer, use an undeclared variable value, and use curly ("smart") quotes which do not compile in C.
4. delete_end does not check for an empty list (head->next crashes when head is NULL). The corrected program fixes all of these and adds delete at beginning and backward display.

Actual run: InsBeg 10, InsBeg 5, InsEnd 20, InsEnd 30

```
Forward : 5 10 20 30
Backward: 30 20 10 5
DelBeg -> Deleted Value: 5 ; DelEnd -> Deleted Value: 30
Forward : 10 20 ; Backward: 20 10
DelEnd -> 20 ; DelEnd -> 10 ; DelEnd -> List is empty
```

Complexity (with start and last):

Insert at beginning	O(1)
Insert at end	O(1)
Delete at beginning	O(1)
Delete at end	O(1)
Display either way	O(n)

(A singly linked list needs O(n) for delete at end.)

11.4 Insert After / Delete a Given Node in a DLL [SUPPLEMENT]

Insert NEW after node P (P not last):

```
   NEW->prev = P;  NEW->next = P->next;
   P->next->prev = NEW;  P->next = NEW;
   [P]<=>[Q]   becomes   [P]<=>[NEW]<=>[Q]
(If P is the last node, set last = NEW and skip P->next->prev.)
```

Delete node P (middle node):

```
   P->prev->next = P->next;
   P->next->prev = P->prev;
   free(P);
(If P is first, update start; if P is last, update last.)
```

These show the main benefit of a DLL: no need to search for the previous node.

11.5 Circular Doubly Linked List [SUPPLEMENT]

> **Definition:** A circular doubly linked list is a doubly linked list in which the last node's next points to the first node and the first node's prev points to the last node.

```
START
  |
  v
[ |10| ] <==> [ |20| ] <==> [ |30| ]
  ^  ^                        |  |
  |  +----- 10.prev = 30 ----+  |      (first->prev = last)
  +-------- 30.next = 10 -------+      (last->next = first)
```

- No NULL pointers at all (except start = NULL for an empty list).
- The last node is start->prev, so no separate last pointer is needed and both ends are reachable in O(1).
- A single node has next = prev = itself.

Program (insert at end, delete at beginning, display):

Program: cdll.c

```
/* Circular doubly linked list:
   insert at end, delete at beginning, display */
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *start = NULL;

void insert_end(int value)
{
    struct node *ptr = (struct node *)malloc(sizeof(struct node));
    struct node *last;
    if (ptr == NULL)
        return;
    ptr->data = value;
    if (start == NULL)
    {
        /* single node points to itself both ways */
        ptr->next = ptr->prev = ptr;
        start = ptr;
    }
    else
    {
```

```
        /* last node is start->prev: no traversal */
        last = start->prev;
        ptr->next = start;
        ptr->prev = last;
        last->next = ptr;
        start->prev = ptr;
    }
}

void delete_beg(void)
{
    struct node *temp = start;
    if (start == NULL)
    {
        printf("List is empty\n");
        return;
    }
    printf("Deleted Value: %d\n", temp->data);
    if (start->next == start)            /* only one node */
        start = NULL;
    else
    {
        start->prev->next = start->next;
        start->next->prev = start->prev;
        start = start->next;
    }
    free(temp);
}

void display(void)
{
    struct node *ptr = start;
    if (ptr == NULL)
    {
        printf("List is empty\n");
        return;
    }
    do
    {
        printf("%d\t", ptr->data);
        ptr = ptr->next;
    } while (ptr != start);
    printf("\n");
}

int main(void)
{
    insert_end(10);
    insert_end(20);
    insert_end(30);
    display();
    delete_beg();
    display();
    printf("Last node via start->prev: %d\n", start->prev->data);
    while (start != NULL)
        delete_beg();
    return 0;
}
```

Actual output:

```
10 20 30
Deleted Value: 10
20 30
Last node via start->prev: 30
```

Explanation of insert_end: last = start->prev; the new node goes between last and start: ptr->next = start, ptr->prev = last, last->next = ptr, start->prev = ptr. Four links change; all O(1).

11.6 Singly vs Doubly Linked List

Point	Singly linked list	Doubly linked list
Node parts	data, next	prev, data, next
Traversal	Forward only	Forward and backward
Memory per node	1 pointer	2 pointers
Delete a given node	Need its previous node (search, O(n))	O(1) using node->prev
Delete at end	O(n)	O(1) with last pointer
Insert before a node	O(n) (find previous)	O(1) given the node
Complexity of code	Simpler	More pointer updates
Uses	Stacks, simple lists	Browser back/forward, undo/redo, music players

11.7 Comparison of All Linked List Types

Type	Pointers per node	Last node points to	Traversal	Typical use
Singly	1 (next)	NULL	Forward	Stack, queue, polynomials
Circular singly	1 (next)	First node	Forward, circular	Round robin, repeat playlist
Doubly	2 (prev, next)	NULL	Both ways	Back/forward navigation, deque
Circular doubly	2	First node (first->prev = last)	Both ways, circular	Music player (next/prev with loop)

11.8 Chapter 11 Exam Preparation

Quick Revision

- **DLL node:** prev, data, next. First prev = NULL, last next = NULL.
- With start and last, all four end operations are O(1).
- Insert/delete in a DLL changes up to 4 links; always fix both directions.
- **Circular DLL:** last->next = first and first->prev = last; last = start->prev.

Important Definitions

- **Doubly linked list:** each node has links to both previous and next nodes.
- **Circular doubly linked list:** a doubly linked list whose first and last nodes are linked to each other in both directions.

Common Mistakes

- Updating next but forgetting prev (backward traversal breaks).
- Not setting start when inserting the first node at the end (or last when inserting the first node at the beginning).
- Not setting the new end's outer pointer to NULL after a deletion.
- Using different names (head/start) for the same pointer.

Practice Questions

1. [S] What is a doubly linked list? Give its node structure. Ans: 11.1.
2. [S] Write functions to insert at the beginning and end of a DLL.

   Ans: 11.2.1, 11.2.2, program 11.3.

3. [S] Write a function to delete the last node of a DLL. Ans: 11.2.4.
4. [S] Write a function to display a DLL. Ans: display() in 11.3.
5. [A] Explain a circular doubly linked list and write insertion and deletion. Ans: 11.5.
6. [A] Differentiate singly and doubly linked lists. Ans: 11.6.
7. [A] Find the errors in the slide's insert_end for a DLL.

   Ans: 11.3 correction list, points 2 and 3.
8. [A] DLL: 10 <=> 20 <=> 30. Write the statements to delete 20 given a pointer p to it.

   Ans: p->prev->next = p->next; p->next->prev = p->prev; free(p);
9. [A] Why is delete at end O(1) for a DLL with a last pointer but O(n) for a singly list even with a last pointer?

   Ans: after deleting, the new last node must be found. In a DLL it is last->prev; in a singly list we must walk from start to the node before last.

Chapter 12: Applications of Linked Lists

Sources: *pptx slides 82-85 ("Linked List Application", "Represent Polynomial using Linked List", "Polynomial Addition"); Module 2 outline item "Implementation of Stack & Queue Data Structures using Linked List Approach". Lecture outcome (slide 82): "Analyze problem requirements to select the appropriate type of linked list and implement an efficient solution using C."*

12.1 Stack and Queue Using Linked List

[SOURCE] **Course outcome:** "Construct stack and queue operations using linked list implementations." These are covered in full earlier:

- Linked stack - Chapter 2, sections 2.9 - 2.10 (push/pop at the front).
- Linked queue - Chapter 4, sections 4.7 - 4.8 (insert at rear, delete at front, with front and rear pointers).
- Linked priority queue - Chapter 5, section 5.8.

Why a linked list suits them:

ADT	Which end of the list	Why
Stack	Insert and delete at the FRONT (START = TOP)	Both are O(1) at the head of a singly list
Queue	Insert at the END (rear ptr), delete at the FRONT	Insert at end is O(1) with a rear pointer; delete at front is O(1). (Deleting at the end would be O(n).)

No fixed size, so no overflow except when memory runs out.

12.2 Polynomial Representation Using Linked List

[SOURCE]

```
typedef struct link
{
    int coeff;           /* coefficient */
    int pow;             /* power (exponent) of x */
    struct link *next;
} my_poly;
```

Each node stores one TERM. Terms are kept in DECREASING order of power.

```
P1 = 2x^3 + 5x^2 + 7
   start1 -> [2|3| ]-> [5|2| ]-> [7|0|X]
             coeff pow

P2 = 5x^3 + 6x + 2
   start2 -> [5|3| ]-> [6|1| ]-> [2|0|X]
```

Why a linked list: a polynomial like 4x^100 + 1 would waste 99 array slots; the list stores only the non-zero terms, and terms can be added or removed easily.

12.3 Polynomial Addition

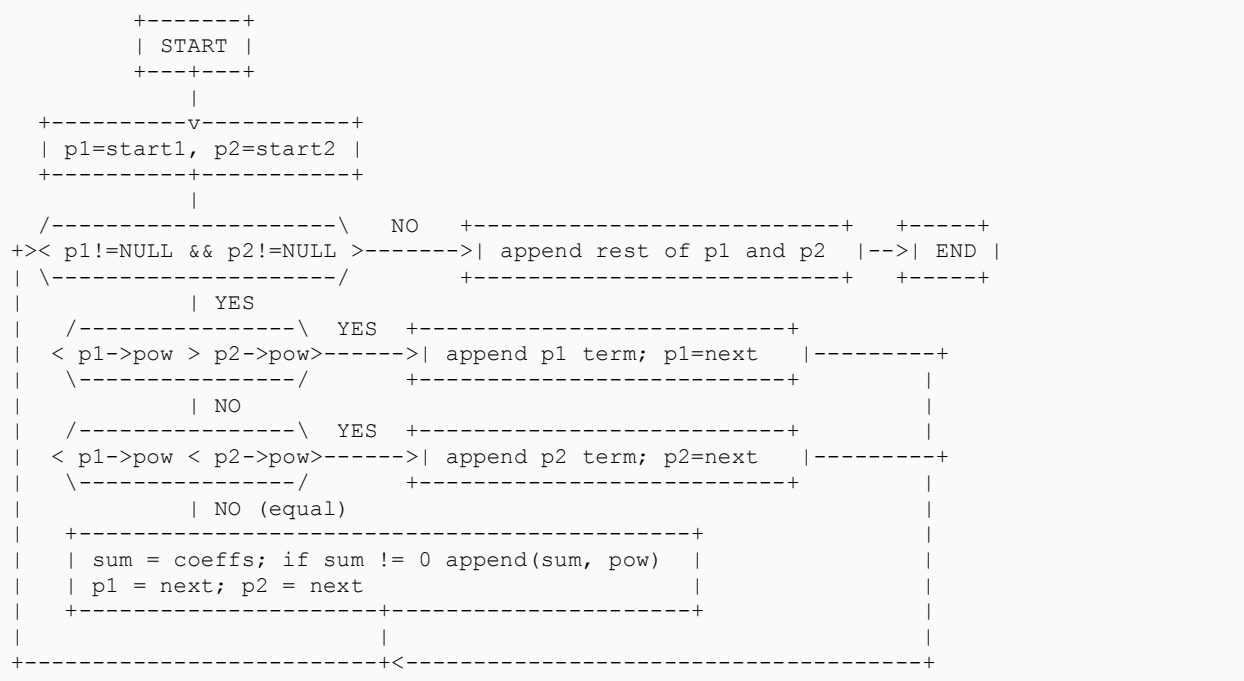
[SOURCE figure, slide 85]

```
start1 -> [2|3] -> [5|2] -> [7|0] -> NULL
   +
start2 -> [5|3] -> [6|1] -> [2|0] -> NULL
   =
start3 -> [7|3] -> [5|2] -> [6|1] -> [9|0] -> NULL
i.e. (2x^3 + 5x^2 + 7) + (5x^3 + 6x + 2) = 7x^3 + 5x^2 + 6x + 9
```

Algorithm [SUPPLEMENT - the slide shows only the picture]:

```
Input : P1, P2 sorted in decreasing powers. Output: P3 = P1 + P2.
Step 1: Set p1 = start1, p2 = start2, P3 empty
Step 2: Repeat while p1 != NULL and p2 != NULL
          IF p1->pow > p2->pow: append (p1->coeff, p1->pow); p1 = next
          ELSE IF p1->pow < p2->pow: append (p2->coeff, p2->pow); p2 = next
          ELSE (equal powers): sum = p1->coeff + p2->coeff
               IF sum != 0, append (sum, p1->pow)
               p1 = p1->next ; p2 = p2->next
Step 3: Append all remaining terms of p1 (if any)
Step 4: Append all remaining terms of p2 (if any)
Step 5: END
```

Flowchart [SUPPLEMENT]:



Dry run (slide example):

Step	p1 term	p2 term	Compare	P3 so far
1	2x^3	5x^3	equal	7x^3
2	5x^2	6x^1	2 > 1	7x^3 + 5x^2
3	7x^0	6x^1	0 < 1	7x^3 + 5x^2 + 6x
4	7x^0	2x^0	equal	7x^3 + 5x^2 + 6x + 9
end	NULL	NULL	-	nothing left

12.4 Program: Polynomial Addition [SUPPLEMENT]

Program: poly.c

```
/* Polynomial representation and addition using linked lists
   Terms are stored in decreasing order of power */
#include <stdio.h>
#include <stdlib.h>

typedef struct link
{
    int coeff;
    int pow;
    struct link *next;
} my_poly;
```

```
/* Append a term after *tail (or as the first node if the list is empty).
   Keeping a tail pointer makes each append O(1). */
void appendTerm(my_poly **start, my_poly **tail, int coeff, int pow)
{
    my_poly *newTerm = (my_poly *)malloc(sizeof(my_poly));
    if (newTerm == NULL)
        return;
    newTerm->coeff = coeff;
    newTerm->pow = pow;
    newTerm->next = NULL;
    if (*start == NULL)
        *start = newTerm;
    else
        (*tail)->next = newTerm;
    *tail = newTerm;
}

my_poly *addPoly(my_poly *p1, my_poly *p2)
{
    my_poly *result = NULL, *tail = NULL;
    while (p1 != NULL && p2 != NULL)
    {
        if (p1->pow > p2->pow)
        {
            appendTerm(&result, &tail, p1->coeff, p1->pow);
            p1 = p1->next;
        }
        else if (p1->pow < p2->pow)
        {
            appendTerm(&result, &tail, p2->coeff, p2->pow);
            p2 = p2->next;
        }
        /* equal powers: add coefficients */
        else
        {
            if (p1->coeff + p2->coeff != 0)
                appendTerm(&result, &tail, p1->coeff + p2->coeff, p1->pow);
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    while (p1 != NULL)                          /* copy leftover terms */
    {
        appendTerm(&result, &tail, p1->coeff, p1->pow);
        p1 = p1->next;
    }
    while (p2 != NULL)
    {
        appendTerm(&result, &tail, p2->coeff, p2->pow);
        p2 = p2->next;
    }
    return result;
}

void display(my_poly *p)
{
    while (p != NULL)
    {
        if (p->pow == 0)
            printf("%d", p->coeff);
        else
            printf("%dx^%d", p->coeff, p->pow);
        p = p->next;
        if (p != NULL)
            printf(" + ");
    }
    printf("\n");
}

void freePoly(my_poly *p)
{
    my_poly *t;
    while (p != NULL)
    {
```

```
        t = p;
        p = p->next;
        free(t);
    }
}

int main(void)
{
    my_poly *start1 = NULL, *start2 = NULL, *start3;
    my_poly *tail1 = NULL, *tail2 = NULL;

    appendTerm(&start1, &tail1, 2, 3);   /* 2x^3 + 5x^2 + 7 */
    appendTerm(&start1, &tail1, 5, 2);
    appendTerm(&start1, &tail1, 7, 0);

    appendTerm(&start2, &tail2, 5, 3);   /* 5x^3 + 6x + 2 */
    appendTerm(&start2, &tail2, 6, 1);
    appendTerm(&start2, &tail2, 2, 0);

    start3 = addPoly(start1, start2);
    printf("P1     = "); display(start1);
    printf("P2     = "); display(start2);
    printf("P1+P2  = "); display(start3);

    freePoly(start1);
    freePoly(start2);
    freePoly(start3);
    return 0;
}
```

Code explanation:

- appendTerm(&start, &tail, c, p): creates a node and links it after tail. Passing the ADDRESSES of start and tail (my_poly **) lets the function change the caller's pointers. Keeping a tail pointer makes each append O(1).
- addPoly(): the three-way comparison of powers from the algorithm; the two "while" loops copy whatever is left of the longer polynomial.
- A zero sum (e.g. 3x^2 + (-3x^2)) is skipped so P3 has no 0 terms.
- display() prints x^0 terms as just the coefficient.
- freePoly() releases every node of each list.

Actual output:

```
P1     = 2x^3 + 5x^2 + 7
P2     = 5x^3 + 6x^1 + 2
P1+P2  = 7x^3 + 5x^2 + 6x^1 + 9
(matches the slide's result 7,3 -> 5,2 -> 6,1 -> 9,0)
```

Complexity: O(m + n) time for polynomials with m and n terms (each term is visited once); O(m + n) space for the result.

12.5 Music Player / Playlist [SOURCE slide 83, picture only]

The slide shows a music player with a playlist (Song 1, Song 2, Song 3, Song 4) built as a linked list. Explanation [SUPPLEMENT]:

- Each song is a node; "next" moves to the next song.
- A DOUBLY linked list supports both Next and Previous buttons.
- A CIRCULAR (doubly) linked list supports "repeat playlist": after the last song, next goes back to Song 1.
- Songs can be inserted or removed anywhere without shifting.

12.6 Choosing the Right Linked List [SUPPLEMENT, for the slide-82 outcome]

Requirement	Suitable choice
Only add/remove at one end (stack)	Singly linked list (head = top)
Add at one end, remove at the other (queue)	Singly list with front and rear pointers
Move forward and backward	Doubly linked list
Cycle through items repeatedly (round robin, repeat playlist)	Circular (singly or doubly) list
Insert/delete at both ends in O(1) (deque)	Doubly list with start and last, or circular doubly list
Sparse data (polynomial, sparse matrix)	Singly list storing only non-zero terms
Frequent random access by index	Array (not a linked list)

12.7 Chapter 12 Exam Preparation

Quick Revision

- **Stack via list:** push/pop at head. Queue via list: insert at rear, delete at front.
- **Polynomial node:** coeff, pow, next; terms in decreasing power.
- **Addition:** compare powers; larger power copied; equal powers added; leftovers copied. O(m+n).

Practice Questions

1. [S] How is a polynomial represented using a linked list? Represent 2x^3 + 5x^2 + 7. Ans: 12.2.
2. [S] Explain polynomial addition using linked lists with an example.

   Ans: 12.3 diagram, algorithm and dry run.
3. [S] Explain how a stack and a queue are implemented using linked lists.

   Ans: 12.1 + Chapters 2 and 4.
4. [A] Add 4x^4 + 3x^2 - 5 and 2x^3 - 3x^2 + 5. Show the resulting list.

   Ans: x^2 terms cancel (3 - 3 = 0) and constants cancel (-5 + 5 = 0); result 4x^4 + 2x^3 -> [4|4]->[2|3]->NULL.
5. [A] Which linked list type would you use for a browser's back/forward history? Why? Ans: doubly linked list - move in both directions.

Chapter 13: Master Summary Tables for Last-minute Revision

13.1 All Structures at a Glance

Structure	Principle	Insert / Delete at	Empty / Full test
Stack (array)	LIFO	top / top	top=-1 / top=MAX-1
Stack (linked)	LIFO	head / head	top=NULL / malloc fails
Linear queue	FIFO	rear / front	front=-1 or front>rear / rear=MAX-1
Circular queue	FIFO	rear / front (mod SIZE)	count=0 / count=SIZE
Linked queue	FIFO	rear / front	front=NULL
Priority queue	by priority	ordered position / front	f>r / r=MAX-1 (array version)
Deque	both ends	front or rear / front or rear	count=0 / count=SIZE
Singly LL	links	anywhere	start=NULL
Circular LL	links, last->first	anywhere	start=NULL
Doubly LL	prev+next	anywhere	start=NULL

13.2 Master Complexity Table

Operation	Time	Reason
Stack push / pop / peek (array, linked)	O(1)	only TOP changes
Queue enqueue / dequeue (array)	O(1)	index update
Queue enqueue / dequeue (linked, rear)	O(1)	rear/front pointers
Circular queue enqueue / dequeue	O(1)	modulo index update
Deque all four end operations	O(1)	modulo index update
Priority queue insert (sorted array / sorted list)	O(n)	find place / shift
Priority queue delete (sorted)	O(1)	always at front
Infix->postfix, postfix evaluation, parenthesis check	O(n)	one scan, each item pushed/popped once
SLL insert/delete at beginning	O(1)	change START only
SLL insert/delete at end	O(n)	walk to last / 2nd last
SLL search / count / display / reverse	O(n)	one traversal
SLL sort (slide method)	O(n^2)	nested traversal
CLL insert beg/end, delete beg (last p.)	O(1)	last->next is first
CLL delete end	O(n)	find node before last
DLL insert/delete both ends (last p.)	O(1)	prev pointer
Polynomial addition (m, n terms)	O(m+n)	each term once
Recursive factorial(n)	O(n)	n+1 calls

Auxiliary space: O(1) for all list/stack/queue operations above except copy (O(n)), recursion (O(n) call stack) and the expression algorithms (O(n) stack).

13.3 Key Formulas and Conditions

```
Array stack       : empty top == -1 ; full top == MAX-1
Linear queue      : full rear == MAX-1 ; empty front == -1 || front > rear
Circular queue    : rear = (rear+1) % SIZE ; front = (front+1) % SIZE
                    empty count == 0 ; full count == SIZE
Deque backward    : i = (i - 1 + SIZE) % SIZE
Postfix evaluation: opnd2 = pop(); opnd1 = pop(); push(opnd1 op opnd2)
SLL end           : temp->next == NULL
CLL end           : temp->next == start
CDLL last node    : start->prev
malloc / calloc   : (type *)malloc(n * sizeof(type)) ;
                    (type *)calloc(n, sizeof(type))
```

13.4 Exam Answer Writing Tips (general)

- Definition first, then a labelled diagram, then the algorithm/code, then an example or dry run, then complexity.
- For every list/stack/queue function, mention the special cases: empty, one element, full (arrays).
- In "differentiate" questions give at least 5-6 points in a table, with an example row.
- In conversions, show the stack table step by step - the final answer alone usually earns little.
- Label NULL, START/TOP/FRONT/REAR in every diagram.

Appendix A: Source Files Reviewed and Where They Are Covered

31 files were found in the folder (plus .DS_Store system files, ignored). All were read. For PDFs and slides, pages with little or no extractable text were rendered and looked at as images.

MODULE 1 (Ms. Shahzia Sayyad)

File	Pages	Covered in
Introduction to Data Structures.pdf	39	Chapter 1
Stack-1.pdf	17	Chapter 2
Stack-2.pdf	15	2.9-2.10, Chapter 3
Stack-3.pdf	11	Chapter 3
Queue-1.pdf	20	Chapter 4
Queue-2.pdf	14	5.1-5.8
Queue-3.pdf	14	5.9-5.12
Programs/INFIXPOSTFIX.C	-	3.6
Programs/EVALPOST.C	-	3.9
Programs/LinearQueue_Array.c	-	4.6
Programs/CircularQueue.txt	-	5.4
Programs/priorityQueue.docx	-	5.7
Programs/DequeProgramUsingArray.pdf	5	5.10

MODULE 2 (Mr. Atul Kachare)

File	Pages	Covered in
Lecture 2 Importance of DS, Linear vs Nonlinear Structures.pdf	15	Chapter 1
Lecture 3 Pointers & Array vs Linked List.pdf	14	Chapter 6
Lecture 4 Types of Linked List and Linked List Representation.pdf	16	8.1-8.4
Lecture 5 Dynamic Memory Management.pdf	11	Chapter 7
Lecture 6 Singly Link List Ops - 1	11	8.6, 8.10, 8.11, 8.15
Lecture 7 Singly Link List Ops - 2	13	8.7-8.9, 8.12, 8.13, 8.16
Lecture 8 Singly Link List Ops - 3	10	8.14, 8.17, 8.18
Module 2 Linked List.pptx	85 sl.	Chapters 1, 6-12; delete at position, reverse,sort, copy (8.19-8.21) appear only in this deck
Module 2 Linked List1.pptx	85 sl.	duplicate of the above (only slide dates differ)
LL Prog/create List & Display.docx	-	9.1, 9.2
LL Prog/Count Number of nodes.docx	-	9.3
LL Prog/sum and Avg.docx	-	9.4
LL Prog/SEARCH element.doc	-	9.5
LL Prog/REVERSEL.doc	-	9.6
LL Prog/EMP.docx	-	9.8
LL Prog/CIR_SLL.C	-	10.4
LL Prog/Linked QUEUE.C	-	4.8

Lecture PDFs 2-8 are printed extracts of the same pptx deck.

Slides that are empty in the files (shown live in class) and were filled with **[SUPPLEMENT]** content:

- Stack-1 slide 16 "Program to implement Stack using Array" -> 2.8
- Stack-2 slides 9, 11, 12, 14 (conversion exercises left blank) -> 3.3, 3.5, 3.7
- Stack-2 slide 13 "Program to Convert Infix ... " -> INFIXPOSTFIX.C, 3.6
- Queue-1 slide 19 "Program to implement queue using Array" -> 4.6
- Queue-2 slide 11 "Program to implement circular queue" -> 5.4
- Lecture 3 / pptx slide 20 "Call by Value & Call by Reference" (animation, not visible in the PDF) -> 6.4
- pptx slide 66 "Linked List Operations" (embedded video) -> 8.4
- pptx slide 81 circular doubly linked list (outcome only) -> 11.5
- pptx slide 83 music player (image only) -> 12.5
- pptx slide 85 polynomial addition (diagram only, no code) -> 12.3-12.4

Scope note: the notes contain no material on trees or graphs beyond the introductory definitions, and nothing on sorting/searching algorithms beyond the binary-search motivation and linked-list sort/search. This guide keeps to that scope.

Appendix B: Index of Corrections to the Source Notes

#	Where	Problem (see section)
1	Stack-1 push/pop algorithms	no "stop" after OVERFLOW/UNDERFLOW (2.7)
2	Stack-2 infix->postfix algo	says pop "higher" only; must be higher or equal for left-assoc ops (3.4)
3	INFIXPOSTFIX.C	no <ctype.h>; priority() missing return; '^' treated left-assoc; gets() (3.6)
4	EVALPOST.C	no <ctype.h>; gets(); no underflow / divide-by-zero checks (3.9)
5	Stack-3 parenthesis checker	missing GOTO in empty-stack case and missing ELSE "Invalid" (3.10)
6	Stack-3 factorial printf	"%d = %d" with one argument (3.12)
7	Queue-1 linked init code	malloc(sizeof(Node); bracket missing; "Struct"/"If" capitals (4.7)
8	Queue-1 linked dequeue algo	temp never freed; step numbering skips (4.7.2)
9	Linked QUEUE.C	malloc(sizeof(Node*)) - heap overflow; no <stdlib.h>; "Display Stack" (4.8)
10	priorityQueue.docx	missing brace in main; c uninitialised; wrong bubble-sort bounds (5.7)
11	DequeProgramUsingArray.pdf	(front-1)%SIZE gives -1 -> writes queue[-1] (5.10)
12	Lecture 3 pointer table	"return pointer to local variable" is unsafe (6.3)
13	Lecture 5 malloc/calloc examples	cast written without brackets (7.2)
14	Lecture 6-8 del_end	missing closing brace (8.16)
15	Lecture 8 insert_at / del_at	rejects position count()+1; accepts negative positions; "Else`" (8.14,8.19)
16	pptx rev1()	returns uninitialised value on empty list (8.20.2)
17	pptx copy display	%u used for pointers; use %p (8.21.2)
18	REVERSEL.doc	NULL dereference at the end and for 0/1-node lists - crashes (9.6)
19	EMP.docx	heading says 50000, code tests 5000 (9.8)
20	CIR_SLL.C	first/last not updated in main - crash; last node not displayed (10.4)
21	pptx circular list functions	extra bracket in print; local "last"; single-node delete bug (10.2, 10.4)
22	pptx doubly list functions	prev/next not initialised; head vs start; undeclared value; curly quotes (11.3)

Appendix C: Turbo C Style vs Standard C

In your notes (Turbo C)	Standard C (used in this guide)
void main()	int main(void) { ... return 0; }
#include <conio.h> clrscr(); getch(); at the end gets(str);	not available on gcc/clang - omit omit (or system("cls") on Windows only) omit, or getchar(); fgets(str, size, stdin);
malloc without <stdlib.h>	#include <stdlib.h>
isdigit/isalpha without <ctype.h> printf("%u", pointer) int is 2 bytes	#include <ctype.h> printf("%p", (void *)pointer) int is usually 4 bytes

If your examiner expects Turbo C style, writing void main() and getch() is acceptable in that setting; the data-structure logic is what is marked.

— End of document —